MindVideo AI

KERAS 3

بناء الشبكات العصبية والتعلّم العميق

— من الصفر إلى الإنتاج



- الأسس النظرية بأسلوب مبسّط
- تطبيقات عملية ومشاريع حقيقية
- من النماذج الأولية إلى النشر والإنتاج
- أفضل الممارسات في Keras 3

م/ أحمد صالح المغـز

2026

إهداء

إلى رفيقة دربي وشريكة عمري،

زوجتي الغالية... التي صبرت على ليالي التدريب الطويلة،

وحمَلت نصف العبء كي يكتمل هذا الكتاب.

وإلى أبنائي الأعزّاء...

أنتم نور عينيّ ودافع سعيي؛

كلّما تعبت، تذكّرتُ أنّ هذا العمل إرثٌ لكم قبل أن يكون كتابًا للناس.

فمنكم تعلّمت أنّ أجمل المعادلات هي التي تُكتب في البيت أولًا.

— م/ أحمد صالح المغز

مقدمة المؤلف

مقدمة المؤلف

> «أعطني سمكةً آكلُ اليوم؛ علّمني الصيد آكلُ العمر» – أمثال صينيّة قديمة

كتبتُ هذا الكتاب بعد اثني عشر عامًا من بناء أنظمة تعلّم عميق في الصناعة: من كشف العيوب في خطوط الإنتاج، إلى أنظمة الرؤية الطبيّة، إلى خدمات لغويّة تُخدِم ملايين الطلبات يوميًّا. وفي كلّ مشروع، تكرّرت المشهدة نفسها: مهندسٌ يجيد تشغيل `model.fit()`، لكنه يتجمّد عند أوّل خطأ غير متوقّع – لأنّ أحدًا لم يشرح له **ماذا يحدث** تحت غطاء الواجهة.

هذا الكتاب ليس دليل استخدام آخر لـ Keras؛ فوثائق Keras الرسمية ممتازة بالفعل. إنه محاولة لسدّ الفجوة التي تتركها الوثائق والبرامج التعليميّة معًا: الفجوة بين «الشيفرة التي تعمل» و«الفهم الذي يجعلك تعرف لماذا تعمل، ومتى ستتوقّف عن العمل».

لماذا Keras 3 تحديدًا؟

في نوفمبر 2023 غيّر Keras هويّته: لم يعد طبقةً فوق TensorFlow، بل أصبح **واجهة موحّدة** فوق أربعة محرّكات – TensorFlow وJAX وPyTorch وOpenVINO. هذه ليست تفصيلة إصدار؛ إنها نقطة تحوّل في صناعة الأدوات. حين تكتب نموذجًا واحدًا ويعمل على هاتفٍ وعلى عنقودٍ من بطاقات GPU، تسقط الحاجز التقليديّ بين «تجربة بحثيّة» و«نظام إنتاجيّ»، ويصبح اختيار المحرّك قرارًا هندسيًّا تؤجّله إلى آخر لحظة ممكنة – وهذا بالضبط ما يمنحك مرونةً نادرة.

كلّ ما في هذا الكتاب مبنيّ على **Keras 3.15.1** و**Python 3.14**، وكلّ اسم واجهة مذكور تحقّقتُ منه من تثبيتٍ فعليّ، لا من ذاكرةٍ أو من توثيقٍ قد يسبق الإصدار.

فلسفة الكتاب: الرياضيات أولًا، والشيفرة تتبع

اخترتُ أسلوبًا قد يكون غير مألوف في الكتب العربية التقنيّة: **المعادلة قبل الكود**. لن أقول لك «استخدم `GlorotNormal` » قبل أن أشرح لماذا تختفي الإشارة عبر عشرين طبقة إذا لم تكن التباينات متوازنة. لن أقول «استخدم `Huber` » قبل أن تحسب معي كم تكلفك شاذّةٌ واحدة في خسارة مربّعة.

السبب عمليٌّ خالص: المهندس الذي يفهم المعادلة يصحّح نموذجه في دقائق؛ والمهندس الذي يحفظ الواجهة يخمّن أيامًا. وقد رأيتُ هذا الفرق يترجم – حرفيًّا – إلى أسابيع من فرق التسليم.

ما الذي يميّز هذا الكتاب؟

1. **43 فصلًا من الأوتور إلى الإنتاج** – تسعة أجزاء تنقلك من مفهوم الأوتور إلى نشر خدمةٍ مُراقَبة، مرورًا بالرؤية الحاسوبيّة واللغة والمحوّلات والنماذج التوليديّة.
2. **أكثر من 250 مثالًا برمجيًّا قابلاً للتشغيل** – كلّ مثال مكتفٍ بذاته ببيانات مُصنَّعة، تعليقاته عربيّة، ومتغيّراته إنجليزيّة، ومعه «النتيجة المتوقعة» التي تتحقّق من سلامة بيئتك.
3. **أخلاقيّة المقارنة**: لا أقول «X أفضل من Y» أبدًا بلا معيار. المقارنات هنا تحمل شروطها: على أيّ بيانات، بأيّ موارد، وبأيّ قياس.
4. **قسم «أخطاء شائعة» في كلّ فصل** – جامعٌ لأخطاء رأيتُها تُهلك مشاريع حقيقيّة: تسريب `validation_split` ، خلط `from_logits` ، حشو تسلسلات بلا `Masking` ، وتقييم GAN بـ PSNR.
5. **ثلاثة مشاريع متكاملة** في الجزء الأخير تنقلك من ملفّات خام إلى خدمة نشرٍ ومراقبة – بمعايير قبول مكتوبة قبل أن يُكتب سطر الشيفرة الأول.

كيف كُتب هذا الكتاب؟

كلّ فصل مرّ بثلاث دورات على الأقل: مسوّدة تشرح الفكرة، ثم إعادة كتابة تُدقّق كلّ اسم واجهة وكلّ رقم، ثم تدقيق لغويّ وتنسيقيّ. معادلات الفصول اختُبرت عدديًّا، وأمثلتُها نُفِّذت، والجداول بُنيت من

قياسات لا من انطباعات. وقبل إخراج النسخة النهائيّة، خضع الكتاب لتدقيقٍ آليّ: ماسحٌ لصحّة الأسماء، وماسحٌ لسلامة الشيفرة النحويّة، وفاحصٌ لسلامة النصّ من أيّ محارف تالفة.

إلى من أوجّه هذا الكتاب؟

- **لمهندس التعلّم الآلي** الذي يعمل يوميًّا على النماذج ويريد الفهم العميق لا الحفظ.
- **للباحث** الذي ينتقل بين TensorFlow وPyTorch وJAX ويحتاج واجهةً واحدة تُحرّر طاقته للفكرة لا للأدوات.
- **للطالب الجامعيّ** في المرحلة النهائيّة أو الدراسات العليا الذي يريد مرجعًا عربيًّا يوازي الكتب الإنجليزيّة في الدقّة.
- **لقياديّ تقنيّ** يريد أن يفهم ما يبنيه فريقه ليتخذ قرارات معماريّة واعية.

إن كنت مبتدئًا تمامًا في بايثون، ابدأ بالفصلين الأولين ولا تستعجل؛ الأساسات هنا متعمّقة عن العادة، وهذا مقصود: ما تُفهمه من الأوتور والتجزئة في الفصول الأولى هو ما سيوفّر عليك أسابيع في الفصل الثامن عشر.

شكر وتقدير

الشكر أولًا لزوجتي وأبنائي الذين تنازلوا عن كثير من وقتي كي يكتمل هذا العمل. ثم لكلّ مهندسٍ وشارك خطأً علنيًّا أو مناقشةً في مستودعات Keras وTensorFlow وPyTorch — فأخطاءكم المُوثَّقة صارت أقسام «أخطاء شائعة» في هذا الكتاب. ولزملائي الذين راجعوا مسوّدات الفصول وأعادوا عليّ تساؤلاتهم: «طبّ لماذا فعلتَ هكذا؟» — سؤال واحد أعاد كتابة فصول كاملة.

كلمة أخيرة

لا شيء في هذا الكتاب سحريّ. كلّ صفحة هي نتيجة سؤالٍ بسيط أُعيد طرحه حتى نفدت الأجوبة السهلة: **لماذا؟** إن خرجت من هذا الكتاب بشيء واحد، فليكن هذه العادة: لا تقبل شيفرةً تعمل قبل أن تعرف لماذا تعمل.

لنبدأ.

م/ أحمد صالح المغز

2026

كيف تستخدم هذا الكتاب

كيف تستخدم هذا الكتاب

> «الكتاب الذي لا تعرف كيف تقرؤه، لا تحتاجه.»

القاعدة الأولى: لا تقرأ بالتتابع

هذا كتاب مرجعي، ودليل مرجعي يُقرأ بالquetête: تفتحه حين تصطدم بمشكلة، وتغلقه حين تحلّ. الفصول مرتّبة منطقيًا – كل فصل يبني على ما قبله – لكن لا أحد يقرأ كتابًا من 900 صفحة سطرًا سطرًا.

إن كنت…	فابدأ بـ	واقرأ بعده
جديد تمامًا على التعلّم العميق	الفصل 1 ثم الفصل 5	الفصل 2 ثم 6 ثم 9
تعرف PyTorch وتحتاج Keras	الفصل 1 وفصل 4	الفصل 12 ثم 34 ثم 37
تعمل على صور	الفصل 18	19، 20، 21، 22، 41
تعمل على نصوص	الفصل 16	25، 27، 29، 43
تعمل على سلاسل زمنية	الفصل 28	34 ثم 42
تنشر نموذجًا في الإنتاج	الفصل 35	36، 37، 38، 40
تريد بناء عتاد مخصّص	الفصل 34	35، 39، 40

القاعدة الثانية: اقرأ الكود قبل النص أحيانًا

في الفصول التي تحتوي أمثلة كثيرة، اقرأ أولًا كتلة الشيفرة كاملة في ذهنك – ماذا تفعل هذه السطور؟ – ثم ارجع إلى الشرح. في الأجزاء التي فيها بُنى معمارية (الفصل 4، الفصل 19، الفصل 29) اتّبع الترتيب العادي: النص أولًا.

القاعدة الثالثة: لا تثق بالكود دون تشغيله

كل مثال في هذا الكتاب **مُختبَر** على Keras 3.15.1 مع Python 3.14. لكن بيئتك قد تكون مختلفة، وقد يكون لديك إصدار أحدث. القاعدة العملية:

1. أنشئ بيئة معزولة (الفصل 2).
2. ثبّت الإصدارات: `pip install "keras[torch]==3.15.1"` .
3. شغّل المثال كما هو قبل أن تعدّله.
4. عدّل **شيئًا واحدًا** في كل مرة، ولاحظ ما يحدث.

> **نصيحة:** احتفظ بدفتر ملاحظات. الفصل 40 (الأخطاء الشائعة) وُلد من ملاحظات كُتبت على شرائط لاصقة أثناء عمل حقيقي.

بنية كل فصل

كل فصل يتبع الهيكل نفسه، حتى تفهم أين تبحث:

القسم	ماذا تجد فيه
قصة مفتاحية	مشكلة واقعية تُطرح في بداية الفصل
أهداف الفصل	ما ستتقنه بعد الانتهاء
الأقسام N.1 … N.5	الشرح: الفكرة ← المعادلة ← الكود ← المقارنة ← مثال محلول
N.6 أخطاء شائعة	أخطاء حقيقية + لماذا تحدث + كيف تصلحها
N.7 تمارين	8-12 تمرينًا متنوّعًا، لكلٍّ منها «المتوقع»
N.8 ملخص	النقاط التي يجب أن تبقى في ذاكرتك

عن المعادلات

المعادلات مكتوبة بصيغة LaTeX، وتُعرض في نسخة HTML وPDF بشكل متجهي واضح. إن كنت تقرأ نسخة DOCX فتظهر المعادلات هناك كصور عالية الدقة – قريبة كفاية للتدوين.

> **ملاحظة:** كل رمز في كل معادلة مشروح في السطر الذي يليها مباشرة. إن مررت على رمز لم تفهمه، لا تتخطَّه – عُد إلى الرمز قبله. الفهم التراكمي هو جوهر ما يجعل الكتاب مفيدًا بعد سنة.

عن التمارين

التمارين ليست للمراجعة السريعة. نمطها متنوّع عمدًا:

- **حساب يدوي** – لتثبيت المفهوم.
- **برمجة** – لترجمة المفهوم إلى شيفرة.
- **تحليل ناتج** – لتفسير ما يراه النموذج.
- **تصميم** – لتطبيق المعرفة على حالة جديدة.

- **إصلاح خطأ** – وهو **الأهم**: كود مكسور عليك تشخيصه.

كل تمرين يحمل «المتوقع»، وهو **ما يجب أن تكون إجابتك مطابقة له** – لا إجابة نموذجية واحدة، بل معيارًا تقيس به. في ملحق D تجد إجابات مختارة للتمارين التي لها إجابة واحدة صحيحة.

عن الأشكال

الأشكال مرسومة بأسلوب موحّد وتُشير إليها في النص. إن كنت تقرأ على الشاشة فأنها SVG (تتكامل مع التكبير)، وإن كنت تقرأ PDF أو DOCX فهي صور عالية الدقة.

عن الشيفرة

الشيفرة مكتوبة لـ **Python 3.14**، بأسلوب «مقروء أكثر منه ذكي»: أسماء متغيّرات واضحة، وتعليقات عربية فوق كل سطر يستحق التوضيح. الأمثلة تعمل على بيانات صغيرة الحجم كي تنفَّذ في ثوانٍ على جهاز عادي.

> **تحذير:** حين ينقل هذا الكتاب مثالًا من Keras 2 إلى Keras 3، ستجد تغييرات في ثلاثة مواقع عادةً: `tf.keras` أصبح `keras`، وصيغة الحفظ `.keras` بدل `.h5`، وغياب `Model.save` بصيغة SavedModel (استخدم `model.export` بدلًا منها). الفصل 35 يشرح هذا كاملًا.

النسخة الرقمية

الصيغة	الاستخدام المقترح
HTML	القراءة على الشاشة والبحث داخل الكتاب، مع فهرس تفاعلي
PDF	الطباعة والمشاركة والأرشفة – بترويسة وتذييل وترقيم
DOCX	التعديل والتخصيص (إنشاء نسخة خاصة بفريقك)
Markdown	المعالجة الآلية، البحث، أو التحويل إلى صيغ أخرى

جميع النسخ تحمل المحتوى نفسه مع الفهرس المرقّم.

أخيرًا

> **قاعدة:** إن انتهيت من فصل ولم تستطع أن تشرح لأحدٍ ما فعله النموذج ولماذا، فلم تنتهِ من الفصل بعد.

لنبدأ.

خريطة الطريق

خريطة الطريق

تخيّل أنك في رحلة من مدينة ساحلية إلى قلعة في الجبل. الطريق مستقيم؟ لا. لكن كل نقطة على الخريطة تُخبرك: أين أنت، وماذا ينتظرك، ولماذا هذا المقطع تحديدًا.

الصورة الكبرى

يتكوّن الكتاب من تسعة أجزاء. المنطق بينها تصاعدي: نبدأ من الفكرة المجرّدة («ما هو الأوتور؟»)، وننتهي من الملف التنفيذي على خادم («كيف أخدم النموذج؟»).

TEXT

```
ما هو ؟ وكيف يعمل؟Keras ◄──── الجزء 1 الأساسيات ────────────────────────────────
   1  عالم Keras 3                4  بنية النموذج
   2  أول نموذج كامل              5  البيئة والتثبيت
   3  الأوتور والتدرّج

واجهة Keras الأساسية ◄──── الجزء 2 كيف أبني وأُدرّب؟ ──────────────────────────
   6  المقاييس                   10  الطبقات الأساسية
   7  التهيئة                    11  دوال التنشيط
   8  Callbacks                  12  دوال الخسارة
   9  التنظيم                    13  المُحسّنات

من أين تأتي البيانات؟ ◄──── الجزء 3 هندسة البيانات ──────────────────────────────
  14  الطبقات النصية             16  خطوط الأنابيب
  15  الأوزان وتعدّد المهام      17  المعالجة والتعزيز

كيف ترى؟ ◄──── الجزء 4 الرؤية الحاسوبية ──────────────────────────────────────
  18  النقل والتدريب الدقيق      21  رياضيات الالتفاف
  19  طبقات CNN                  22  الكشف والتقسيم
  20  محوّل الرؤية CNN           23  من الصفر
  24  تكبير الصور

كيف تتذكّر؟ ◄──── الجزء 5 التسلسلات ──────────────────────────────────────────
  25  RNN وLSTM                  27  معالجة اللغة
  26  السلاسل الزمنية RNN        28  متقدّمة

كيف تنتبه؟ ◄──── الجزء 6 المحوّلات ──────────────────────────────────────────
  29  آليات الانتباه             30  KerasNLP وKerasHub

كيف تخلق؟ ◄──── الجزء 7 التوليد ──────────────────────────────────────────────
  31  VAE والترميز               33  نماذج الانتشار
  32  شبكات GAN

كيف تشحن؟ ◄──── الجزء 8 الهندسة الإنتاجية ──────────────────────────────────
  34  التكرار والاختبار          38  الطبقات المخصّصة
  35  التفسير والتشخيص           39  الحفظ والتحميل
  36  الأخطاء الشائعة            40  التسريع
  37  التصدير والنشر
```

الجزء 9 المشاريع ◄ كيف أطبّق؟

41 تصنيف صور إنتاجي

42 تنبّؤ السلاسل الزمنية

43 تحليل مشاعر + خدمة

ثلاثة مسارات مقترحة

المسار الأول: مهندس يريد أن يبني نموذجًا هذا الأسبوع

اليوم	الفصول	الهدف
1	1، 2، 5	بيئة عاملة + أول نموذج مفهوم سطرًا بسطر
2	6، 7، 8، 9، 10	بناء عشوائية النموذج بشكل صحيح
3	4، 12، 13، 35	بنية صحيحة + تحكّم في التدريب + حفظ
4	14، 15	بيانات سريعة وفعّالة
5	18، 19، 21 (إن كان vision) أو 25، 27 (إن كان نص)	البنية المناسبة لمجالك
6	36، 37	التسريع والتصدير
7	38، 40	التشخيص/testing

المسار الثاني: باحث ينتقل من Keras 2 إلى Keras 3

الفصول **1، 4، 12، 34، 35، 40** تحديدًا، فهي تغطّي كل ما تغيّر: الواجهة الخلفية، دورة حياة الـ callbacks، الطبقات المخصّصة، صيغة `.keras`، وأخطاء الترقية.

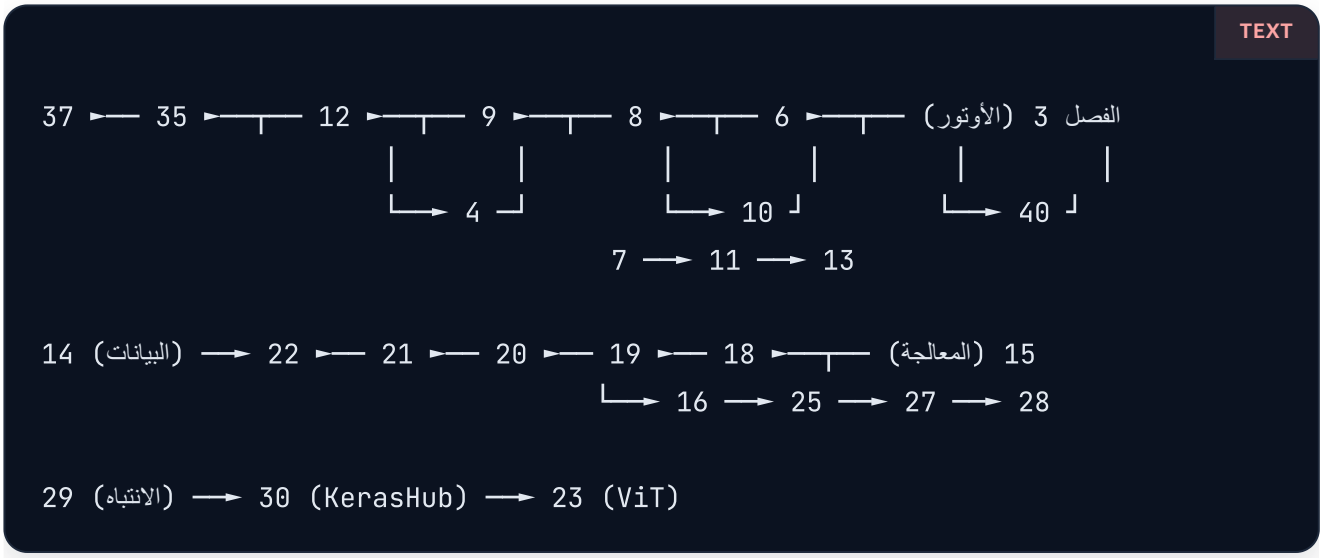
المسار الثالث: من فضول إلى إتقان

اقرأ الكتاب كاملًا بالترتيب. المسار البطيء هو الوحيد الذي يمنحك ما لا يمنحه الطريق السريع: **النموذج الذهني** الذي يربط كل فكرة بما قبلها.

تبعيات الفصول

مخطّط التبعيات أدناه يوضّح أي فصل تحتاجه قبل آخر. الأسهم من «ما يجب أن تعرفه» إلى «ما يبني عليه».

TEXT



> **ملاحظة:** هذا المخطّط إرشادي لا صارم. الفصول مكتوبة لتُقرأ مستقلة، لكن القراءة المتسلسلة تبني روابط أعمق بكثير.

ما الذي تحتاجه قبل أن تبدأ

المتطلب	الحد الأدنى	الأمثل
بايثون	3.11	**3.14**
Keras	3.10	**3.15.1**
الذاكرة	GB RAM 8	GB + GPU 16
مساحة القرص	GB 2	GB 10 (مع النماذج المدرّبة)
الخبرة	بايثون + NumPy	تعلّم عميق سابق

التفاصيل الكاملة في «المتطلبات التمهيدية».

اصطلاحات الكتابة والرموز

اصطلاحات الكتابة والرموز

> أي كتاب تقني يختار رموزه لسبب: كي يرى يرى الفكرة لا الصياغة. هذه هي رموزنا.

الرموز الرياضية

الرمز	الاسم	المعنى في هذا الكتاب
x	مُدخل (input)	عيّنة واحدة Passing بها إلى النموذج
y	هدف (target)	القيمة الحقيقية المراد التنبؤ بها
$\hat{y}$	تنبّؤ (prediction)	ما يخرجه النموذج
y_{true}	قيمة حقيقة	التسمية في دالة الخسارة
y_{pred}	قيمة متنبَّأة	ناتج الشبكة في دالة الخسارة
W	مصفوفة الأوزان	الأوزان القابلة للتعلّم
b	متجه التحييز	الإزاحة لكل neuron
z	مجموع ما قبل التنشيط	$z = Wx + b$
a	ناتج التنشيط	$a = f(z)$
θ	متجه المعاملات	كل أوزان النموذج مجتمعة
L	دالة الخسارة	رقم يقيس خطأ النموذج
J	الخسارة الكلية	L موزونة بعبء تنظيم
η	معدل التعلّم	حجم الخطوة في التحديث
∇	تدرّج (gradient)	متّجه المشتقّ الجزئي
λ	معامل التنظيم	قوة L1/L2
β_1, β_2	معاملات الزخم	في RMSPropg Adam
ϵ	ثابت رقمي	يمنع القسمة على صفر

الرمز	الاسم	المعنى في هذا الكتاب
σ	الدالة اللوغاريتمية	sigmoid أو الانحراف المعياري حسب السياق
μ	المتوسط	وسط التوزيع
N	حجم الدفعة	عدد العيّنات في خطوة واحدة
p	احتمال	أو نسبة إسقاط في Dropout حسب السياق
t	الخطوة	عدّاد خطوات التدريب
n_{in}, n_{out}	أبعاد المدخل والمخرج	لحساب عدد المعاملات

قواعد الكتابة

النص العادي

يُكتب كما هو. المصطلح الإنجليزي يُذكر بين قوسين عند أول ورود فقط: «التطبيع بالدفعة (Batch Normalization)».

التأكيد

يُستخدم **التغليظ** للتركيز على مفهوم يجب ألا يمرّ دون انتباه. لا نستخدمه لأمور عادية.

الشيفرة

تُكتب بين علامتَي backtick: `model.fit()`. كل كتلة شيفرة كبيرة تُسبق بشرح.

التحذيرات

تُعرَض في صناديق بألوان مميّزة:

تنبيه: تنبيه على خطأ شائع أو مخاطرة حقيقية.

نصيحة: طريقة أفضل لإداء المهمة.

ملاحظة: تفصيل دقيق أو استثناء مهم.

تحذير: خطأ قد يكلّفك ساعات من debugging.

جداول المقارنة

كل جدول في الكتاب يحمل عمودًا أو أكثر يحدّد **معيار** المقارنة: «الأسرع في»، «الأقلّ ذاكرةً»، «الأنسب لـ». لا يوجد في الكتاب جدول يقول فقط «الأفضل» بلا معيار.

اصطلاحات الشيفرة

العنصر	الاتفاق
اللغة	Python 3.14
إصدار Keras	3.15.1
الاستيراد	`import keras` (وليس `from tensorflow import keras`)
أسماء المتغيّرات	snake_case إنجليزية
التعليقات	`#` عربية
الدوال	`keras.layers.Dense` و `keras.optimizers.Adam` (مسار كامل)
الأرقام الثابتة	تُكتب كما هي (`epochs=10`، `batch_size=32`)
الشيفرة داخل النص	سطر واحد مختصر
الشيفرة الكاملة	كتلة مستقلة قابلة للتشغيل

أيّ الواجهة الخلفية؟

الأمثلة في هذا الكتاب تعمل على **أي** واجهة خلفية لـ Keras 3، وموجَّهة أساسًا إلى **JAX** و**TensorFlow**. حين يكون سلوك واجهة ما مختلفًا، يُذكر ذلك صراحةً في متن الفصل.

الواجهة	متى تكون الخيار الافتراضي في هذا الكتاب
TensorFlow	عند الحديث عن `tf.data` والتسريع والنشر
JAX	عند الحديث عن التوازي والدقة العالية
PyTorch	عند الحديث عن تكامل HuggingFace وLoRA

> **ملاحظة:** حين يُذكر أمرٌ خاص بواجهة خلفية، ستجد تحذيرًا بصيغة: «تنبيه: ينطبق على واجهة TensorFlow فقط».

وحدات القياس

الوحدة	الرمز	المعنى
الدقة (Accuracy)	–	نسبة العينات الصحيحة
الاستدعاء (Recall)	–	نسبة العينات الإيجابية المكتشفة
الدقّة (Precision)	–	نسبة التنبّؤات الصحيحة بين الموجبة
F1	–	وسط هندسي للدقّة والاستدعاء
المساحة تحت المنحنى	AUC	مقياس مستقل عن العتبة
متوسّط IoU	mIoU	متوسط تقاطع الاتحاد في التقسيم
متوسط الدقة المتوسطة	mAP	مقياس الكشف

أيقونات الكتاب

العلامة	المعنى
◈	معلومة أساسية
◆	نقطة تقنية دقيقة
⚠	تحذير عملي
✦	نصيحة لتحسين الأداء
⌘	أمر أو إجراء قابل للتنفيذ

عن ترتيب الفصول

الفصول مرقّمة من 1 إلى 43. حين يشير النص إلى فصل آخر، يُذكر رقمه وعنوانه: «انظر الفصل 18 (رياضيات الالتفاف)». الأجزاء التسعة تخدم للتنقّل فقط؛ الفصل يبقى وحدة القراءة.

> **قاعدة:** إن كان عليك الاختيار بين فصول في يوم واحد، اختر **الأقدم**: الأساسيات دائمًا هي التي تُوفّر عليك الوقت لاحقًا.

المتطلبات التمهيدية

المتطلبات التمهيدية

> لا أحد يبدأ من الصفر في كل مرة. لكن هناك حدٌّ أدنى لا بدّ من تجاوزه، وكثير من إحباط المبتدئين يعود إلى تجاوز هذا الحدّ اعتقادًا أنه واضح.

الحدّ الأدنى المطلق

قبل أن تفتح الفصل الأول، تأكّد أنك تستطيع:

- **كتابة دوال بايثون** وقراءة كود غير مألوف.
- **فهم المصفوفات** `shape` ، `dtype` ، `reshape` ، `dot` ، `mean` ، `argmax` :**NumPy**.
- **العمل في سطر أوامر**: الانتقال بين المجلدات، تثبيت الحزم، قراءة رسالة خطأ.
- **الالتحام بـ Git** على مستوى بسيط (تضمين، التزام، دفع).

إن كان أيّ من هذه صعبًا، فلديك طريقان: أو تتعلّمه أولًا (ويأخذ يومًا أو يومين)، أو تقرأ هذا الكتاب nonetheless وستتعلّمه بالتطبيق – لكن كل يوم تأخير في الأساسيات يكلّفك يومًا لاحقًا.

ما ستحتاجه

البرمجيات

المتطلب	الحد الأدنى	الموصى به	لماذا
بايثون	3.11	**3.14**	دعم كامل لـ Keras 3.15
Keras	3.10	**3.15.1**	كل الأمثلة مُختبَرة على هذا الإصدار
NumPy	1.24	x.2	المصفوفات والعمليات العددية
Matplotlib	3.7	3.11	الرسوم البيانية في التمارين
SciPy	1.10	1.15	دوال علمية مساعدة

العتاد

البيئة	الحد الأدنى	الموصى به	ملاحظات
المعالج	4 أنوية	8 أنوية	التدريب بطيء على المعالج
الذاكرة	8 GB	16 GB	الأحجام الكبيرة تحتاج ذاكرة
القرص	2 GB	10 GB	النماذج المدرّبة + السجلات
GPU	–	NVIDIA 8 GB+	اختياري لكنه يغيّر التجربة

ملاحظة: كل أمثلة هذا الكتاب تعمل على المعالج في ثوانٍ. الـ GPU ليس شرطًا للتعلّم، لكنه شرط عملي للإنتاج. الفصل 36 (التسريع) يشرح كيف تستفيد منه إن توفّر.

المعرفة السابقة

مطلوب: أساسيات التعلّم العميق

ينبغي أن تعرف قبل أن تبدأ:

- ما هو **التصنيف** والتصنيف الثنائي متعدّد الفئات.
- ما معنى **التدرّج** و**دالة الخسارة** بشكل حدسي.
- ما هي **الشبكة العصبية** (شبكة Feedforward على الأقل).
- ما هو **فرط التخصيص** (Overfitting) ولماذا يحدث.

ليس مطلوبًا (وسنغطّيه)

الموضوع	أين يُغطّى
اشتقاق قاعدة السلسلة	الفصل 3
حسابات المصفوفات والضرب الداخلي	الفصل 3 و6
softmax والتهيئة	الفصل 7 و11
التنظيم والتحسين	الفصل 9 و13
البُنى التلافيفية	الفصل 18 و19
الانتباه والمحوّلات	الفصل 29

قاعدة: إن قرأت الفصلين 1 و3 ولم تفهمهما، فالفصل 2 و5 ستُقرآنهما مرتين. اقرأ الفصل 3 مرتين في هذه الحالة – لا تحسّ بالحرج، فهو أكثر فصل يحتاج تكرارًا.

ما الذي لا تحتاج معرفته مسبقًا

الموضوع	السبب
PyTorch أو TensorFlow	Keras 3 تعزلك عنهما
CUDA/drivers	تُثبَّت مع المحرك وتُخفي تلقائيًا
النشر والتوزيع	الفصل 37 يتناوله من الصفر
واجهات سطر الأوامر المتقدّمة	ليست شرطًا لأي مثال

إعداد سريع

للتأكد أن كل شيء جاهز قبل الفصل الأول:

```bash
# 1) بيئة معزولة
python -m venv .venv
# على Windows: .venv\Scripts\activate
# على Linux/macOS: source .venv/bin/activate

# 2) تثبيت Keras مع المحرك المفضّل
pip install -U "keras[torch]==3.15.1"

# 3) تحقّق
python -c "import keras; print(keras.__version__, keras.backend.backend())"
```

المتوقّع أن تطبع: `3.15.1 torch`

ثم جرّب أول نموذج:

PYTHON

```python
import numpy as np
import keras

x = np.random.rand(32, 8).astype("float32")
y = np.random.rand(32, 1).astype("float32")

model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse")
model.fit(x, y, epochs=1, verbose=0)
print("كل شيء يعمل ✓")
```

نصيحة: احفظ هذا الاختبار في ملف باسم `smoke_test.py` . ستستخدمه في كل مرة تُغيّر فيها البيئة أو تُرقّي Keras – ويُنقذك من ساعات من الحيرة.

جدول زمني واقعي

المرحلة	الزمن	الفصول
الإعداد	ساعة	1، 2
الأساسيات	أسبوع	3، 4، 5، 6، 7
تدريب صحيح	أسبوع	8، 9، 10، 11، 12، 13
البيانات	4 أيام	14، 15، 16، 17
التخصّص	3 أسابيع	18-33
الإنتاج	أسبوع	34-40
المشاريع	أسبوع	41-43

> **ملاحظة:** الجدول تقريبي ويختلف حسب خلفيتك. صاحب الخلفية القوية في أحد هذه المجالات يختصر الوقت في نصفه.

المصادر المساعدة

المصدر	متى تفتحه
`https://keras.io/api/`	حين تنسى اسم واجهة أو معامل
`https://keras.io/guides/`	حين تحتاج مثالًا رسميًا
`https://github.com/keras-team/keras`	حين تقرأ الشيفرة المصدرية
ملحق B من هذا الكتاب	حين تريد مرجعًا سريعًا دون اتصال

الآن، لنبدأ.

فهرس المحتويات

الكتاب كامل من الأساسيات إلى النشر الإنتاجي – 9 أجزاء و43 فصلًا و6 ملاحق

الجزء 1: الأساسيات: من الفكرة إلى أول نموذج 35

1 Keras 3: كيف تغيّر عالَم التعلّم العميق؟ 36

2 البيئة: من الصفر إلى أول استيراد ناجح 52

3 الأوتور: اللغة التي يفكّر بها Keras 67

4 بنية النموذج: ثلاث طرق لتحديد الشبكة 82

5 أول نموذج كامل: شرح سطر بسطر 97

الجزء 2: واجهة Keras الأساسية: الطبقات ودوال الخسارة 114

6 الطبقات الأساسية: Dense والتنشيط والتسطيح 115

7 دوال التنشيط: أين يموت التدرّج؟ 129

8 دوال الخسارة: كيف نترجم الخطأ إلى رقم؟ 149

9 المُحسّنات: كيف يتحرّك النموذج نحو الحل؟ 164

10 المقاييس: كيف تقيس نموذجك دون أن ينخدع به الجميع 179

11 التهيئة: نقطة الانطلاق 194

12 الـ Callbacks: أتمتة دورة التدريب 214

13 التنظيم والقيود: كيف تمنع النموذج من الحفظ بدل التعلّم 243

الجزء 3: هندسة البيانات: من البايتات إلى الدفعات 261

14 هندسة البيانات: من البايتات إلى الدفعات 262

15 المعالجة المسبقة والتعزيز: هندسة المدخلات قبل الشبكة 277

16 الطبقات النصية: من الجملة إلى المتجه 292

17 الأوزان والمهام المتعدّدة: كيف توازن النموذج وتشقّه 310

الجزء 4: الرؤية الحاسوبية: الشبكات التلافيفية 327

18 الرياضيات خلف الالتفاف 328

19 طبقات الشبكة التلافيفية في Keras 348

20 شبكة CNN من الصفر 364

21 نقل المعرفة والتدريب الدقيق 377

22 الكشف والتجزئة الدلالية 389

23 محوّل الرؤية (Vision Transformer) 402

24 تكبير الصور (Super-Resolution) 413

الجزء 5: التسلسلات واللغة والسلاسل الزمنية 426

25 الشبكات التكرارية و LSTM و GRU 427

26 الشبكات التكرارية المتقدّمة 440

27 معالجة اللغة الطبيعية عمليًا 453

28 السلاسل الزمنية 466

الجزء 6: المحوّلات: انتباه، محوّل، KerasHubg 479

29 المحوّلات وآلية الانتباه 480

30 منظومة KerasHub وKerasNLPg 492

الجزء 7: النماذج التوليدية: من الترميز إلى الانتشار 503

31 الترميز الآلي المتغيّر (VAE) 504

32 شبكات التوليد التنافسيّة (GAN) 518

33 نماذج الانتشار (Diffusion Models) 531

الجزء 8: الهندسة الإنتاجية: من النموذج إلى الخدمة 542

34 الطبقات والنماذج المخصّصة 543

35 الحفظ والتحميل 559

36 تسريع التدريب 573

37 التصدير والنشر 585

38 التكرار والاختبار 595

39 التفسير والتشخيص 609

40 الأخطاء الشائعة وكيف تُصلحها 624

الجزء 9: مشاريع تطبيقية متكاملة 634

41 مشروع تصنيف صور إنتاجيّ 635

42 مشروع التنبّؤ بالسلاسل الزمنيّة 648

43 مشروع تحليل المشاعر ونشره كخدمة 662

الملاحق

A قاموس المصطلحات عربي-إنجليزي

B مرجع Keras 3 السريع

C ورقة المعادلات

D إجابات مختارة للتمارين

E فهرس الأخطاء الشائعة

F المراجع والمصادر

الجزء الأول

الأساسيات: من الفكرة إلى أول نموذج

تبني الأساس الذي يقوم عليه الكتاب كلّه: ما Keras 3 ولماذا وُجد، كيف تُهيّئ بيئة لا تنهار بعد أسبوع، كيف تفكّر بالأوتور والبثّ والتدرّج التلقائي، ثم كيف تُخرج أول نموذج كامل يعمل.

ما تكسبه من هذا الجزء

تخرج وأنت قادر على تثبيت بيئة نظيفة، وقراءة رسائل الأخطاء، وبناء نموذج أوّلي وتدريبه وتقييمه بلا نسخ أعمى.

فصول هذا الجزء

1 Keras 3: كيف تغيّر عالَم التعلّم العميق؟

2 البيئة: من الصفر إلى أول استيراد ناجح

3 الأوتور: اللغة التي يفكّر بها Keras

4 بنية النموذج: ثلاث طرق لتحديد الشبكة

5 أول نموذج كامل: شرح سطر بسطر

1

الفصل 1: Keras 3: كيف تغيّر عالَم التعلّم العميق؟

بطاقة الفصل

الفصل 1

مهمّة هذا الفصل

يضع الصورة الكاملة: ما Keras 3 وما ليس هو، وكيف يقف فوق JAX وTensorFlow وPyTorch بمحرّك واحد، وما الذي تغيّر عن Keras 2.

ماذا تكسب منه

تختار الأداة الصحيحة قبل أن تكتب سطرًا، وتعرف أي كود قديم سيحتاج تعديلًا عند الترقية – بدل أن تكتشف ذلك في منتصف المشروع.

الدالة	دورها	متى تستخدمها
`keras.__version__`	رقم الإصدار المثبَّت فعليًا	أول سطر في أي تقرير خطأ أو طلب دعم
`keras.backend.backend()`	اسم المحرّك الخلفي النشط	عندما يختلف السلوك بين جهازك وجهاز زميلك
`keras.config.set_backend()`	تبديل المحرّك في وقت التشغيل	لتجربة الكود نفسه على محرّكين بلا إعادة تثبيت
`keras.ops`	طبقة العمليات الموحّدة عبر المحرّكات	بدلًا من tf.* أو torch.* في أي كود تريد نقله
`keras.saving.load_model()`	تحميل نموذج محفوظ بصيغة keras.	عند استهلاك نموذج جاهز أو استئناف عمل

قصة مفتاحية: في ربيع 2026، استلم مهندسٌ فريقًا عامِلًا فيه نموذج كشف احتيال مكتوبٌ قبل ثلاث سنوات بـ Keras 2 وواجهة TensorFlow الخلفية. طلب من الفريق ترقية الحزمة، فاكتشف أن أول أمر كسره لم يكن الشبكة نفسها بل سطر الاستيراد: `from tensorflow import keras` لم يعد بوابة الدخول الموصى بها، و `model.save("model.h5")` لم يعد الطريق الآمن لحفظ النموذج. هذا الفصل يجيب سؤالًا أعمق: **ما الذي تغيّر فعلًا في عالم التعلّم العميق بين 2023 و2026، ولماذا كان هذا التحوّل في جوهره لا في الأدوات؟**

أهداف الفصل

- [ ] التمييز بين Keras كـ«واجهة استخدام» (API) وبين TensorFlow وJAX وPyTorch كـ«محرّكات تنفيذ» (runtimes).
- [ ] وصف بنية Keras 3 متعددة الواجهات الخلفية، ومعرفة أيّ واجهة خلفية تناسب أيّ عبء عمل.
- [ ] سرد التغييرات الجوهرية من Keras 2.x إلى Keras 3.0: الاستيراد، `keras.ops`، صيغة `.keras`، ومسار التصدير.
- [ ] استعراض أبرز ما جاء في Keras 3.15: تصدير `sliding_window`، PyTorch، `MultiOptimizer`، والدوال الرياضية الجديدة.
- [ ] تحديد موضع منظومة KerasHub/KerasNLP ومصرّح keras-cv المؤرشف ضمن الخريطة الأكبر.
- [ ] تطبيق خطة ترحيل عملية من مشروع Keras 2 إلى Keras 3 دون إعادة كتابة كل شيء.

1.1 ما هو Keras — وما ليس هو

تخيّل أنك تشتري سيارة. المحرك (engine) هو TensorFlow أو JAX أو PyTorch: هو ما يشغّل السيارة فعليًا ويصنع القوة والدّة. أمّا لوحة القيادة (dashboard) فهي Keras: هي ما تتفاعل معه أنت — المقود، العدّادات، زرّ التشغيل. في Keras 2 كانت لوحة القيادة ملتصقة بمحرّك واحد بُني من الصفر له؛ في Keras 3 صارت اللوحة تعمل على أكثر من محرّك، وتتبدّل المحرّك تحت يديك دون أن تتعلّم لغة جديدة.

هذا التمييز هو أهمّ ما ستفهمه في هذا الكتاب. **Keras ليس إطارًا منافسًا لـTensorFlow أو JAX أو PyTorch**، ولا يدّعي استبدالها؛ Keras 3 هو **طبقة واجهة** (user-facing API) فوقها، ويمنحك نموذجًا واحدًا للكتابة (model definition)، ودورة تدريب واحدة (`compile` / `fit` / `evaluate` / `predict`)، وفضاء عمليات موحّدًا (`keras.ops`).

ولكل مكوّن في الخريطة مهمة مختلفة:

المكوّن	وظيفته	ما لا يقدّمه
Keras	تعريف النموذج، الطبقات، دورة التدريب، الحفظ والتحميل	لا ينفّذ ضرب مصفوفات بنفسه في وضع الإنتاج المعتاد
TensorFlow	تنفيذ الرموز، التدريب الموزّع، أدوات النشر (TFLite، TensorFlow Serving)	واجهته كانت مرتبطة بـ `tf.keras` لا بواجهة مستقلة
JAX	تنفيذ غير متزامن (async)، jit، sharding على أجهزة متعددة	منحنى تعلّم أطول لمن يأتي من جلسات TF
PyTorch	منظومة متنامية (Hugging Face، TorchScript، torch.compile)	تاريخًا لم يكن Keras يتكامل معه دون «الطبقة الفاضلة»

> **ملاحظة:** مع Keras 3، من ينوي الاعتماد على «إطار واحد» لأعماله يستطيع أن يكتب طبقة النماذج مرة واحدة، ثم يختار المحرّك بحسب المهمة: TensorFlow للخدمة، JAX للبحث والتوزيع على أجهزة متعددة، PyTorch حين يكون التكامل مع منظّمة خارجية شرطًا تعاقديًا.

1.2 البنية متعددة الواجهات الخلفية

كيف يبدّل Keras المحرّك؟ الجواب: **طبقة تحويل (dispatch layer)**. عند الاستيراد، يقرأ Keras متغيّر البيئة `KERAS_BACKEND`، أو يستدعي `keras.config.set_backend(...)` صراحةً، ثم يوجّه كل عملية حسابية إلى المحرّك المختار.

```python
import os
os.environ["KERAS_BACKEND"] = "jax"  # اختيار المحرّك قبل استيراد keras

import keras  # الاستيراد العام: لا tensorflow ولا torch

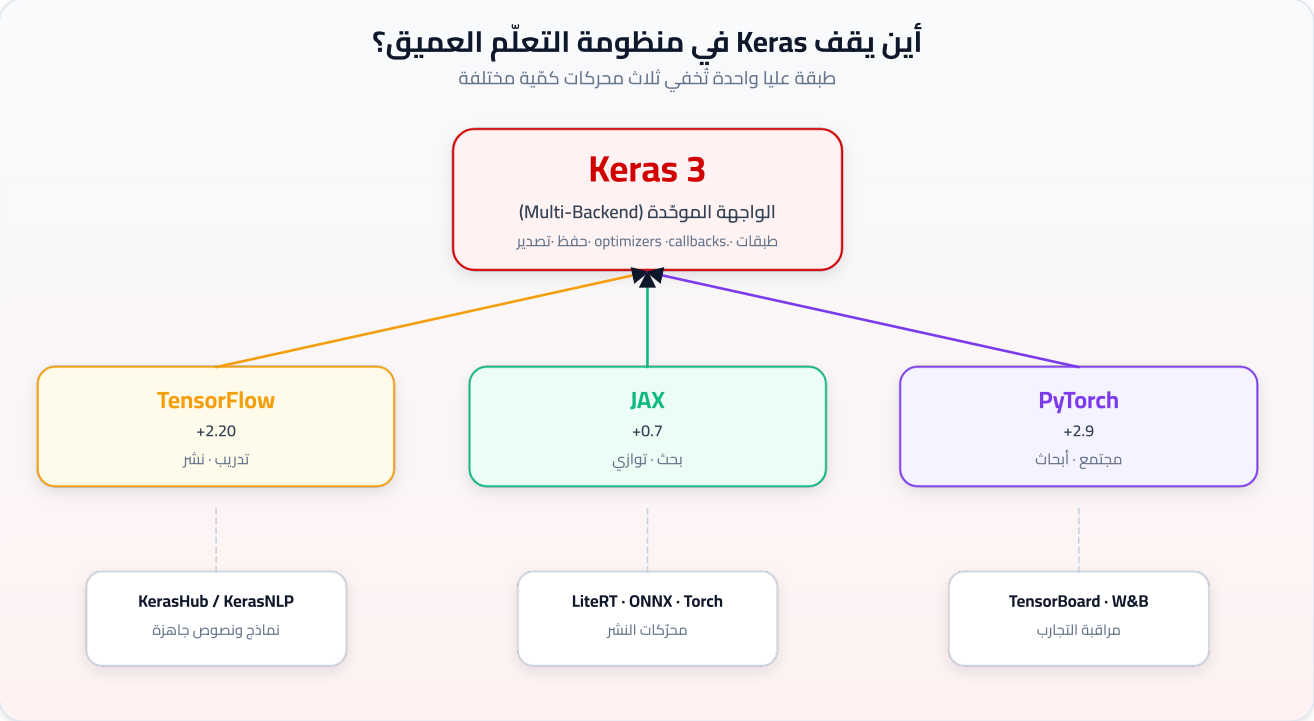
print("إصدار Keras:", keras.__version__)      # النتيجة المتوقعة: 3.15.1
print("الواجهة الخلفية:", keras.backend.backend())  # النتيجة المتوقعة: jax
```

الواجهات الخلفية المدعومة رسميًا في Keras 3.15 هي: **TensorFlow**، و**JAX**، و**PyTorch**، بالإضافة إلى **OpenVINO** بوصفه واجهة **للاستدلال فقط** (inference-only) – أي أنه يُستخدم لتشغيل نموذج مُدرَّب، لا لتدريبه. النتيجة العملية: قد تختار OpenVINO على جهاز Intel صناعي لخدمة الاستدلال، ثم تُدرّب النموذج نفسه على PyTorch في الخادم.

هذه البنية لها كلفة وكسبة. الكلفة أن كل عملية يجب أن تُنفَّذ عبر طبقة التحويل، وهذه الطبقة تحاول أن تُبقي التكلفة أقل من واحد إلى اثنين بالمئة من زمن التنفيذ في العمليات الشائعة. الكسبة أن الكود نفسه يصير قابلًا للنقل بين المحرّكات دون تعديل.

> **نصيحة:** لا تخلط محرّكين في العملية نفسها. Keras يقرأ إعداد المحرّك مرّة عند الاستيراد؛ استدعاء `keras.config.set_backend("torch")` بعد بناء نموذج مبنيّ على JAX لا يحوّل ذلك النموذج، بل قد يقودك إلى أخطاء غامضة في المقاييس والأوزان.

1.3 ماذا تغيّر من Keras 2 إلى Keras 3؟



شكل 1 – خريطة منظومة Keras 3 وعلاقتها بالمحرّكات الخلفية

ماذا يحدث لو بقيت على `import tensorflow as tf` ثم `tf.keras` ؟ ستستمر في العمل – Keras 3 يحتفظ بجسر توافق (legacy shim) باسم `keras._tf_keras` لتمرير كود TensorFlow القديم – لكنك ستُنفق جزءًا من ميزتك: ستظل محصورًا داخل TensorFlow، وستفقد قابلية نقل النموذج نفسه إلى JAX أو PyTorch.

الفروق الجوهرية التي يجب أن تستوعبها:

1. **الاستيراد صار موحّدًا.** صار المرجع هو `import keras` ثم `keras.layers.Dense`، لا `from tensorflow import keras`. هذا قابل للاختبار آليًا: ابحث في مشروعك عن `from tensorflow import keras` وستعرف عدد الملفات التي تحتاج تعديلًا.

2. **ظهر `keras.ops` كفضاء عمليات بديل عن `tf.*`.** كل عملية حسابية في كود النموذج (جمع، ضرب مصفوفات، batch matmul، softmax) صارت تُكتب بـ `keras.ops.*`، فيُنفَّذها المحرّك المختار. هذا هو مفتاح قابلية النقل: كود يستخدم `tf.linalg.matmul` داخل طبقة مخصّصة مرتبط بـTensorFlow، وكود يستخدم `keras.ops.matmul` محمول.

3. **صيغة `.keras` صارت الصيغة المفضّلة للحفظ.** `model.save("m.keras")` يُنتج ملفًا بصيغة ZIP يحوي: `config.json` (بنية النموذج)، و `metadata.json` (بيانات وصفية)، وملف

الأوزان، وأي أصول مضمّنة. الملف مكتفٍ ذاتيًا: نقله إلى خادم آخر لا يحتاج الكود الأصلي، ويكفيه `keras.saving.load_model("m.keras")`.

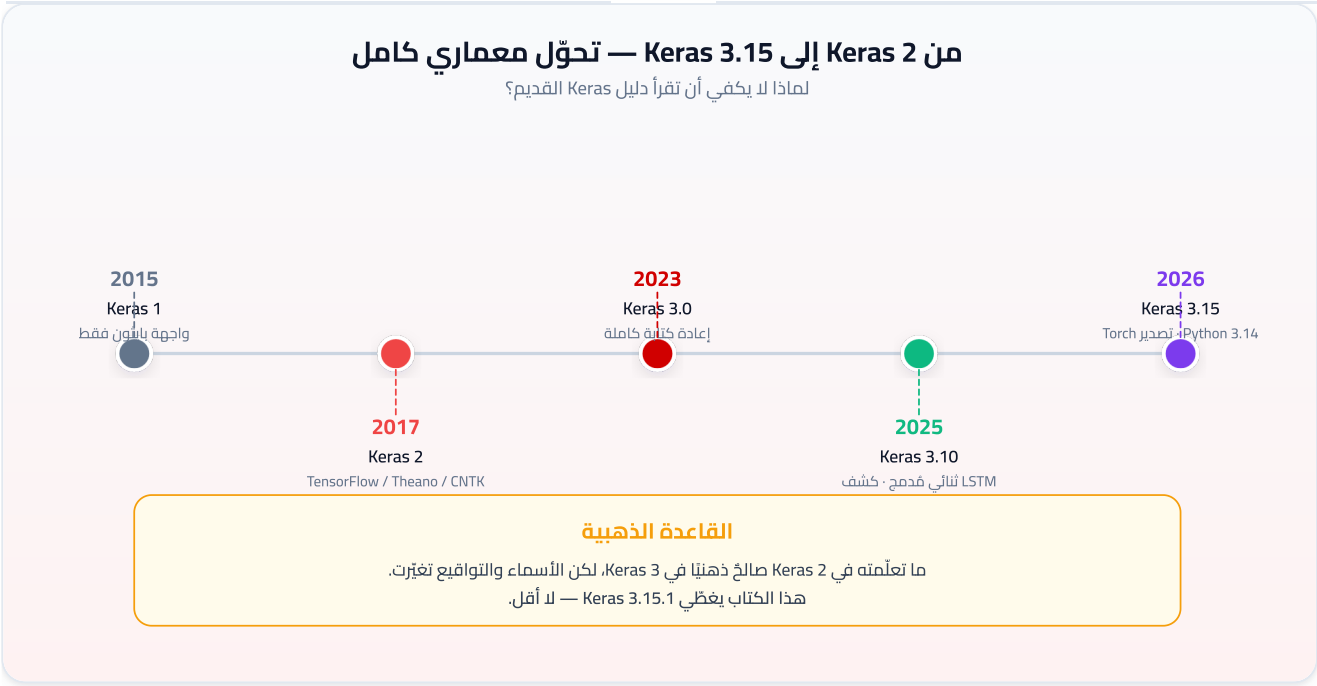
4. **مسار التصدير إلى SavedModel تغيّر.** في Keras 3 لم يعد `model.save("m.savedmodel")` مدعومًا؛ المسار الصحيح هو `model.export(...)` بصيغة `tf_saved_model`، ويُنتج قطعة استدلال (inference-only artifact) تحوي المسار الأمامي فقط (`call()`) ولا تحوي منطق التدريب. للنماذج المصنّفة بـSubclassing هناك قيود إضافية على هذا المسار، وستعالَج في الفصل الرابع.

5. **تغيّر أسماء المقاييس وواجهات الحفظ.** صار الاسم `keras.metrics.F1Score` بدل `f1_score`، و`keras.metrics.AUC` بدل `auc`، وواجهة الحفظ الموحّدة هي `keras.saving.load_model` / `keras.saving.save_model`.

6. **أساليب المعالجة المسبقة انتقلت إلى الطبقات.** `ImageDataGenerator` أصبح مسارًا مهجورًا؛ البديل هو طبقات داخل الرسم البياني مثل `keras.layers.RandomFlip` و`keras.layers.Rescaling`، وهي قابلة للتسلسل داخل ملف `.keras`.

عند الفحص العملي، التكلفة الحقيقية للترحيل ليست في الشبكة: هي في **عدد الملفات** التي تستورد `tf.keras`، وفي الاستخدامات الصامتة لـ`tf.` داخل دوال الخسارة أو الطبقات المخصّصة.

1.4 أبرز ما جاء في Keras 3.15



شكل 2 – الخط الزمني لإصدارات Keras من x.2 إلى 3.15

الإصدار 3.15.0 (يونيو 2026) وإصدار التصحيح 3.15.1 (يوليو 2026) قدّما مجموعة تغييرات عملية، لا مجرد تحسينات داخلية:

- **تصدير Keras إلى PyTorch الأصلي:** صارت `model.export("model.pt2", format="torch")` تُنتج برنامج PyTorch قابلًا للتحميل بـ `torch.export.load`. هذه ميزة جديدة كسرت حاجزًا حقيقيًا: نقل نموذج مُدرَّب في Keras إلى منظومة إنتاج قائمة على PyTorch دون إعادة كتابة الشبكة. كما أُضيف دعم تصدير LiteRT (TFLite) للنماذج ذات الواجهة الخلفية PyTorch.
- **نافذة منزلقة في الانتباه:** `keras.layers.MultiHeadAttention` و`keras.layers.GroupQueryAttention` يقبلان الآن المعامل `sliding_window`، وهو ما يجعل الانتباه على السياقات الطويلة (نصوص، سلاسل زمنية) أرخص حسابيًا لأن كل موضع لا يرى إلا جواره.
- **`keras.optimizers.MultiOptimizer`:** يسمح بإسناد مُحسّنات مختلفة إلى شبكات فرعية مختلفة داخل النموذج نفسه – مثلًا مُحسّن بمعدّل تعلّم مرتفع للرأس الجديد ومُحسّن بمعدّل منخفض للجسم المدَرَّب (body) أثناء النقل والت Fine-tuning.

- **دوال رياضية جديدة في `keras.ops`:** أضيفت `unique` و`fabs` و`fmax` و`fmin` و`erfc` و`dsplit` و`percentile` و`nanpercentile`، وفي `keras.ops.linalg` أُضيفت `pinv` (المصفوفة شبه العكسية) و`matrix_rank`، وفي `keras.ops.image` أُضيفت `sobel_edges` (كشف الحواف) و`ssim` (التشابه البنيوي). وصار `keras.ops.transpose` يقبل محاور سالبة.
- **تعزيز أمني:** حزم x.3.15 أضافت تقييدًا في فكّ تسلسل بيانات IMDB وReuters وCIFAR (منع `pickle` غير المقيد)، والتحقّق من مجموعات H5 المتداخلة، ورفض ملفات `.keras` المضغوطة التي قد تُرهق الذاكرة عند فكّها (decompression bomb). هذه ليست تفاصيل تجميلية: تحميل ملف `.keras` من مصدر غير موثوق كان ثغرة تنفيذ شيفرة، والآن يُرفض.
- **دعم Python 3.14:** عولجت تغيّرات التحويل المنطقي (coercion) في Python 3.14، وأُصلحت بعض دوال الأشكال.

كم يكلّف اختيار المحرّك؟

سؤال واقعي: «ما الفرق الزمني بين محرّك وآخر؟» المعادلة التي تحكم الجواب عملية لا نظرية:

$$T_{\text{train}} = E \times \left\lceil \frac{N}{B} \right\rceil \times t_{\text{step}}$$

حيث T_{train} هو زمن التدريب الكلي، وE عدد الحقبات (epochs)، وN عدد عينات التدريب، وB حجم الدفعة (batch size)، و$\lceil N/B \rceil$ عدد خطوات التحديث في الحقبة الواحدة بعد التقريب لأعلى، وt_{step} زمن الخطوة الواحدة (مسار أمامي + مسار خلفي + تحديث أوزان).

القراءة العملية للمعادلة: العلاقة بين المحرّكات تظهر في **العاملين الثالث والرابع** فقط. المحرّك الأسرع (أو الأسرع في استثمار العتاد المتوازي) يقلّل t_{step}، والبنية التي توزّع البيانات تقلّل t_{step} كذلك. أما E وN وB فهي قراراتك أنت لا قرارات المحرّك. لذلك: **لا تقارن محرّكين على أساس نموذج يحسب 10 حقبات؛ قارنهما على t_{step} وحدها** (وهي ما يعكسه قياس throughput بالعيّنات/الثانية).

1.5 المنظومة حول Keras: KerasHub وKerasNLP وkeras-cv

المنظومة المحيطة بـ**KerasHub**: Keras 3 هو مستودع النماذج المدرَّبة مسبقًا لـKeras 3 (نماذج BERT وGPT-2 وViT وغيرها) بواجهة موحّدة عبر المحرّكات. وأصبح **KerasNLP** جزءًا من KerasHub: مستودع `keras-team/keras-nlp` لم يعد مستقلًا، وهو الآن يوجّه إلى `keras-team/keras-hub`. أمّا **keras-cv** فقد أصبح **مؤرشفًا (archived)**؛ المشروع متوقف، وقد انتقلت كثير من قدراته إلى نواة Keras – ومن أوضح الأمثلة عملية `keras.ops.image.sobel_edges` و`keras.ops.image.ssim` اللتان كانتا في حزمة KerasCV منفصلة (الحواف والتشابه البنيوي).

ما معنى ذلك عمليًا؟ ثلاث قواعد:

1. لا تبدأ مشروعًا جديدًا على `keras-cv`؛ استخدم `keras.ops.image` وطبقات Keras 3 الأساسية.
2. إذا كان لديك كود KerasNLP، فإن مسار الترقية هو الانتقال إلى حزمة `keras-hub` وتثبيت نسخة متوافقة مع Keras 3.15.
3. النماذج المدرَّبة مسبقًا في KerasHub تُحمَّل وتُهيَّأ داخل نموذج Keras 3 نفسه، فتستفيد من `fit` / `save` / `export` بشكل طبيعي.

1.6 جدول مقارنة: Keras 2.x مقابل Keras 3.15

المقارنة هنا مبنية على **معايير** قابلة للقياس، لا على تفضيلات ذاتية:

المعيار	Keras 2.x	Keras 3.15
بوابة الاستيراد	`from tensorflow import keras`	`import keras` (واحدة لكل المحرّكات)
المحرّك	TensorFlow فقط	TensorFlow / JAX / PyTorch / OpenVINO (استدلال)
مرجع العمليات داخل الطبقات	`tf.*`	`keras.ops.*`
صيغة الحفظ المفضّلة	`.h5` أو SavedModel	`.keras` (مكتفٍ ذاتيًا)
تصدير SavedModel	`model.save("m.savedmodel")`	`model.export("m", format="tf_saved_model")`
جودة واجهة الاستدلال مع OpenVINO	غير متاح	متاح عبر واجهة OpenVINO
زمن الوصول لأوزان FN وقيود CUDA	مرتبط بإصدار TF	يُختار المحرّك مستقلًا
توافق الكود القديم	كامل	عبر جسر `keras._tf_keras` للتوافق فقط

1.7 مثال محلول: ترقية نظام كشف الاحتيال

ماذا يحدث لو كان لديك نظام تصنيف المعاملات إلى «احتيال / سليم» بُني بـKeras 2؟ إليك خطة ترحيل منظّمة:

الخطوة 1 – حصر الاعتماديات. ابحث في المستودع عن `tensorflow` و `tf.keras` . على النمط المعتاد في المشاريع القديمة، ستجد الاستيراد في `train.py` ، وفي `losses.py` ، وفي `callbacks.py` ، وأحيانًا داخل `layers.py` (وهذا الأخطر).

الخطوة 2 – تثبيت البيئة. أنشئ بيئة افتراضية جديدة وأثبّت `keras==3.15.1` مع محرّك واحد. لا تخلط `tensorflow` و`torch` في البيئة نفسها إن لم تكن تحتاجهما: كل منهما يجرّ حزمًا ثقيلة قد تتعارض في إصدارات numpy/h5py.

الخطوة 3 – تعديل الاستيراد والعمليات. استبدل `from tensorflow import keras` بـ`import keras`، ثم استبدل `tf.nn.relu` بـ`keras.ops.nn.relu`، و`tf.matmul` بـ`keras.ops.matmul`، و`tf.reduce_mean` بـ`keras.ops.mean`.

الخطوة 4 – تحديث نقاط الحفظ والتصدير. ملف `.h5` القديم يُحمَّل بـ`keras.saving.load_model("m.h5", compile=False)` ما دام في نطاق Keras 3 المتوافق، ثم يُحفظ فورًا بصيغة `.keras` عبر `model.save("m.keras")`، ثم يُعاد التحميل للتحقّق من سلامة البنية. أما للنشر، فالمسار هو `model.export(...)` بصيغة مناسبة لهدفك.

الخطوة 5 – التحقّق السلوكي. قارن التوقعات على مجموعة تحقق ثابتة قبل الترقية وبعدها، وتأكّد من تطابق `evaluate` ضمن سماحية معقولة. الفروق الصغيرة توضيح source (مثل عشوائية التهيئة أو اختلاف دقّة الحساب بين المحرّكات)، والفروق الكبيرة تعني بقايا اعتماد على TF.

الكود أدناه يلخّص Step 3 بأبسط صورة ممكنة:

```python
import keras

# Keras 2 (قبل): from tensorflow import keras; import tensorflow as tf
# Keras 3 (بعد): استيراد موحّد + عمليات محايدة عن المحرّك
model = keras.Sequential([
    keras.layers.Input(shape=(20,)),          # طبقة الإدخال صارت صريحة داخل الـSequential
    keras.layers.Dense(64, activation="relu"),  # التنشيط يُمرَّر كوسيط لا كطبقة منفصلة
    keras.layers.Dropout(0.2),                # إسقاط عشوائي أثناء التدريب فقط
    keras.layers.Dense(1, activation="sigmoid"),
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=[keras.metrics.AUC(name="auc")],  # الاسم الصحيح في Keras 3 هو AUC
)
model.summary()  # ملخّص بعدد الأوزان لكل طبقة (205) النتيجة المتوقعة: Dense
```

1.8 أخطاء شائعة

تنبيه: أشهر خطأ في الترحيل هو كتابة `from tensorflow import keras` ثم ظنّ أن المشروع صار «Keras 3». الكود يعمل، لكنه لا يستفيد من تعدّد المحرّكات، ويظل مرتبطًا بـTensorFlow.

إنذار: الخلط بين `model.save("m.savedmodel")` و `model.export(...)`. الأول غير مدعوم في Keras 3؛ والثاني يُنتج قطعة استدلال لا قطعة تدريب. الخلط بينهما ينتج ملفات لا يستطيع `load_model` قراءتها.

ملاحظة: لا تفترض أن كل طبقة مخصّصة في كودك القديم تعمل كما هي. الطبقات التي تستدعي `tf.` مباشرةً ستظل مرتبطة بمحرّك واحد حتى تُحوَّل إلى `keras.ops`.

> **تنبيه:** تغيير المحرّك يغيّر الأرقام. الفروق البسيطة في الخسارة (من 0.5412 إلى 0.5407 مثلًا) ناتجة عن اختلاف ترتيب عمليات الجمع ودقّة الحساب؛ اعتمد سماحية، لا تطابقًا حرفيًا.

التمارين

1. اقرأ ملاحظات إصدار Keras 3.15.0 على GitHub واستخرج ثلاثًا من الإضافات التي تؤثّر في تصميم مشروعك. (المتوقع: مثلًا ، `sliding_window` ، `export_torch` ، `MultiOptimizer` ، مع تبرير أثر كل واحدة على مشروعك).

2. لديك نظام كشف الاحتيال يعمل اليوم على Keras 2 + TensorFlow، ومطلوب تسليمه كخدمة على خادم Intel. أيّ محرّك تختار للخدمة وأيّ محرّك تختار للتدريب، ولماذا؟ (المتوقع: تدريب على TF أو PyTorch بحسب المتاح، واستدلال على OpenVINO لأن واجهته للاستدلال فقط وتُشغَّل على عتاد Intel؛ مع توضيح أن التحويل يتم عبر `model.export`).

3. افحص مستودعًا فيه 30 ملف Python وابحث عن `from tensorflow import keras` و `tf.` داخل الطبقات. صنّف الملفات: أيّها يحتاج تعديلًا فوريًا، وأيّها يمكن تأجيله؟ (المتوقع: الملفات التي تُعرّف دوال خسارة/مقاييس أو طبقات مخصّصة تحتاج تعديلًا عاجلًا؛ ملفات استدعاء inference فقط قد تكتفي بتحويل الاستيراد).

4. أيّ العبارتين التاليتين تختار ولماذا: «Keras 3 إطار منافس لPyTorch» أم «Keras 3 واجهة فوق PyTorch/TF/JAX»؟ اشرح الأثر العملي على قرارك في اختيار المحرّك. (المتوقع: الثانية؛ لأن Keras 3 لا ينفّذ التدريب بنفسه بل يوجّه العمليات، وهذا يتيح كتابة النموذج مرة واحدة وتشغيله على أكثر من محرّك).

5. صحّح الخطأ: `model.save("model.savedmodel")` . (المتوقع: الاستدعاء غير مدعوم؛ البديل `model.export("model", format="tf_saved_model")` لإنتاج قطعة استدلال قابلة للخدمة).

6. شرط تعاقدي في مشروعك يفرض PyTorch، بينما فريق البحث يعمل بJAX. اقترح بنية تحترم الشرط وتبقى قابلة لإعادة الاستخدام. (المتوقع: تعريف النموذج بـ `import keras` و `keras.ops` كي يكون محمولًا، وتدريب النسخة الإنتاجية على Torch، وبقاء نسخة JAX للتجارب؛ مع توثيق الواجهة البرمجية للنموذج كعقد).

7. اذكر ثلاث دوال أضيفت إلى `keras.ops` في 3.15 واذكر فائدتها. (المتوقع: `keras.ops.image.sobel_edges` لكشف الحواف، `keras.ops.image.ssim` لقياس التشابه البنيوي، `keras.ops.linalg.pinv` لحساب المعpseudo العكسي في حل الأنظمة الخطية).

8. ماذا يعني عمليًا أن `keras-cv` مؤرشف؟ ماذا تفعل لو كان مشروعك يستورد منه اليوم؟ (المتوقع: لا تبدأ عليه؛ استبدل العمليات بما يقابلها في `keras.ops.image` وطبقات Keras الأساسية، وقد تحتاج بدائل أو تنفيذًا صغيرًا يدويًا).

9. احسب T_{train} تقريبيًا لمشروعك: $E = 20$ حقبة، $N = 50{,}000$ عينة، $B = 64$. كم خطوة تحديث؟ (المتوقع: $\lceil 50000/64 \rceil = 782$ خطوة لكل حقبة، أي 15{,}640 خطوة إجمالًا؛ ثم قارن زمن الخطوة t_{step} بين محرّكين).

10. لماذا يُعتبر Keras 3 مناسبًا لمشروع في الخدمات المالية يجب أن يبقى قابلًا للترحيل بين بيئات تشغيل مختلفة (GPU سحابي، CPU داخلي، حافة)؟ (المتوقع: لأن النموذج يُعرَّف مرة واحدة عبر واجهة محايدة، ويُصدَّر إلى صيغ متعددة `torch` / `litert` / `openvino` / `onnx` / `tf_saved_model` بحسب البيئة).

11. ما الفرق بين «saved model» بصيغة `.keras` و«تصدير SavedModel»؟ أيّهما تحتفظ به في مستودع النماذج؟ (المتوقع: `.keras` يحوي بنية التدريب كاملة مع إمكانية الاستئناف؛ أما تصدير SavedModel فهو قطعة استدلال-forward فقط، عادةً لا يُحفظ في مستودع التدريب بل يُولَّد عند النشر).

12. اكتب ثلاث قواعد تكتبها في دليل الفريق لمنع انتقال كود Keras 3 إلى `tf.` داخل الطبقات. (المتوقع: منع `import tensorflow` داخل ملفات الطبقات؛ استيراد `keras` فقط؛ استخدام `keras.ops` لكل عملية؛ مراجعة في CI عبر grep عن `tf\.`).

ملخص الفصل

- Keras 3 ليس محرّكًا جديدًا بل واجهة استخدام موحّدة فوق TensorFlow وJAX وPyTorch (وOpenVINO للاستدلال).
- الاستيراد المرجعي `import keras` ، والعمليات داخل الطبقات تكتب بـ `keras.ops.*` لا بـ `tf.*` .
- صيغة `.keras` هي صيغة الحفظ المفضّلة، و `model.export(...)` هو المسار الصحيح لتصدير قطع النشر بصيغ `torch` / `litert` / `openvino` / `onnx` / `tf_saved_model` .
- الإصدار 3.15 أضاف `export_torch` ، و `sliding_window` للانتباه، و `MultiOptimizer` ، ودوالًا رياضية جديدة في `keras.ops` ، مع تعزيز أمني ودعم Python 3.14.
- الفرق بين محرّك وآخر يظهر في t_{step} داخل معادلة الزمن الكلي، لا في عدد الحقبات.
- KerasHub هو اليوم مرجع النماذج المدرَّبة مسبقًا، وKerasNLP جزء منه، وkeras-cv مؤرشف ومزاح إلى نواة Keras.

الفصل 2: البيئة: من الصفر إلى أول استيراد ناجح

بطاقة الفصل

الفصل 2

مهمّة هذا الفصل

يبني بيئة معزولة قابلة للتكرار: تثبيت Keras، اختيار المحرّك، والتأكّد من رؤية المعالج الرسومي قبل كتابة أي نموذج.

ماذا تكسب منه

تنتقل من «يعمل على جهازي» إلى «يعمل دائمًا»، وتُغلق باب أخطاء التثبيت التي تستهلك أيامًا بلا سبب واضح.

الدالة	دورها	متى تستخدمها
`python -m venv .venv`	بيئة معزولة لكل مشروع	قبل أي تثبيت؛ يمنع تعارض الإصدارات
`pip install "keras[torch]"`	تثبيت Keras مع محرّك محدّد	عند بدء مشروع جديد
`KERAS_BACKEND`	متغيّر بيئة يحدّد المحرّك لكل الجلسات	للتثبيت الدائم بدل التبديل في كل سكربت
`keras.config.set_backend()`	تبديل المحرّك داخل السكربت	لتجارب سريعة بين محرّكين
`keras.backend.backend()`	التحقّق النهائي من المحرّك النشط	في أعلى كل سكربت إنتاجي

قصة مفتاحية: استلمت جهازًا جديدًا، وطلبت من الحاسوب «افتح لي بايثون» ثم سألت ما هو ثم سألت: «أين Keras؟» فردّ الجهاز: `ModuleNotFoundError` . ساعتان من البحث، والسبب كان سطرًا واحدًا في ملف إعدادات. في هذا الفصل نتعلّم كيف نُجهّز البيئة مرة واحدة، فتنتهي المشكلة إلى الأبد.

أهداف الفصل

- [ ] إنشاء بيئة بايثون معزولة قابلة للتكرار
- [ ] تثبيت Keras 3.15.1 مع محرّك مناسب
- [ ] اختيار الواجهة الخلفية بمعيار واضح لا بالعادة
- [ ] التحقّق من أن البيئة سليمة قبل كتابة سطر واحد من كود النموذج
- [ ] فهم لماذا لا يكفي تثبيت Keras وحده

2.1 لماذا لا يكفي `pip install keras` ؟

في Keras 2، كان `pip install keras` يعني عمليًا «ثبّت TensorFlow أيضًا». أما Keras 3 فقد فكّ هذا الربط: المكتبة صارت **واجهة فوق محرّكات**، والمحرّك نفسه اختياري.

لذلك فإن الأمر التالي **ينجح** لكنه لا يكفي:

```bash
pip install keras
```

ينجح لأن Keras 3.15.1 تُحمَّل بلا محرّك، لكن أوّل استيراد لـ `keras.ops` سيطلب محرّكًا. إما أن تختار محرّكًا عند التثبيت:

```bash
pip install "keras[torch]"      # محرّك PyTorch
pip install "keras[tensorflow]" # محرّك TensorFlow
pip install "keras[jax]"        # محرّك JAX
```

أو تثبّت Keras والمحرّك منفصلين:

```bash
pip install keras torch
```

نصيحة: صيغة `keras[backend]` أفضل في ملفات `requirements.txt` لأنها تُثبّت المجموعة الكاملة المتوافقة، وتمنع ظهور تناقض في الإصدارات – وهو ما يُفسّر نصف أخطاء التثبيت في بيئات العمل المشتركة.

ما الذي تختاره؟

المعيار	TensorFlow	JAX	PyTorch	OpenVINO
سرعة التدريب على GPU	عالية	الأعلى غالبًا	عالية	لا يُدرّب
توازي النماذج (Sharding)	محدود	الأقوى	محدود	لا
نضج أدوات النشر	الأعلى	متوسط	متوسط	متخصص في Intel
تكامل HuggingFace	جيد	جيد	الأفضل	محدود
حجم التبعيات	كبير	متوسط	كبير	متوسط
الاستخدام المناسب	الإنتاج والخدمات	البحث والتوازي	البحث والمجتمع	الاستدلال على Intel

قاعدة عملية: إن لم يكن لديك سببٌ قاطع، ابدأ بـ **TensorFlow** في العمل الإنتاجي، و**JAX** في العمل البحثي الذي يحتاج توازيًا أو دقة عالية، و**PyTorch** إن كان مجالك NLP أو لديك كود قائم عليه.

2.2 البيئة المعزولة: لماذا؟

تخيّل أنك تعمل على مشروعين: أحدهما يحتاج Keras 3.15 والآخر Keras 2.16. تثبيت الاثنين في نفس بايثون يعني أن أحدهما سيكسر الآخر في كل مرة تنقل بينهما.

الحل: **بيئات معزولة** – كل مشروع في مجلّد خاص به بباكاجاته الخاصة.

على ويندوز

BASH

```
# إنشاء البيئة
python -m venv .venv

# تفعيلها
.venv\Scripts\activate

# ترقية أدوات التثبيت
python -m pip install --upgrade pip

# التثبيت
pip install "keras[torch]==3.15.1"
```

على لينكس وماك

BASH

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install "keras[torch]==3.15.1"
```

هل أُنشئ بيئة لكل مشروع؟

نعم، لكل مشروع.(rule صارمة). لا تتشارك بيئة بين مشروعين لهما متطلّبات مختلفة.



شكل 3 – خطوات تجهيز بيئة تطوير معزولة لـ Keras 3

2.3 تثبيت الإصدارات: من «يعمل» إلى «يعمل دائمًا»

بينما كان مشروعك يعمل على جهازك، ثم على جهاز زميلك، ثم في Docker، ثم في CI – هذا يتطلب تثبيتًا **مُثبَّت الإصدارات** لا تثبيتًا عابرًا.

```bash
# سجّل ما هو مثبَّت الآن
pip freeze > requirements.lock.txt
```

وفي `requirements.txt` اكتب:

```text
keras[torch]==3.15.1
numpy>=1.26,<3.0
matplotlib>=3.8
```

القاعدة: **ثبّت بالإصدار الدقيق** (`==`) ما تتعلّق به بشدة – Keras وNumPy – و**ثبّت بالنطاق** (`>=`) ما هو ثانوي.

ملاحظة: ملف `requirements.lock.txt` هو لقطة (snapshot) كاملة: ثبّت منه حين تحتاج إنتاجًا مطابقًا بايت-ل-بايت. ملف `requirements.txt` هو تصريح النوايا: حدّثه حين تريد تجديد.

2.4 أيّ محرّك يعمل؟

متغيّر البيئة

BASH

```
# ويندوز (PowerShell)
$env:KERAS_BACKEND = "jax"

# ويندوز (دائم)
setx KERAS_BACKEND jax

# لينكس/ماك
export KERAS_BACKEND=jax
```

من داخل الكود

PYTHON

```
import keras
keras.config.set_backend("jax")
```

تحذير: `set_backend` يجب أن يُستدعى **قبل** بناء أي نموذج. تغييره بعد بناء النموذج يخلق حالة غريبة: أوزان مبنية على محرّك، وقياسات على محرّك آخر.

التحقّق

```python
import keras

print(keras.__version__)        # 3.15.1
print(keras.backend.backend())  # jax
print(keras.backend.standardize_dtype("float32"))
```

أي محرّك تختار؟ مقارنة صريحة

المحرّك	الأنسب لـ	سرعة التدريب	توازي النماذج	ملاحظات
TensorFlow	نشر واسع · إنتاج	عالية	متوسط	مجتمع ضخم · أدوات MLOps
JAX	أبحاث · دقة	الأعلى	الأقوى	تدرّج تلقائي متقدّم
PyTorch	أبحاث · تخصيص	عالية	متوسط	منظومة HuggingFace
OpenVINO	استدلال على Intel	استدلال فقط	—	قراءة فقط · لا تدريب

قاعدة عملية

ابدأ بـ TensorFlow إن كنت ستنشر، و JAX إن كنت تبحث أو تحتاج توازيًا هائلًا،
و PyTorch إن كان مجالك NLP/Radio أو تملك كودًا قائمًا عليه.

شكل 4 – مصفوفة المحرّكات الخلفية: أيّ محرّك تختار ولماذا

2.5 هل يرى المحرّك معالجك الرسومي؟

هذا السؤال يُطرح كل مرّة، والإجابة تُغيّر سلوك كل شيء بعد ذلك. إليك سكربت تشخيص حقيقي:

PYTHON

```python
import keras
import numpy as np

print("Keras :", keras.__version__)
print("Backend:", keras.backend.backend())

# اختبار تمرير أمامي فعلي
x = np.random.rand(64, 784).astype("float32")
model = keras.Sequential([
    keras.layers.Input((784,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(10),
])
y = model(x, training=False)
print("Output shape:", y.shape)

# هل الأوزان على معالج؟
try:
    dev = next(iter(model.trainable_variables)).device
    print("Device:", dev)
except Exception as exc:
    print("تعذّر تحديد الجهاز:", exc)
```

كيف تقرأ النتيجة:

النتيجة	المعنى	الإجراء
`Device: cuda:0` أو `gpu`	كل شيء تمام	تابع
`Device: cpu` والمحرّك PyTorch	لا يوجد CUDA أو Torch مبني بـ CPU	ثبّت نسخة GPU من المحرّك
`Device: cpu` والمحرّك JAX	Search J XLA لا يجد GPU	تأكّد من توافق التعريفات
`ModuleNotFoundError`	المحرّك غير مثبَّت	ثبّته

عن CUDA

CUDA ليس «شيئًا تُثبّته في Keras»؛ هو سلسلة من ثلاث طبقات يجب أن تتوافق:

1. **تعريف GPU** (Driver) من موقع NVIDIA.
2. **CUDA Toolkit** – النسخة التي بُني عليها المحرّك.
3. **مكتبة cuDNN** للحسابات العصبية (التلافيف attentiong).

> **نصيحة:** أحدث نسخة CUDA ليست دائمًا الأفضل. المحركات المختبَرة غالبًا تعمل بأداء ممتاز على نسخ CUDA الأقدم قليلًا. تحقّق من جدول التوافق في توثيق محرّكك قبل التثبيت.

2.6 المسارات السريعة: Kaggleg Colab

أحيانًا لا تريد تثبيت شيء. حينها:

```python
# في خلية Colab أو Kaggle
!pip install -q "keras[torch]==3.15.1"
```

المقارنة:

المعيار	Colab / Kaggle	بيئة محلية
زمن البدء	فوري	دقائق
GPU	متاح (محدود زمنيًا)	Depends على عتادك
دوام البيئة	ينتهي عند انتهاء الجلسة	دائم
مناسبة للتجربة	نعم	للتجربة أيضًا
مناسبة للإنتاج	لا	نعم

القاعدة: **جرّب في المتصفح، ابنِ على جهازك.**

2.7 سكربت التحقّق الجاهز

احفظ هذا في `smoke_test.py` واستدعِه في كل بيئة جديدة:

PYTHON

```python
"""فحص بيئة Keras — احفظه باسم smoke_test.py واستدعِه بعد أي ترقية."""
import sys
import numpy as np
import keras

def main() -> int:
    print("=" * 46)
    print("Keras        :", keras.__version__)
    print("Backend      :", keras.backend.backend())
    print("Python       :", sys.version.split()[0])
    print("=" * 46)

    # 1) بناء نموذج
    model = keras.Sequential([
        keras.layers.Input((32,)),
        keras.layers.Dense(16, activation="relu"),
        keras.layers.Dense(1),
    ])

    # 2) تجميع
    model.compile(optimizer="adam", loss="mse")

    # 3) تدريب
    x = np.random.rand(128, 32).astype("float32")
    y = np.random.rand(128, 1).astype("float32")
    hist = model.fit(x, y, epochs=2, batch_size=32, verbose=0)
    assert "loss" in hist.history, "لا يوجد سجل خسارة"

    # 4) تنبّؤ
    out = model.predict(x[:8], verbose=0)
    assert out.shape == (8, 1), f"شكل غير متوقّع: {out.shape}"

    # 5) حفظ وتحميل
    model.save("_smoke.keras")
    reloaded = keras.saving.load_model("_smoke.keras")
    assert reloaded.count_params() == model.count_params()

    print("المعاملات    :", model.count_params())
```

```
    print("الخسارة النهائية:", float(hist.history["loss"][-1]))
    print("\n✓ البيئة سليمة — كل الفحوص ناجحة.")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

نصيحة: اجعل هذا السكربت أول خطوة في خط النشر (CI). فشل هذا الاختبار = فشل النشر، بلا محاجّات.

2.8 الأخطاء الشائعة في هذا الفصل

خطأ 1 – تثبيت Keras بلا محرّك. العرض: `ModuleNotFoundError: No module named 'tensorflow'` عند أول استخدام. السبب: Keras 3 لا يجلب محرّكًا تلقائيًا، والقيمة الافتراضية لواجهة Keras هي TensorFlow. الحل: ثبّت المحرّك الذي تريده صراحةً: `pip install "keras[jax]"`.

خطأ 2 – تبديل المحرّك بعد بناء النموذج. العرض: أخطاء غريبة عند `predict` بعد `set_backend`. السبب: الأوزان والقياسات على محرّكين مختلفين. الحل: اضبط `KERAS_BACKEND` قبل تشغيل البرنامج، لا داخله.

خطأ 3 – نسيان تفعيل البيئة. العرض: `pip install` ينجح لكن `import keras` يفشل. السبب: التثبيت ذهب إلى بايثون النظام لا إلى البيئة. الحل: فعّل البيئة أولًا وتحقّق أن مسار التثبيت صحيح: `python -m pip show keras` .

خطأ 4 – تثبيت بلا إصدارات. العرض: المشروع يعمل على جهازك ويكسر عند الزميل. السبب: تثبيت عابر لأحدث الإصدارات. الحل: `pip freeze` واحفظ الملف.

خطأ 5 – بناء على GPU بلا تحقّق. العرض: التدريب بطيء inexplicably. السبب: المحرك يعمل على المعالج لأن CUDA غير مثبَّت. الحل: شغّل سكربت التشخيص أعلاه واقرأ سطر

`Device` .

خطأ 6 – تثبيت كل شيء في Python النظامي. العرض: أخطاء أذونات على لينكس. السبب: تثبيت عالمي. الحل: استخدم بيئة معزولة دائمًا.

التمارين

1. **تشخيص الواجهة الخلفية.** بيئتك تُظهر `backend: tensorflow` بينما `tensorflow` غير مثبَّت. اشرح بالضبط أين تقرأ Keras قيمة الواجهة، ثم صلحها بطريقتين (متغيّر بيئة، وكود). (المتوقع: ضبط `KERAS_BACKEND` قبل الاستيراد، أو `keras.config.set_backend` قبل بناء أي نموذج.)

2. **تثبيت مُثبَّت.** اكتب `requirements.txt` لمشروع يستخدم Keras 3.15.1 وNumPy وMatplotlib، ثم اشرح لماذا تُثبّت الحزمة الأولى بـ `==` وبقية الحزم بـ `⇒`. (المتوقع: تثبيت Keras بالإصدار الدقيق لأنه الواجهة التي يعتمد عليها الكود بشدّة، والنطاقات لبقية الأدوات.)

3. **فحص بيئي.** أضف إلى سكربت التحقّق فحصًا يُبلّغ عن عدد النوى واسم الجهاز، ثم اشرح كيف تقرأ منه هل GPU مستخدَم فعلًا. (المتوقع: سطر `Device` المضبوط على cuda يعني استعمالًا فعليًا؛ `cpu` يعني أن المحرّك لا يرى GPU.)

4. **اختيار المحرّك.** ثلاثّ سيناريوهات: (أ) خدمة تصنيف صور على خادم إنتاجي، (ب) تدريب نموذج بمئة مليار معامل على مجموعة distributed، (ج) استدلال على جهاز لوحي بمعالج Intel. اختر لكلٍّ محرّكًا مع التبرير بمعيار. (المتوقع: (أ) TensorFlow، (ب) JAX، (ج) OpenVINO أو TensorFlow مع LiteRT.)

5. **مصفوفة التوافق.** صمّم جدولًا يربط كل حزمة بأثر ترقية Keras عليها: `keras` , `tensorflow-text` , `keras-cv` , `keras-hub` , `torch` , `jax` , `tensorflow` . (المتوقع: الانتباه إلى أن `keras-cv` أُرشف ودمجت وظائفه في Keras، وأن `keras-hub` مرتبط بإصدار Keras.)

6. **حدّد الخطأ.** هذا السكربت يفشل عند `model.save("_smoke.keras")` :

```python
    model = keras.Sequential([keras.layers.Input((4,)),
keras.layers.Dense(2)])
    model.save("_smoke.h5")
```

اشرح سبب الخطأ، وصحّحه بطريقتين (تغيير الامتداد، وتغيير الصيغة عمدًا). (المتوقع: صيغة `.keras` هي الموصى بها في Keras 3 لأنها تحفظ البنية والtraining state والأوزان معًا؛ صيغة `.h5` تحفظ الأوزان فقط.)

7. **زمن البدء.** قِس زمن أول `import keras` في بيئتك. ثم قارنه بزمن التشغيل الثاني في العملية نفسها. فسّر الفرق. (المتوقع: الفارق يعود إلى كلفة تحميل المكتبات الأصلية؛ في JAX قد يظهر كلفة ترجمة الدوال.)

8. **عزل عملي.** أنشئ مشروعين في المجلّد نفسه: أحدهما يحتاج `keras[torch]` والآخر `keras[jax]`. اكتب الأوامر بالترتيب، واشرح أي ملف يُعدَّل عند التبديل. (المتوقع: وجود بيئتين مستقلّتين و/أو ملف `requirements.txt` لكل مشروع؛ لا حاجة لتعديل أي ملف عالمي.)

9. **خط أنابيب.** اكتب ملف `setup.sh` (ولينكس/ماك) أو `setup.ps1` (ويندوز) يؤدّي: إنشاء البيئة، التفعيل، تثبيت الإصدارات، وتشغيل `smoke_test.py`. (المتوقع: السكربت ينتهي برمز خروج `0` فقط إذا نجحت كل الفحوص.)

10. **فخّ شائع.** أضِف إلى `smoke_test.py` سطرًا يطبع `keras.backend.standardize_dtype("float16")` ثم سطرًا يطبع `keras.backend.standardize_dtype("float64")`. فسّر لماذا يعيد المحرّك `float16` في الحالة الأولى و `float32` في الثانية. (المتوقع: توحيد الدقة يمنع عملاً عدديًا غير مستقر على المحرّكات التي لا تدعم float64.)

ملخص الفصل

- Keras 3 **لا يجلب** محرّكًا؛ ثبّت `keras[backend]` أو المحرك منفصلًا.
- استخدم **بيئة معزولة** لكل مشروع، و**ثبّت الإصدارات** بشدّة.
- تحقّق من البيئة بسكربت تشخيص صغير **قبل** كتابة كود النموذج.
- `KERAS_BACKEND` قبل الاستيراد، ولا تُبدّل المحرّك بعد بناء نموذج.
- اختبر الـ GPU بسطر `Device` ، لا بالافتراض.
- Colab للتجربة، والجهاز المحلي للإنتاج.

الفصل 3: الأوتور: اللغة التي يفكّر بها Keras

بطاقة الفصل

الفصل 3

مهمّة هذا الفصل

يشرح لغة Keras الداخلية: الأوتور وأبعاده، قواعد البثّ، التدرّج التلقائي، ولماذا وُجدت keras.ops.

ماذا تكسب منه

تقرأ رسائل أخطاء الأشكال وتفهمها، وتكتب عمليات تعمل على أي محرّك دون إعادة كتابة.

الدالة	دورها	متى تستخدمها
`keras.ops.array()`	بناء أوتور من قائمة أو مصفوفة NumPy	عند تحويل بيانات جاهزة إلى مدخلات نموذج
`keras.ops.convert_to_tensor()`	تحويل آمن مع تحديد النوع	قبل تمرير بيانات خارجية إلى النموذج
`keras.ops.matmul()`	ضرب المصفوفات	لفهم ما تفعله Dense داخليًّا
`keras.ops.gradients()`	حساب المشتقات تلقائيًّا	عند بناء حلقة تدريب مخصّصة
`keras.Variable`	مصفوفة قابلة للتحديث	للأوزان والحالات في الطبقات المخصّصة

قصة مفتاحية: «لماذا يصرّ المشغّل على أن الشكل يجب أن يكون `(None, 784)` وليس `(784,)`؟» سؤال بدا ساذجًا، لكن الإجابة عنه غيّرت نظرتك إلى الشبكات كلّها. البُعد الأول ليس عددًا من البيانات، بل **عدد الدفعات** – وقد استغرقت شهرًا لأفهم أن Keras لا تشرح لك الأبعاد، بل تفرض عليك فهمها.

أهداف الفصل

- [ ] فهم بنية الأوتور: الرتبة والشكل والنوع والجهاز
- [ ] إتقان قواعد البث ومنع أخطاء الأبعاد
- [ ] فهم كيف يحسب Keras التدرّج التلقائي
- [ ] التمييز بين `keras.ops` وNumPy ولماذا يُستعملان معًا
- [ ] معرفة الفرق بين المتغيّرات القابلة للتعلّم وغير القابلة للتعلّم

3.1 ما هو الأوتور؟

الأوتور (Tensor) هو مصفوفة أرقام multiaبعاد. إذا كانت المصفوفة العادية (NumPy array) هي «صفًا من الأرقام»، فالأوتور هو **مكعّب** من الأرقام له عدد أوجه (رتبة) وامتداد في كل وجه (شكل).

الرتبة والشكل

الرتبة (rank) هي عدد الأبعاد، والشكل (shape) هو tuple يحدّد طول كل بُعد.

الرمز	المعنى	مثال
الرتبة 0	عدد مُفرد	`3.14`
الرتبة 1	متجه	`[1, 2, 3]` – شكله `(3,)`
الرتبة 2	مصفوفة	جدول 3×2 – شكله `(3, 2)`
الرتبة 3	مكدّس	4 صور رمادية 28×28 – شكله `(4, 28, 28)`
الرتبة 4	دفعة صور	4 صور ملوّنة 28×28 – شكله `(4, 28, 28, 3)`

في Keras، الرتبة 4 هي الرتبة السائدة في الرؤية الحاسوبية: **(دفعة، ارتفاع، عرض، قنوات)**.

لماذا `None` في الأول؟

حين تستدعي `model.summary()` سترى:

```text
Layer (type)                 Output Shape              Param #
=================================================================
input_1 (InputLayer)         (None, 784)               0
dense (Dense)                (None, 128)               100480
```

الـ `None` في البُعد الأول لا تعني «لا شيء»؛ تعني **«غير محدَّد بعد»** – أي «أي عدد من الدفعات». هذا هوricketa الكتابة التي تمنح النموذج مرونة: يمكنك تدريبه على 32 عيّنة واستدعاؤه على 1 عيّنة أو 10 آلاف، بلا تغيير في الشكل.

> **نصيحة:** لا تفهم `None` على أنها «نقص في البيانات». هي **بُعد حرّ** – هذا هو الفرق بين نموذج عامّ ونموذج مرتبط بحجم دفعة محدّد.

النوع والجهاز

```python
import keras
import numpy as np

x = np.random.rand(4, 28, 28, 3).astype("float32")
t = keras.ops.convert_to_tensor(x)

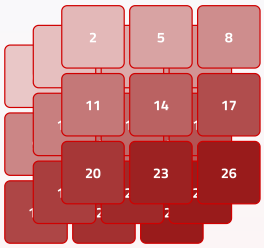
print(t.shape)   # (4, 28, 28, 3)
print(t.dtype)   # float32
```

النوع (dtype) يحدّد: - **مدى القيم**: `uint8` من 0 إلى 255، و `float32` بقيم تقريبية واسعة. - **حجم التخزين**: `float32` أربع بايتات، و `float16` بايتان. - **سلوك التقريب**: `float32` يتّسع لنحو 7 خانات عشرية.

الجهاز (device) يحدّد مكان التخزين والحساب: `cpu` أو `gpu:0` . في Keras 3 يُفترض أن الأوتور يتبع جهاز النموذج تلقائيًا.

> **تحذير:** Keras 3 يعمل افتراضيًا بـ `float32` ، لا بـ `float16` . هذا مقصود: الشبكة العصبية حسّاسة عدديًا، والدقة المنخفضة غير المختلطة تُنتج `NaN` في طبقات مثل `softmax` . الفصل 36 يتناول الدقة المختلطة بالتفصيل.

الأوتور: صندوق أرقام بأبعاد



شكل (3, 3, 3)

٣ محاور × ٣ صفوف × ٣ أعمدة = ٢٧ قيمة

كل محور يُسمّى dim ويُكتب من اليمين لليسار في TensorFlow

المفهوم	الرماد في Keras	مثال
الرتبة (Rank)	ndim	3
الشكل (Shape)	shape	(32, 224, 224, 3)
النوع (DType)	dtype	float32
الجهاز (Device)	device	cpu / gpu
القيمة (Value)	()numpy	[[1.0, 2.0]]

شكل 5 – شبكة الأوتور: الرتبة والشكل والأبعاد

3.2 البث: كيف تتوسّع الأبعاد

البث (Broadcasting) يسمح بعمليتين مختلفتين في الشكل دون تعديل صريح. القاعدة في Keras 3 (NumPy وTensorFlow) تُطبَّق **من اليمين إلى اليسار**:

1. إن كان البُعدان متساويين، تجراهما كما هما.
2. إن كان أحدهما `1` ، وسّعه إلى قيمة الآخر.
3. إن كان أحدهما `None` (حرّ)، وسّعه إلى قيمة الآخر.
4. عدا ذلك: خطأ.

مثال عملي: إضافة متجه إلى مصفوفة

```python
import numpy as np

# مصفوفة 3×2 ومتجه طول 2
a = np.array([[1., 2.], [3., 4.], [5., 6.]])   # (3, 2)
b = np.array([10., 20.])                       # (2,)

# يعمل — لكن ليس بالطريقة التي يتوقّعها المبتدئ
print(a + b)
# [[11. 22.]
#  [13. 24.]
#  [15. 26.]]
```

انظر: المتجه `b` طُويت على طول الصفوف (البُعد الأخير)، لا على طول الأعمدة. والسبب أن رتبة `b` هي 1، وKeras/Numpy تعامل الرتبة المنخفضة على أنها **محاذاة لليمين** (trailing alignment).

الحالة التي تفاجئ الجميع

```python
a = np.array([[1.], [2.], [3.]])   # (3, 1) عمود
b = np.array([[10., 20.]])         # (1, 2) صف

print(a + b)
# [[11. 21.]
#  [12. 22.]
#  [13. 23.]]
```

هذه ليست إضافة متجه إلى متجه، بل **بثّ خارجي**: عمود `(3, 1)` وصفّ `(1, 2)` يلتقيان فينتج جدول `(3, 2)` كامل. هذه بالضبط اللحظة التي يكتشف فيها المطوّر الجديد أن فهم الأبعاد ليس تفصيلًا.

متى يفشل البث؟

```python
a = np.random.rand(3, 1)    # (3, 1)
b = np.random.rand(4, 5)    # (4, 5)
# a + b  →  ValueError
```

المقارنة من اليمين: `1` مقابل `5` ← يُوسَّع `1` إلى `5` ✓. ثم `3` مقابل `4` ← لا أحدهما `1` وليسا متساويين ✗. خطأ.

القاعدة العملية

في Keras، تُكتب الأشكال دائمًا **مع بُعد الدفعة أولًا**. فإذا كان لديك 32 عيّنة، كل شيء يبدأ بـ `32` :

```python
x = np.random.rand(32, 784).astype("float32")    # 32 عيّنة، 784 خاصية
```



شكل 6 – قاعدة البثّ: كيف تتوسّع الأبعاد تلقائيًّا

3.3 التدرّج التلقائي: كيف تتعلّم الشبكة؟

الرسم البياني للحساب

كل ما تفعله الشبكة هو سلسلة تحويلات. تصبح هذه التحويلات **رسمًا بيانيًا** موجّهًا: العُقد هي العمليات، والحواف هي تدفّق البيانات.

في هذا الرسم، لكل حافة **قيمة** (للمُدخل) ولكل حافة **تدرّج** (مشتقّ الخسارة بالنسبة لتلك القيمة).

التمرير الأمامي

يُحسب الناتج من المُدخل:

$$z = Wx + b, \qquad a = f(z)$$

التمرير العكسي

يعود من الخسارة نحو المُدخلات بجمع تدرّجات الأجزاء بماعدة السلسلة:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial W}$$

donde: $\frac{\partial L}{\partial a}$ هو تدرّج الخسارة بالنسبة لناتج الطبقة، و$\frac{\partial a}{\partial z}$ هو مشتقّ دالة التنشيط، و$\frac{\partial z}{\partial W}$ هو التدرّج بدلالة الأوزان.

قاعدة السلسلة في صورة أبسط

تخيّل سلسلة من ثلاث حلقات: $L = a \cdot b \cdot c$. كم يتغيّر L إذا تغيّر a؟

$$\frac{\partial L}{\partial a} = b \cdot c$$

الضرب simply. والآن تخيّل سلسلة من ثلاث حلقات لكن كل حلقة هي دالة:

$$L = f(g(h(x)))$$

إذن:

$$\frac{dL}{dx} = \frac{dL}{df} \cdot \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{dx}$$

هذا هو كل التدرّج التلقائي. لا أكثر ولا أقل. Keras لا تفعل شيئًا سحريًا: هي تملك الرسم البياني، وتُنفّذ هذه الضربيات نيابةً عنك.

كيف تتحكّم في الرسم؟

PYTHON

```python
import keras

x = keras.ops.convert_to_tensor([[1.0, 2.0]])
w = keras.ops.convert_to_tensor([[1.0], [1.0]])

with keras.name_scope("forward"):
    z = keras.ops.matmul(x, w)       # الجداء
    a = keras.ops.relu(z)            # التنشيط
    loss = keras.ops.sum(a)          # «الخسارة» البسيطة

print("Loss:", float(loss))
print("Forward value (z):", z)
```

ملاحظة مهمة: لا توجد دالة `keras.ops.gradients` في Keras 3، فلا تبحث عنها في التوثيق. التدرّجات تُحسب تلقائيًا داخل خطوة التدريب (`train_step`) ولا تُعرَض في الواجهة العامة؛ إن احتجتَ تدرّجًا يدويًا فاستعمل أداة التفاضل الخاصّة بالمحرّك (`jax.grad` أو `torch.autograd.grad`). وما توفّره Keras نفسها هو `keras.ops.custom_gradient`: مزخرف يُتيح لك كتابة تدرّج صريح لعملية ما بدل أن يتولّاه المحرّك تلقائيًا.

```python
import keras


@keras.ops.custom_gradient
def cube(x):
    """f(x) = x^3 مع تدرّج صريح مخصّص."""

    def grad(*args, upstream=None):
        if upstream is None:
            (upstream,) = args
        return 3.0 * x**2 * upstream

    return x**3, grad


v = keras.ops.convert_to_tensor(2.0)
print("f(2) =", float(cube(v)))   # النتيجة المتوقعة: 8.0
```



شكل 7 – الرسم البياني للحساب: التمرير الأمامي والعكسي

3.4 لِماذا وُجدت؟ `keras.ops`

Keras 3 مبنية على فكرة **واجهة واحدة فوق محرّكات مختلفة**. ولهذا لا يمكنك الاعتماد على `numpy` مباشرة داخل طبقة: عملية NumPy تعمل على معالج Python خارج تسريع المحرّك.

لذلك وُجدت `keras.ops` : مكتبة عمليات تعمل على **أي** محرّك مدعوم.

```python
import keras

x = keras.ops.array([1.0, 2.0, 3.0])

keras.ops.softmax(x)       # تطبيع احتمالي
keras.ops.relu(x)          # تنشيط
keras.ops.matmul(x, x.T)   # ضرب مصفوفي
keras.ops.mean(x)          # متوسط
keras.ops.argmax(x)        # فهرس أكبر قيمة
```

القاعدة: داخل نموذج أو دالة خسارة مخصّصة، استخدم `keras.ops` لا NumPy. التسريع والنقل بين المحرّكات يبقيان سليمين.

> **نصيحة:** عند قراءة شيفرة Keras نفسها ستجد `keras.ops` في كل مكان. تعلّمها مبكّرًا — فهي لغة Keras الحقيقية.

3.5 المتغيّرات: ما الذي يتغيّر أثناء التدريب؟

في Keras 3، كل حالة قابلة للتعلّم داخل النموذج هي **متغيّر** (`keras.Variable`).

```python
import keras

v = keras.Variable([1.0, 2.0, 3.0], name="weights")
print(v)              # <Variable shape=(3,), dtype=float32, value=[1., 2., 3.]>
print(v.shape, v.dtype)
```

القابل للتعلّم وغير القابل

```python
model = keras.Sequential([
    keras.layers.Input((8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(1),
])

print("قابل للتعلّم   :", len(model.trainable_variables))
print("غير قابل للتعلّم:", len(model.non_trainable_variables))
for w in model.non_trainable_variables:
    print("   •", w.name, w.shape)
```

> **نصيحة:** `BatchNormalization` هو المثال الأوضح. أثناء التدريب يحسب متوسط وتباين الدفعة، ثم يحتفظ بهما كمتغيّرات **غير قابلة للتعلّم** (تُحدَّث بمعدّل متحرّك لا بالمُحسّن). أثناء الاستدلال لا توجد دفعة، فيستخدم هذه القيم المحفوظة. لو لم تكن موجودة، لما كان بالإمكان التنبّؤ على عيّنة واحدة.

كيف يُحدَّث المتغيّر؟

عبر `assign` أو `assign_add`، وهما الطريق الوحيد الآمن:

```python
v.assign([4.0, 5.0, 6.0])      # استبدال كامل
v.assign_add([1.0, 1.0, 1.0]) # جمع
```

3.6 أخطاء شائعة في هذا الفصل

خطأ 1 – نسيان بُعد الدفعة. العرض: `ValueError: Expected the last dimension to be 784`. السبب: مرّرت مصفوفة شكلها `(784,)` بينما النموذج يتوقّع `(None, 784)`. الحل: `x = x[np.newaxis, :]` أو `keras.ops.expand_dims(x, 0)`.

خطأ 2 – استخدام NumPy داخل دالة خسارة مخصّصة. العرض: بطء ملحوظ، أو خطأ عند التشغيل على محرّك مختلف. السبب: `numpy` لا يعرف المحرّك. الحل: استبدل بـ `keras.ops`.

خطأ 3 – تعديل المتغيّر مباشرة. العرض: `ValueError: Attempting to update a variable in-place`. السبب: المكتبات الحديثة التدرّج التلقائي تمنع التعديل المباشر. الحل: `v.assign(...)` أو `v.assign_add(...)`.

خطأ 4 – الخلط بين الأبعاد في الإضافة. العرض: `operands could not be broadcast together`. السبب: افتراض أن المتجه يُضاف على الأعمدة. الحل: أضف `axis` صراحةً أو أعد تشكيل المدخلات.

خطأ 5 – توقّع `None` أن يعني صفرًا. الخلط: تمرير دفعة فارغة. الحل: `None` يعني «أي عدد»، والصفر يعني «لا شيء» – وهو خطأ مختلف تمامًا.

خطأ 6 – الخلط بين `float64` و `float32`. العرض: بطء في الحساب، أو `NaN` في الطبقات الأخيرة. الحل: `keras.ops.cast(x, "float32")`.

التمارين

1. **رتبة وشكل.** احسب رتبة وشكل كلٍّ من: `np.zeros(5)` ، `np.zeros((3, 4))` ، `np.zeros((8, 28, 28, 3))` ، `np.zeros((2, 3, 4))` . ثم اشرح أيّها يقابل دفعة صور. (المتوقع: الرتب 1، 2، 3، 4 على التوالي؛ الأخير هو دفعة 8 صور ملوّنة.)

2. **قواعد البث.** حدّد ناتج كل عملية: `(5,)` ، `(5, 3, 4) + (3, 1)` ، `(5, 3, 4) + (4,)` ، `(2, 3) + (2, 3)` ، `(3, 4) + (5, 1, 1)` . (المتوقع: الأولى والثانية والرابعة تعمل؛ الثالثة تفشل لأن `5` لا يمكن بثّه على `1` في نفس المستوى.)

3. **إصلاح بثّ.** هذا الكود يُنتج نتيجة غير متوقّعة:

```python
a = np.arange(6).reshape(3, 2)
b = np.array([100, 200])
print(a + b)
```

اشرح النتيجة، ثم اكتب تصحيحًا يُضيف `b` إلى **كل صف** كما قصدنا. (المتوقع: البث يوسّع `b` على البُعد الأخير فيُضاف إلى الأعمدة؛ التصحيح: `a + b[None, :]` أو `np.add(a, b[:, None])` لإضافة `b` إلى كل صف.)

4. **التدرّج يدويًا.** احسب تدرّج $L = 3w^2$ عند $w = 2$، وتحقّق من النتيجة عدديًا (بالفرق المحدود) وبواسطة `keras.ops.custom_gradient` . (المتوقع: $\frac{dL}{dw} = 6w = 12$.)

5. **قاعدة السلسلة.** احسب $\frac{dL}{dx}$ لـ $L = \sin(x^2)$ عند $x = 1$، ثم أكّد النتيجة عدديًا. (المتوقع: $\frac{dL}{dx} = 2x\cos(x^2) = 2\cos(1) \approx 1.0806$.)

6. **مقارنة المتغيّرات.** ابنِ نموذجًا فيه `Dense` و `BatchNormalization` ، ثم اطبع `len(model.trainable_variables)` و `len(model.non_trainable_variables)` واشرح كل مجموعة. (المتوقع: المتغيّرات القابلة هي أوزان Dense وتحيزاتها؛ غير القابلة هي متوسّط BatchNorm وتباينه، أي اثنان لكل طبقة BN.)

7. **أين المتغيّر؟** أضف `keras.layers.Layer` مخصّصة بمتغيّر داخلي، ثم اطبع `model.trainable_variables` قبل وبعد `model.fit`. (المتوقع: عدد المتغيّرات لا يتغيّر – التدريب يحدّث القيم لا البنية.)

8. **dtype.** قارن `np.float32` و`np.float64` في عملية على 10 ملايين عنصر: الوقت والحجم. ثم اشرح لماذا يعتمد Keras 3 على `float32`. (المتوقع: `float64` أبطأ بمرّتين تقريبًا وأكبر حجمًا، و`float32` كافٍ لدقة التدريب المعتادة.)

9. **سؤال مفاهيمي.** لماذا يُعدّ `None` بُعدًا حرًّا لا قيمةً ناقصة؟ ما الذي يمنعه من أن يسبّب خطأ عند الاستدعاء بدفعة مختلفة الحجم؟ (المتوقع: لأنه رمز "أي عدد"، فيُوسَّع تلقائيًا إلى حجم الدفعة الفعلي.)

10. **اكتشاف الخلل.** هذا السكربت يعطي `ValueError`:

PYTHON

```
import keras
x = keras.ops.array([1.0, 2.0])
w = keras.ops.array([1.0, 2.0])
y = keras.ops.matmul(x, w)
```

يطلب `matmul` متجهين (المتوقع: `matmul`). اشرح السبب بدقّة، وأصلحه بطريقتين: تغيير `w` متجهًا أو جعل `keras.ops.dot` متساويي الطول (مصفوفة × متجه) لا (متجه × متجه)؛ التصحيح صفيًّا.)

ملخص الفصل

- الأوتور = رتبة + شكل + نوع + جهاز؛ و `None` تعني «أي عدد».
- البث يُطبَّق من اليمين: تساوٍ أو `1` أو حرّ؛ وعندها يقع الخطأ.
- التدرّج التلقائي = قاعدة السلسلة، نفّذتها Keras نيابةً عنك.
- `keras.ops` هي لغة Keras الحقيقية – استخدمها لا NumPy.
- المتغيّرات القابلة للتعلّم تُحدّثها المُحسّنات، وغير القابلة تُحدّثها القواعد (مثل إحصاءات BatchNorm).
- Keras 3 يعمل بـ `float32` افتراضيًا، وهذا مقصود.

الفصل 4: بنية النموذج: ثلاث طرق لتحديد الشبكة

بطاقة الفصل

الفصل 4

مهمّة هذا الفصل

يقارن ثلاث واجهات لبناء النموذج – Sequential وFunctional وSubclassing – ويعطي قاعدة اختيار صريحة بينها.

ماذا تكسب منه

تبني النموذج بالواجهة المناسبة من أول مرة، وتتجنّب إعادة الكتابة عند إضافة فرع أو مخرج ثانٍ.

الدالة	دورها	متى تستخدمها
`keras.Sequential`	تتابع خطّي بسيط	لشبكات بلا فروع ولا مدخلات متعدّدة
`keras.Input`	تعريف شكل المدخل واسمه	أول خطوة في الواجهة الوظيفية
`keras.Model(inputs, outputs)`	بناء رسم بياني صريح	عند وجود فروع أو مخرجات متعدّدة
`keras.layers.Concatenate`	دمج فروع في متجه واحد	في البنى متعدّدة المدخلات
`keras.saving.register_keras_serializable`	تسجيل صنف مخصّص للحفظ	عند حفظ نموذج يحوي طبقة مخصّصة

قصة مفتاحية: نموذج يعمل في دفتر Jupyter، لكن حفظه يفشل. السبب لم يكن خطأً في الشيفرة، بل في **البنية**: بُني النموذج بطريقة لا تحفظ نفسها. في هذا الفصل نختار طريقة البناء بعناية، لأن الطريقة نفسها تحدّد ما يمكنك فعله لاحقًا: الحفظ، والتصدير، وإعادة الاستخدام.

أهداف الفصل

- [ ] التمييز بين الواجهات الثلاث: Subclassingg Sequentialg Functional
- [ ] اختيار الواجهة المناسبة لكل موقف
- [ ] إتقان بناء النماذج متعددة المدخلات والمخرجات
- [ ] فهم مشاركة الأوزان والفخّ الذي يحملها
- [ ] معرفة الثمن الذي تدفعه عند الوراثة الصريحة

4.1 ثلاث طرق، ثلاث قدرات

حين كتب François Chollet الفصل الأول من كتاب **Deep Learning with Python**، كان أمامه خيار: أن يقدّم Keras كواجهة واحدة أم كثلاث واجهات مختلفة. اختار الثانية – بحجّة أن القيود المفروضة على المستخدم يجب أن تعكس قيود ما يمكن فعله حقًا.

المسح السريع

المعيار	Sequential	Functional	Subclassing
سهولة الكتابة	الأعلى	متوسطة	الأدنى
مدخلات متعدّدة	✗	✓	✓
مخرجات متعدّدة	✗	✓	✓
فروع وتفرّعات	✗	✓	✓
مشاركة الأوزان	✗	✓	✓
حلقة تدريب مخصّصة	محدود	محدود	✓
حفظ `.keras` كاملًا	✓	✓	يحتاج `get_config`
تصدير سهل	✓	✓	يحتاج عملًا إضافيًّا
مناسب للبحث	✗	✗	✓

ثلاثة أساليب — واحدٌ صحيح لكل موقف

Subclassing — تخصيص حر
مرونة قصوى وتحكّم كامل
للأبحاث والبنى الفريدة
أصعب في الحفظ والتصدير

Functional — رسم بياني
أقوى خيار لأغلب المشاريع
يدعم مدخلات/مخرجات متعددة
سهل التصدير والتسلسل

Sequential — تتابع خطي
أبسط صياغة، طبقة بعد طبقة
مثالي للمبتدئين وMLP
لا يدعم تفرّعات متعددة

قاعدة اختيار عملية

ابدأ دائمًا بـ Functional.

إن لم تستطع التعبير عن بنية الشبكة برسم بياني، فأنت أمام حالة خاصة

لا تعقّد النموذج: تعقيد البنية يجلب تعقيد الحفظ والاختبار والنشر.

القاعدة العملية: Functional أولاً · Sequential للمبتدئين · Subclassing عند الضرورة.

شكل 8 – ثلاث واجهات لبناء النماذج: Sequential Functional Subclassing

4.2 الواجهة الأولى: Sequential

المتّسلسل يعني: طبقة بعد طبقة، دون فروع.

```python
import keras

model = keras.Sequential([
    keras.layers.Input(shape=(784,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(10),
])

model.summary()
```

متى تستخدمه؟ حين تكون الشبكة خطّية سلسلة: طبقة تلي طبقة. هذا يغطّي Majority من شبكات الـ MLP البسيطة.

> **ملاحظة:** لا يمكن لـ Sequential إضافة مدخلين. إن حاولت تمرير قائمتين، ستحصل على خطأ. وهذا مقصود: الـ Sequential يجبرك على البساطة، وهي ميزة في المرحلة الأولى.

4.3 الواجهة الثانية: Functional — الأصل الحقيقي

في الـ Functional، كل طبقة **تُعطى مدخلات صريحة** وتُنتج **مخرجات مسمّاة**. ثم تجمع هذه المخرجات في `keras.Model`.

```python
import keras

inputs = keras.Input(shape=(784,), name="input_layer")
x = keras.layers.Dense(128, activation="relu")(inputs)
x = keras.layers.Dropout(0.2)(x)
x = keras.layers.Dense(64, activation="relu")(x)
outputs = keras.layers.Dense(10, name="output_layer")(x)

model = keras.Model(inputs=inputs, outputs=outputs)
```

لاحظ الفرق الجوهري: `(inputs)` بعد تعريف الطبقة. هذه هي **استدعاء الطبقة على مُدخل**. كل نداء يُنتج "عقدة" في الرسم البياني.

لماذا هذه الواجهة هي «الأصل»؟

لأن Keras 3 نفسه مبني عليها. حين ترى `model.summary()` يعرض طبقات فيها `Connected to: ['input_layer']`، فأنت ترى لغة الرسم البياني مباشرة.

الفرعين: حالة لا يستطيع Sequential التعبير عنها

PYTHON

```python
import keras

# مدخلان
img_in = keras.Input(shape=(32, 32, 3), name="image")
meta_in = keras.Input(shape=(5,), name="metadata")

# معالجة كل فرع
img_feat = keras.layers.Conv2D(16, 3, activation="relu")(img_in)
img_feat = keras.layers.GlobalAveragePooling2D()(img_feat)
meta_feat = keras.layers.Dense(16, activation="relu")(meta_in)

# الدمج
merged = keras.layers.Concatenate()([img_feat, meta_feat])
out = keras.layers.Dense(1, activation="sigmoid")(merged)

model = keras.Model(inputs=[img_in, meta_in], outputs=out)
```

فرع واحد بمخرجين

PYTHON

```python
import keras

inputs = keras.Input(shape=(100,))
shared = keras.layers.Dense(64, activation="relu")(inputs)

head_a = keras.layers.Dense(3, activation="softmax", name="task_a")(shared)
head_b = keras.layers.Dense(1, activation="sigmoid", name="task_b")(shared)

model = keras.Model(inputs=inputs, outputs=[head_a, head_b])
```

فخّ مشاركة الأوزان

هذا هو الفخّ الذي يوقع المبتدئين، وهو **مقصود** في التصميم:

```python
import keras

inputs = keras.Input(shape=(10,))
layer = keras.layers.Dense(8, activation="relu")

a = layer(inputs)    # نداء أول
b = layer(inputs)    # نفس الطبقة — نداء ثانٍ!

print(layer.weights[0].shape)    # (10, 8) — مصفوفة واحدة فقط
```

تحذير: النداءان لطبقة واحدة **لا ينشئان طبقتين**. تتحمّل الطبقة نفسها الأوزان، وتُشارَك في المسارين. هذا هو معنى «مشاركة الأوزان» (weight sharing) — وهو بالضبط ما تحتاجه في الشبكات مثل Siamese. وإن أردت طبقتَين مستقلّتين، فعّل `layer_a = Dense(8)` و`layer_b = Dense(8)` منفصلتين.

الـ Functional API: تدفّق الرسم البياني



كل ناتج يُسمّى، وكل ناتج يُمرَّر إلى الطبقة التالية — والمخرجات تُجمَع في قائمة.

شكل 9 – الرسم البياني الوظيفي: العُقد والحواف والتفريعات

4.4 الواجهة الثالثة: Subclassing

حين تحتاج منطقًا لا يمكن أن يصفه الرسم البياني – مثل حلقة تكرارية بمتغيّر، أو تمرير إضافي، أو تحكّم داخلي في التدرّج – Careersرث الصريح هو الحل.

PYTHON

```python
import keras

class MyModel(keras.Model):
    def __init__(self, units=64, **kwargs):
        super().__init__(**kwargs)
        self.dense1 = keras.layers.Dense(units, activation="relu")
        self.dense2 = keras.layers.Dense(units, activation="relu")
        self.out = keras.layers.Dense(1)

    def call(self, x, training=False):
        # منطق مخصّص: Residual connection على مرحلتين
        h = self.dense1(x)
        h = self.dense2(h)
        return self.out(x + h)
```

القاعدة الذهبية: أنشئ الطبقات في `__init__` لا في `call`

PYTHON

```python
# ✗ خطأ: إنشاء طبقة داخل call
def call(self, x):
    d = keras.layers.Dense(64)    # خطأ: طبقة جديدة في كل استدعاء!
    return d(x)

# ✓ صح: أنشئها مرة واحدة في __init__
def __init__(self):
    super().__init__()
    self.d = keras.layers.Dense(64)
```

لماذا؟ لأن الطبقات المنشأة في `call` **لا تُسجَّل** في النموذج: لا تظهر في `summary`، ولا تُحفظ مع الأوزان، ولا يعرف المُحسّن أن يحدّثها. النتيجة نموذج «يشتغل» ولا يتعلّم.

بنية النموذج: `build` و `compute_output_shape`

PYTHON

```python
class ScaledDense(keras.layers.Layer):
    def __init__(self, units, scale=2.0, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.scale = scale

    def build(self, input_shape):
        self.kernel = self.add_weight(
            shape=(input_shape[-1], self.units),
            initializer="glorot_uniform",
            trainable=True,
            name="kernel",
        )
        super().build(input_shape)

    def call(self, inputs):
        return inputs @ self.kernel * self.scale

    def get_config(self):
        config = super().get_config()
        config.update({"units": self.units, "scale": self.scale})
        return config
```

ملاحظة: `build()` تُستدعى مرّة واحدة عند أول تمرير، وتُنشئ الأوزان. و `get_config()` هي ما يسمح لـ Keras بإعادة بناء الطبقة عند التحميل – بدونها لا يمكن حفظ النموذج.

4.5 مقارنة تفصيلية: أيّهما تختار؟

السؤال	إن كان جوابك «نعم»	اختر
هل أحتاج مدخلات متعدّدة أو مخرجات متعدّدة؟	✓	Functional
هل أحتاج فروعًا أو مسارات متعدّدة؟	✓	Functional
هل أريد أن أحفظ النموذج وأعيد تحميله بسهولة؟	✓	Functional
هل أحتاج مشاركة أوزان عبر فروع؟	✓	Functional
هل أحتاج حلقة تدريب مخصّصة أو خسرًا غير قياسي يتعلّم؟	✓	Subclassing
هل أحتاج تمرير معلومات إضافية داخل `call` ؟	✓	Subclassing
هل هي شبكة خطّية بسيطة للتجربة السريعة؟	✓	Sequential

التسلسل العملي الموصى به

1. **ابدأ بـ Sequential** للنموذج الأول: تعلّم الحركة بلا تعقيد.
2. **انتقل إلى Functional** فور حاجتك لتفرّع أو مدخلات متعدّدة – وهي الحالة الغالبة في المشاريع الحقيقية.
3. **لا تلجأ إلى Subclassing** إلا حين يعجز الرسم البياني عن تمثيل فكرتك، وعندها كافئ نفسك: هل تحقّقت من قابلية الحفظ والتسلسل (serialization)؟

> **قاعدة:** نموذج يمكن حفظه واستخدامه بعد عام أفضل من نموذج أدقّ بنصف نقطة مئوية ولا يمكن حفظه.

4.6 مثال محلول: نموذج متعدّد المهام

المطلوب: تصنيف المنتج إلى ثلاث فئات **و** تقدير سعره. المخرجان يتفرّعان من م مشترك.

PYTHON

```
import keras

def build_model(input_dim=784, hidden=128):
    # ── الجذع المشترك
    inputs = keras.Input(shape=(input_dim,), name="input")
    x = keras.layers.Dense(hidden, activation="relu", name="shared_1")
(inputs)
    x = keras.layers.BatchNormalization(name="bn_1")(x)
    x = keras.layers.Dropout(0.3, name="drop_1")(x)
    x = keras.layers.Dense(hidden // 2, activation="relu", name="shared_2")
(x)

    # ── الرأسان
    cls = keras.layers.Dense(3, activation="softmax", name="category")(x)
    reg = keras.layers.Dense(1, name="price")(x)

    model = keras.Model(inputs=inputs, outputs=[cls, reg],
name="multi_task")
    return model


model = build_model()
model.summary(line_length=78)
```

التدريب: كل مخرج يحتاج خسارته ومقياسه:

```python
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss={
        "category": keras.losses.CategoricalCrossentropy(),
        "price": keras.losses.MeanSquaredError(),
    },
    metrics={
        "category": [keras.metrics.CategoricalAccuracy()],
        "price": [keras.metrics.MeanAbsoluteError()],
    },
)

history = model.fit(
    x_train, {"category": y_category, "price": y_price},
    epochs=10, batch_size=64, validation_split=0.2,
)
```

ملاحظة مهمة: عندما يكون المخرجان قاموسًا، يجب أن تكون مفاتيح القاموس **أسماء المخرجات** كما في `keras.Model(outputs=[cls, reg])` – وإلا أخطأ Keras في مطابقة الخسائر.

4.7 الأخطاء الشائعة

خطأ 1 – تمرير قائمة إلى Sequential. العرض: `ValueError: The model expects a single input`. الحل: انتقل إلى Functional.

خطأ 2 – إنشاء طبقة داخل `call`. العرض: النموذج لا يتعلّم؛ الخسارة ثابتة. الحل: أنشئ الطبقات في `__init__`.

خطأ 3 – توقّع مستقلّية النداء. العرض: طبقتان seemed مستقلّتان لكنهما تتشاركان الأوزان. الحل: أنشئ كائنَي Dense منفصلين.

خطأ 4 – نسيان `get_config`. العرض: `ValueError: Could not locate class 'MyLayer'`. الحل: نفّذ `get_config` وسجّل الطبقة بـ

`@keras.saving.register_keras_serializable(package="MyPkg")` .

خطأ 5 – خلط المفتاح والقيمة في `fit` مع مخرجات قائمة. العرض: `KeyError: 'category'`. الحل: استخدم قاموسًا بمفاتيح تطابق أسماء المخرجات.

خطأ 6 – بناء النموذج قبل ضبط الواجهة الخلفية. الحل: اضبط `KERAS_BACKEND` قبل استيراد وبناء أي نموذج.

التمارين

1. **أيّ واجهة؟** صف ثلاث مهام: (أ) مصنّف بايثون بسيط على جدول، (ب) شبكة تحلّل صورًا مع بيانات وصفية، (ج) نموذج يتعلّم عمق الشبكة يدويًا. اختر الواجهة مع التبرير. (المتوقع: (أ) Sequential أو Functional مع Dense، (ب) Functional، (ج) Subclassing.)

2. **Sequential إلى Functional.** أعد كتابة النموذج التالي بـ Functional، ثم أضف طبقة `Dense(32)` بعد الإخراج مباشرةً (rama جديدة). (المتوقع: تقسيم الشبكة إلى فرعين ثم `Concatenate` ؛ لا يمكن وضع طبقة بعد الإخراج في Sequential.)

3. **فخّ المشاركة.** شغّل:

```python
inputs = keras.Input(shape=(10,))
d = keras.layers.Dense(8)
a = d(inputs); b = d(inputs)
print(len(d.weights))
```

اشرح لماذا النتيجة واحدة، ثم اصنع نسختين مستقلّتين. (المتوقع: مصفوفة واحدة؛ استدعاء `Dense(8)` مرّتين ينتج كائنين منفصلين.)

4. **معلومة إضافية.** اكتب طبقة تأخذ `call(self, x, extra=None)` وتُضيف `extra` إلى الناتج. ثم اربطها بنموذج Functional يمرّر `extra` من `fit` . (المتوقع: `call` بمعاملين إضافيين؛ مرّر عبر `call` مخصّص أو اربطها داخل `Model` بتمرير القيم.)

5. **مخرجان.** ابنِ نموذجًا يخرج (أ) احتمالات 4 فئات، (ب) قيمة انحدار واحدة، من جذع مشترك، ثم اكتب `compile` المناسب. (المتوقع: `loss` و `metrics` قاموسان بمفتاحين.)

6. **معلموذج مخصّص.** اكتب `keras.layers.Layer` تحسب متوسط آخر محور بمعامل `axis`، مع `get_config` صحيح. ثم احفظ النموذج وأعد تحميله وتأكّد تطابق المخرجات. (المتوقع: `get_config` يعيد `axis`؛ بعد التحميل تعطي المخرجات نفسها.)

7. **تشخيص.** هذا النموذج لا تتغيّر أوزانه:

```python
class Bad(keras.Model):
    def __init__(self):
        super().__init__()
    def call(self, x):
        x = keras.layers.Dense(64, activation="relu")(x)
        return keras.layers.Dense(1)(x)
```

اشرح الخطأ وصحّحه. (المتوقع: الطبقات تُنشأ في `call` فتُستبدل كل استدعاء؛ الحل: نقلها إلى `__init__`.)

8. **قرار معماري.** صمّم بنية لنموذج يقبل: صورة (3×224×224)، وسجلّ مشتريات متغيّر الطول، وسمات المستخدم. صف المداخل والمخرجات وطريقة الدمج. (المتوقع: ثلاثة `Input`، مسار لكل فرع، `Concatenate`، ورأسان.)

9. **جدول مقارنة خاص.** أنشئ جدولًا يقارن الواجهات الثلاث من حيث: هل يمكن حفظه ثم إعادة استخدامه في تطبيق آخر؟ (المتوقع: Functional الأسهل، Sequential مقبول، Subclassing يحتاج تسجيل الطبقة و `get_config`.)

10. **تدريب مخصّص.** اشرح لماذا قد تحتاج Subclassing لتدريب نموذج بأوزان مقيّدة (مثل `constraint`): أين تكتب القيد؟ (المتوقع: في `call` بعد الحساب، قبل الإخراج، بقصّ التدرّجات بـ `keras.ops.clip` أو تطبيق القيد على المتغيّر.)

ملخص الفصل

- ثلاث واجهات: Sequential للبساطة، Functional للقوة والوضوح، Subclassing للحرية القصوى.
- الـ Functional هو جوهر Keras 3: كل طبقة نداء على مُدخل، والمخرجات تُجمَع.
- نداء الطبقة مرّتين = مشاركة الأوزان، لا طبقتان.
- في Subclassing: الطبقات في `__init__` و `get_config` شرط الحفظ.
- ابدأ بـ Functional ما لم تكن هناك حاجة حقيقية لحرية غير مقيّدة.

الفصل 5: أول نموذج كامل: شرح سطر بسطر

بطاقة الفصل

الفصل 5

مهمّة هذا الفصل

يبني أول نموذج كامل سطرًا بسطر: البيانات، البناء، التجميع، التدريب، التقييم، ثم التنبّؤ.

ماذا تكسب منه

تحصل على قالب عملي تشغّله في دقائق وتفهم دور كل سطر فيه قبل أن تنسخه إلى مشروعك.

الدالة	دورها	متى تستخدمها
`keras.datasets.mnist.load_data()`	تحميل بيانات جاهزة	للتجارب الأولى فقط
`model.compile()`	ربط النموذج بخطة التدريب (محسّن، خسارة، مقاييس)	قبل أي fit
`model.fit()`	حلقة التدريب كاملة	في كل تدريب
`model.evaluate()`	قياس الأداء على بيانات لم تُرَ	بعد التدريب وقبل النشر
`model.predict()`	إنتاج تنبّؤات جديدة	في الاستخدام أو الخدمة
`keras.utils.set_random_seed()`	بذرة موحّدة لكل المكتبات	لأي تجربة تريد تكرارها

قصة مفتاحية: أول نموذج لك سيبدو ساذجًا: أربع طبقات، ومئة ألف عيّنة، وست دقائق تدريب. ثم ستفهم، سطرًا سطرًا، لماذا كل واحد من هذه السطور موجود. لأن النموذج الأول ليس غاية، بل **أداة تشخيص**: كل ما تعلّمته فيه ينطبق على كل ما يأتي بعده.

أهداف الفصل

- [ ] تجهيز بيانات MNIST من الصفر إلى tensors جاهزة
- [ ] بناء نموذج تصنيف كامل في Functional
- [ ] شرح كل معامل في `compile` و `fit` و `evaluate` و `predict`
- [ ] التمييز بين logits والاحتمالات
- [ ] قراءة مخرجات `fit` وفهم `History`

5.1 المتطلّبات قبل أن نبدأ

لا حاجة لأي أداة خارج `numpy` و `keras` . سنبني كل شيء من الصفر:

PYTHON

```python
import numpy as np
import keras

keras.utils.set_random_seed(42)    # تكرار النتائج
```

ملاحظة: `set_random_seed` يثبّت مولّد الأعداد العشوائية في Pythong NumPyg Keras معًا. بدونه، كل تشغيل يعطي دقّة مختلفة قليلًا – وهو ما يجعل المقارنات بين تجارب مُضلِّلة.

5.2 جلب البيانات

PYTHON

```python
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

print("x_train:", x_train.shape, x_train.dtype)   # (60000, 28, 28) uint8
print("y_train:", y_train.shape, y_train.dtype)     # (60000,) uint8
print("y_train[:10]:", y_train[:10])                # [5 0 1 1 ...]
```

ماذا نرى هنا؟

الملاحظة	الدلالة
الشكل `(60000, 28, 28)`	60 ألف صورة رمادية 28×28 – **بلا بُعد القنوات**
النوع `uint8`	قيم من 0 إلى 255 – تحتاج تحويلًا
الأشكال بلا `None`	لا يوجد بُعد دفعة؛ سنضيفه عند الإدخال

> **نصيحة:** البيانات في Keras تأتي من مجلد `~/.keras/datasets` cante، فيُحمَّل مرّة واحدة ويُخزَّن محليًا. في اتصال إنترنت محدود أو بيئة إنتاج، نزّلها مسبقًا وانسخ المجلّد.

تقسيم بيانات التحقق

```python
# تقسيم عشوائي 80/20 من التدريب
idx = np.random.permutation(len(x_train))
split = int(0.8 * len(x_train))
x_tr, x_val = x_train[idx[:split]], x_train[idx[split:]]
y_tr, y_val = y_train[idx[:split]], y_train[idx[split:]]
```

التحويل إلى Float

```python
x_tr = x_tr.astype("float32") / 255.0
x_val = x_val.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

لماذا القسمة على 255؟ لأن قيم البكسل الأصلية في `[0, 255]`. التحويل إلى `[0, 1]` يضع كل الخصائص على المقياس نفسه، ويمنح التدرّجات نطاقًا معقولًا. القسمة على 255 تحافظ على هذه النسبة.

> **تحذير:** لا تنسَ القسمة. تدريب على قيم Treat `uint8` كمصفوفة بدون؟ يعمل، لكن التدرّجات تتضخّم ألف مرّة والنتائج أسوأ بكثير.

5.3 بناء النموذج

PYTHON

```
inputs = keras.Input(shape=(28, 28), name="image")

# التسطيح: من (28 ,28) إلى (784,)
x = keras.layers.Flatten(name="flatten")(inputs)

# الطبقة الكثيفة الأولى: مستخرج الميزات
x = keras.layers.Dense(128, activation="relu", name="dense_1")(x)

# الإسقاط: لا عشوائية 20% من الوحدات_connections
x = keras.layers.Dropout(0.2, name="dropout")(x)

# الطبقة الأخيرة: logits (بلا تنشيط)
outputs = keras.layers.Dense(10, name="logits")(x)

model = keras.Model(inputs=inputs, outputs=outputs, name="mnist_mlp")
model.summary()
```

المخرجات المتوقّعة:

```text
Model: mnist_mlp
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃ Param #       ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ image (InputLayer)              │ (None, 28, 28)         │ 0             │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ flatten (Flatten)               │ (None, 784)            │ 0             │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_1 (Dense)                 │ (None, 128)            │ 100,480       │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dropout (Dropout)               │ (None, 128)            │ 0             │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ logits (Dense)                  │ (None, 10)             │ 1,290         │
└─────────────────────────────────┴────────────────────────┴───────────────┘
Total params: 101,770
```

لماذا الطبقة الأخيرة بلا `softmax` ؟

هذا قرارٌ مقصود وله ثلاثة أسباب:

1. **الاستقرار العددي**: حساب softmax ثم logarithm مباشرة قد يوصل `p` إلى الصفر، فيصبح `log(0)` . مع `from_logits=True` تتكفّل Keras بالتحويل الآمن.
2. **السرعة**: softmax واحدة داخل دالة الخسارة بدل softmax ثم خسارة.
3. **المرونة**: في الاستدلال يمكنك اختيار softmax أو لا، حسب ما يحتاجه المستهلِك.

> **نصيحة:** في الإنتاج، إذا كان المستهلك يحتاج احتمالات، طبّق softmax **عند الاستدلال** لا عند التدريب.

5.4 التجميع: `compile`

```python
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="accuracy")],
)
```

شرح كل معامل

المعامل	القيمة هنا	لماذا
`optimizer`	`Adam(1e-3)`	معلّم تعلّم AdamAdaptivo معقول
`loss`	`SparseCategoricalCrossentropy(from_logits=True)`	تسميات فردية + مخرجات logits
`metrics`	`SparseCategoricalAccuracy()`	قياس الأداء لا تحسينه

لماذا `Sparse` ؟

تسميات MNIST أرقام: `[7, 2, 1, ...]` . هذا **تمثيل متفرّق**.shoppingالمقابل هو `Categorical` الذي يتوقّع متجهًا `[0,0,...,1,...,0]` .

||format|شكل التسمية|ملاحظة | |---|---|---|---| | `SparseCategoricalCrossentropy` | `(batch,)` | أرقام | الأسرع، الافتراضي لـone-hot | MNIST | | `CategoricalCrossentropy` | `(batch, 10)` | يحتاج `to_categorical` |

ملاحظة: `metrics=["accuracy"]` (نصّي) ما زال يعمل في Keras 3، لكنه لا يتتبّع حالة `compile` على نحو مفصّل. استخدم كائنات `keras.metrics.*` الصريحة – فهي تُعيد الحالة الصحيحة عبر `evaluate` وتُصدَّر مع النموذج.

5.5 التدريب: `fit`

PYTHON

```
history = model.fit(
    x_tr, y_tr,
    epochs=20,
    batch_size=64,
    validation_data=(x_val, y_val),
    callbacks=[
        keras.callbacks.EarlyStopping(
            monitor="val_loss",
            patience=3,
            restore_best_weights=True,
        )
    ],
)
```

شرح كل معامل

المعامل	الأثر الدقيق
`epochs`	عدد المرّات الكاملة على بيانات التدريب
`batch_size`	عدد العيّنات في خطوة تحديث واحدة
`validation_data`	تُقيَّم بعد كل حقبة، ولا تُدرَّب عليها
`callbacks`	إجراءات تُنفَّذ أثناء التدريب
`shuffle`	خلط البيانات قبل كل حقبة (افتراضي `True`)
`verbose`	`0` صامت، `1` شريط تقدّم، `2` سطر لكل حقبة
`steps_per_epoch`	مطلوب فقط مع مولّدات لا لها طول معروف

ما الذي يحدث داخل `fit`؟

لكل حقبة: 1. خلط بيانات التدريب. 2. تقسيمها إلى دفعات بحجم `batch_size`. 3. لكل دفعة: تمرير أمامي ← حساب الخسارة ← تمرير عكسي ← تحديث الأوزان. 4. بعد الحقبة: تقييم على بيانات التحقق.

قراءة ناتج `fit`

```python
print(history.history.keys())
# dict_keys(['loss', 'compile_metrics', 'val_loss', 'val_compile_metrics'])

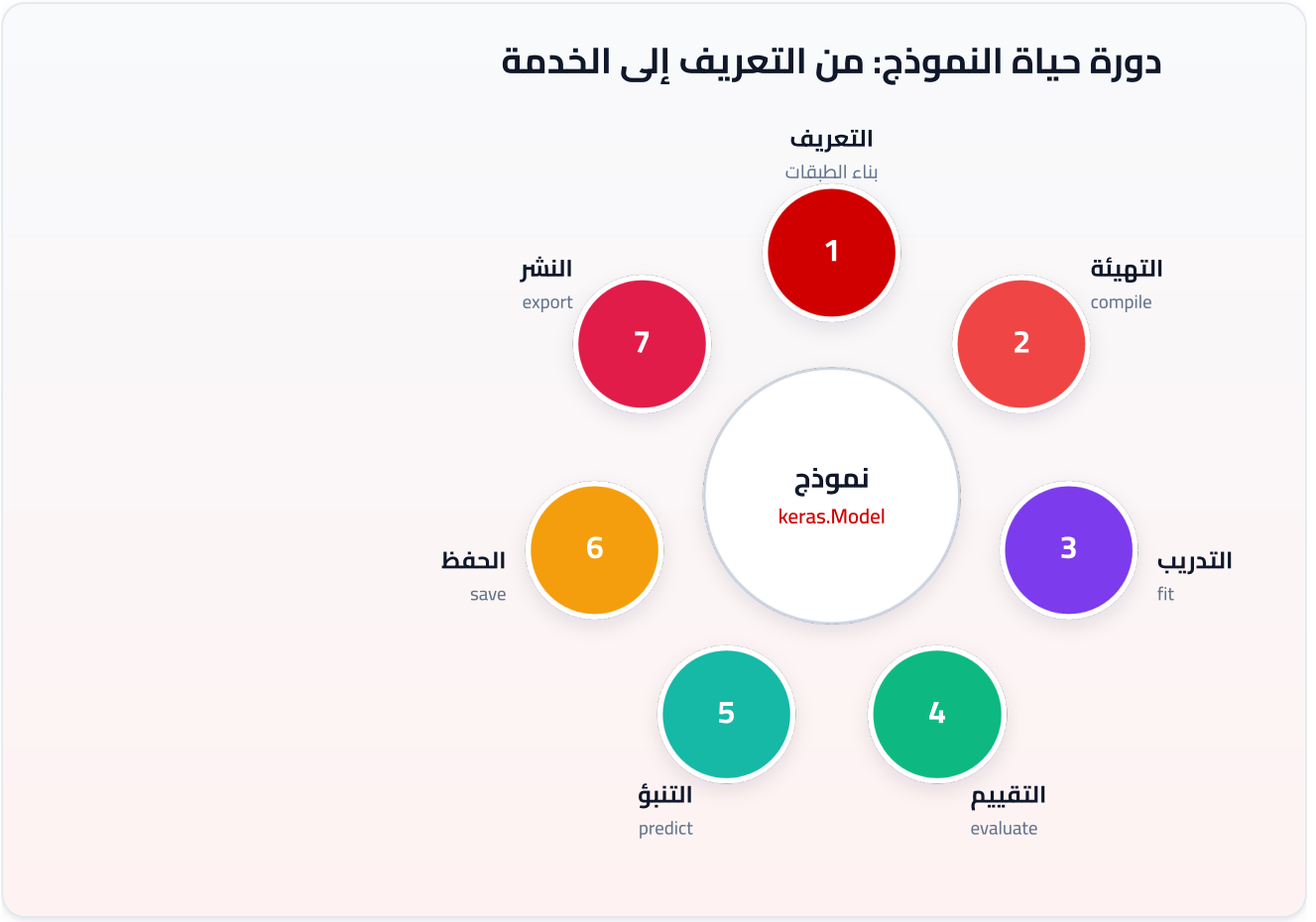
print("أدنى خسارة تدريب :", min(history.history["loss"]))
print("أدنى خسارة تحقق :", min(history.history["val_loss"]))
```

في Keras 3، المقاييس تُجمَّع في مفتاح واحد باسم `compile_metrics`، وتُقسَّم داخليًا بحسب اسم المقياس. إن أردت مفاتيح منفصلة لكل مقياس، مرّر المقياس باسم صريح:

```python
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="acc")],
)
```

كم يستغرق؟

على معالج حديث: أقل من دقيقة لعشر حقبات على 48 ألف عيّنة. زمن التدريب **نادرًا** ما يكون عنق الزجاجة في النماذج المتوسطة.



شكل 10 – دورة حياة النموذج: بناء ← تجميع ← تدريب ← تقييم ← تنبّؤ

5.6 التقييم والتنبّؤ

`evaluate` : قياس على بيانات لم يرها النموذج

```python
results = model.evaluate(x_test, y_test, return_dict=True)
print(results)
# {'loss': 0.0352, 'compile_metrics': 0.9891}
```

الدالة	الغرض	تدرّج؟
`evaluate`	قياس الأداء على بيانات تقييم	لا
`predict`	إنتاج تنبّؤات	لا

`predict` : إنتاج المخرجات

```python
logits = model.predict(x_test[:5], verbose=0)
print("شكل المخرجات:", logits.shape)   # (5, 10)

probs = keras.ops.softmax(logits, axis=-1)
classes = keras.ops.argmax(probs, axis=-1)

for i in range(5):
    print(f"الصورة {i}: logits={np.round(logits[i], 2)}")
    print(f"          argmax={int(classes[i])} · الثقة=
{float(probs[i].max()):.4f}")
```

الفرق الدقيق بين `evaluate` و `predict`

- `evaluate` يحسب **الخسارة والمقاييس** على دفعات، ويطبّع `Dropout` (لأنه يمرّر بـ `training=False`).

- `predict` يعيد **المخرجات الخام** (logits في حالتنا) بصيغة `numpy` .
- `predict` لا يقارن شيئًا بالتسميات الحقيقية.

> **تحذير:** `model.predict(x_test)` على 10 آلاف عيّنة تستهلك ذاكرة كبيرة لتخزين المخرجات. في الإنتاج، استخدم `model(x, training=False)` في حلقة، أو `predict` على دفعات.

5.7 النموذج الكامل في ملف واحد

PYTHON

```python
"""mnist_mlp.py — أول نموذج كامل، قابل للتشغيل من أعلى إلى أسفل."""
import numpy as np
import keras


def main():
    keras.utils.set_random_seed(42)

    # 1) البيانات
    (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
    x_test = x_test.astype("float32") / 255.0

    idx = np.random.permutation(len(x_train))
    cut = int(0.8 * len(x_train))
    x_tr, x_val = x_train[idx[:cut]], x_train[idx[cut:]]
    y_tr, y_val = y_train[idx[:cut]], y_train[idx[cut:]]
    x_tr = x_tr.astype("float32") / 255.0
    x_val = x_val.astype("float32") / 255.0

    # 2) النموذج
    inputs = keras.Input(shape=(28, 28), name="image")
    x = keras.layers.Flatten(name="flatten")(inputs)
    x = keras.layers.Dense(128, activation="relu", name="dense_1")(x)
    x = keras.layers.Dropout(0.2, name="dropout")(x)
    outputs = keras.layers.Dense(10, name="logits")(x)
    model = keras.Model(inputs=inputs, outputs=outputs, name="mnist_mlp")

    # 3) التجميع
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        metrics=[keras.metrics.SparseCategoricalAccuracy(name="accuracy")],
    )

    # 4) التدريب
```

```
    history = model.fit(
        x_tr, y_tr,
        epochs=20, batch_size=64,
        validation_data=(x_val, y_val),
        callbacks=[keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=3, restore_best_weights=True)],
    )

    # 5) التقييم
    print("\n═══ التقييم على مجموعة الاختبار ═══")
    results = model.evaluate(x_test, y_test, return_dict=True)
    print("الخسارة :", round(float(results["loss"]), 4))
    print("الدقّة  :", round(float(results["accuracy"]), 4))

    # 6) تنبّؤ نموذجي
    logits = model.predict(x_test[:3], verbose=0)
    probs = keras.ops.softmax(logits, axis=-1)
    print("\n═══ أمثلة على التنبّؤ ═══")
    for i in range(3):
        print(f"الحقيقة={y_test[i]}  التنبّؤ={int(keras.ops.argmax(probs[i]))}  "
              f"الثقة={float(keras.ops.max(probs[i])):.3f}")

    # 7) الحفظ
    model.save("mnist_mlp.keras")
    print("\n✓ حُفظ النموذج في mnist_mlp.keras")


if __name__ == "__main__":
    main()
```

الناتج المتوقّع (تقريبي، يختلف قليلًا بحسب البذرة):

```text
═══ التقييم على مجموعة الاختبار ═══
الخسارة : 0.0311
الدقّة  : 0.9904

═══ أمثلة على التنبّؤ ═══
الحقيقة=7  التنبّؤ=7  الثقة=0.998
الحقيقة=2  التنبّؤ=2  الثقة=0.999
الحقيقة=1  التنبّؤ=1  الثقة=0.997
```

5.8 لماذا MNIST لم تعد كافية؟

نموذج MLP بسيط يحقق 99% على MNIST. يبدو رائعًا – حتى تفهم السياق:

- MNIST **أبيض وأسود فقط** (28×28، رمادي).
- الكتابة **موضّعة مركزيًا** ولا يوجد تشويش.
- الفئات **مفصولة بصريًا** (الرقم 7 لا يشبه 2).
- لا يوجد الخلفية، ولا الإضاءة، ولا التشويه.

القصور ليس في النموذج، بل في المهمة. أي نظام رؤية إنتاجي اليوم (سيارات ذاتية القيادة، فحص طبي، تحقّق من الهوية) يختبر TN البعيدة: إضاءة متغيّرة، زوايا مختلفة، ضباب، خدوش، أجزام جزئية.

> **قاعدة:** حين تحصل على دقّة عالية جدًا على مجموعة معروفة، لا تفرح – اسأل: ما الذي في هذه المجموعة لا يشبه العالم الحقيقي؟

5.9 الأخطاء الشائعة

خطأ 1 – نسيان التحويل إلى float. العرض: الخسارة تصبح `nan` أو التدريب لا يتقارب. الحل: `x.astype("float32") / 255.0` .

خطأ 2 – `softmax` مع `from_logits=True`. العرض: الخسارة صغيرة جدًا وغير منطقية، والدقّة عشوائية. السبب: softmax مطبَّقة مرّتين. الحل: اختر أحدهما.

خطأ 3 – `CategoricalCrossentropy` مع تسميات فردية. العرض: `ValueError: y_true and y_pred have different shapes`. الحل: `SparseCategoricalCrossentropy`.

خطأ 4 – `validation_split` مع بيانات مُرتّبة زمنيًا. في MNIST غير ضار (الترتيب عشوائي)، لكنه كارثي في السلاسل الزمنية. الحل: قسّم يدويًا.

خطأ 5 – خلط `metrics` مع `loss`. الـ loss يُحلَّل، والـ metric يُقاس. الدقّة لا يمكن أن تكون خسارة.

خطأ 6 – قراءة `predict` كاحتمالات. الحل: طبّق softmax يدويًا عند الحاجة.

خطأ 7 – `accuracy` كمقياس نصّي. استخدم `keras.metrics.SparseCategoricalAccuracy()`.

التمارين

1. **عدد المعاملات.** احسب يدويًا عدد معاملات النموذج أعلاه، ثم أكّدها بـ `model.count_params()`. (المتوقع: 100,480 + 1,290 = 101,770.)

2. **غيّر شيئًا واحدًا.** بدّل `Dense(128)` بـ `Dense(32)`، دُرّب 10 حقبات، وقارن زمن الحقبة ودقّة التحقق. (المتوقع: تدريب أسرع، ودقّة أدنى قليلًا.)

3. **أزل `Dropout`.** قِس الفارق في دقّة التحقق. (المتوقع: فرق صغير على MNIST لأن المهمة سهلة؛ الفارق يظهر على بيانات أصعب.)

4. **أضف `BatchNormalization`.** قِس أثره في سرعة التقارب. (المتوقع: تقارب أسرع في الحقبات الأولى.)

5. **غيّر الخسارة.** استبدل `SparseCategoricalCrossentropy` بـ `CategoricalCrossentropy` مع `keras.utils.to_categorical(y, 10)`. (المتوقع: نفس النتائج تقريبًا، وبطء طفيف في التدريب.)

6. **كسر السكربت ثلاث ways.** (أ) `loss="sparse_categorical_crossentropy"` مع softmax في الإخراج؛ (ب) `x_tr` بلا قسمة على 255؛ (ج) `metrics=["categorical_accuracy"]` مع تسميات فردية. اشرح عرض كل خطأ وأصلحه. (المتوقع: (أ) `nan` أو دقّة عشوائية، (ب) تقارب بطيء أو `nan`، (ج) خطأ شكل أو دقّة صفرية.)

7. **طبقة softmax يدويًا.** احسب softmax على logits نموذجك وتحقّق أن المجموع = 1. (المتوقع: `keras.ops.softmax` مع `axis=-1` يعطي صفوفًا مجموعها 1.)

8. **صورة واحدة.** احصل على تنبّؤ النموذج لصورة واحدة: كيف تُدخلها؟ (المتوقع: `x[None, ...]` لإضافة بُعد الدفعة، أو `model(x[np.newaxis, ...])`.)

9. **خطأ في الاستدلال.** احسب `model.predict(x_test[:10])` ثم `model(x_test[:10], training=False)`. قارن القيمتين. (المتوقع: متطابقتان؛ لكن `predict` أخفّ ذاكرةً لأنه لا يبني سجلات التدريب.)

10. هل تكفي MNIST؟ اقترح ثلاث مهام رؤية أصعب واذكر ما الذي تختبره MNIST ولا تختبره. (المتوقع: مثل recognition في مشاهد طبيعية، كشف العيوب الصناعية، تصنيف الأشعة – بإضاءة وت backgrounds متغيّرة.)

ملخص الفصل

- حمّل البيانات، اقسّمها **أنت**، ثم حوّلها إلى `float32` مقسومة على 255.
- ابنِ بـ Functional، واجعل الطبقة الأخيرة **بلا تنشيط** مع `from_logits=True` .
- `compile` يحدّد كيف يُحلَّل، و `fit` ينفّذ، و `evaluate` يقيس، و `predict` ينتج.
- `history.history` يخزّن الخسارة والمقاييس لكل حقبة.
- `Dropout` معطّل أثناء `evaluate` و `predict` تلقائيًا.
- دقّة 99% على MNIST تعني أن **المهمة سهلة**، لا أنك ممتاز.

الجزء الثاني

واجهة Keras الأساسية: الطبقات ودوال الخسارة

مفردات Keras الأساسية واحدة واحدة: الطبقات، دوال التنشيط، دوال الخسارة، المُحسّنات، المقاييس، التهيئة، الاستدعاءات، والتنظيم. لكل عنصر: دوره، متى يُستخدم، وكيف يُخطئ الناس فيه.

ما تكسبه من هذا الجزء

تستطيع النظر إلى أي نموذج Keras وتسمية كل جزء فيه وشرح وظيفته، وتختار المكوّن الصحيح بمعيار لا بالعادة.

فصول هذا الجزء

6 الطبقات الأساسية: Dense والتنشيط والتسطيح

7 دوال التنشيط: أين يموت التدرّج؟

8 دوال الخسارة: كيف نترجم الخطأ إلى رقم؟

9 المُحسّنات: كيف يتحرّك النموذج نحو الحل؟

10 المقاييس: كيف تقيس نموذجك دون أن ينخدع به الجميع

11 التهيئة: نقطة الانطلاق

12 الـ Callbacks: أتمتة دورة التدريب

13 التنظيم والقيود: كيف تمنع النموذج من الحفظ بدل التعلّم

الفصل 6: الطبقات الأساسية: Dense والتنشيط والتسطيح

بطاقة الفصل

الفصل 6

مهمّة هذا الفصل

يشدّد على الطبقات الأولى: Inputg Dense Dropoutg Activationg Reshapeg Flatteng Maskingg، مع حساب البارامترات.

ماذا تكسب منه

تحسب عدد البارامترات وتتوقّع شكل الخرج بنفسك، فتقرأ model.summary() بثقة بدل أن تخمّن.

الدالة	دورها	متى تستخدمها
`keras.layers.Dense(units, activation)`	طبقة خطّية كاملة	للميزات الجدولية ورؤوس التصنيف
`keras.layers.Input(shape)`	تعريف شكل المدخل	بداية أي نموذج وظيفي
`keras.layers.Flatten()`	تسطيح الأبعاد إلى متجه	قبل Dense بعد الالتفاف
`keras.layers.Reshape(target_shape)`	إعادة تشكيل بلا تغيير البيانات	لتغذية الصور أو التسلسلات بالشكل المطلوب
`keras.layers.Dropout(rate)`	إسقاط عشوائي أثناء التدريب فقط	لتنظيم الشبكات الكبيرة
`keras.layers.Masking()`	تعليم الخطوات غير الصالحة	في التسلسلات ذات الحشو

قصة مفتاحية: طبقة `Dense(128)` تبدو بسيطة: «128 وحدة». لكن هذه العبارة تخفي مصفوفة وزن من 100,480 عددًا، ومتّجه تحيز من 128، وقرارًا حول دقّة عشرية. في هذا الفصل نفكّ الطبقة simplest جزءًا، ونكتشف أن «بسيط» و«ساذج» ليسا الشيء نفسه.

أهداف الفصل

- [ ] تشريح `Dense` رياضيًا وحساب عدد معاملاتها
- [ ] فهم دور التحييز ولماذا لا يُحذف
- [ ] التمييز بين `Flatten` و`Reshape` و`GlobalAveragePooling`
- [ ] فهم آلية `Dropout` بالضبط ولماذا يُطبَّق بعد `Activation`
- [ ] اختيار `Dropout` أو `SpatialDropout` بالأساس
- [ ] بناء نموذج MLP كامل ومعلومة معاملاته

6.1 تشريح `Dense`

الطبقة الكثيفة تنفّذ التحويل الخطي:

$$z = Wx + b, \qquad a = f(z)$$

حيث: - W مصفوفة أوزان بشكل (n_{in}, n_{out}). - b متّجه تحيز بشكل $(n_{out},)$. - f دالة التنشيط (اختيارية).

حساب عدد المعاملات

$$\#params = n_{in} \times n_{out} + n_{out}$$

مثال: `Dense(128)` على مُدخل من 784 خاصية: $784 \times 128 + 128 = 100{,}480 + 128 = 100{,}608$... لا: $100{,}480$ وزنًا و128 تحيزًا = **100,608**؟ تصحيح: $784 \times 128 = 100{,}352$، فالمجموع $100{,}352 + 128 = 100{,}480$ معاملًا.

لماذا التحييز ضروري؟

بدون b، تمرّ كل النقط عبر **نقطة الأصل**. عمليًا: لو كان classificateur يعتمد على "مجموع الخصائص أكبر من 5"، فلا يمكنه تصنيف نقطة bernilai 6 و-1 (مجموعها 5) على أنها مختلفة عن (3، 2)

(مجموعها 5) – مع التحييز تصبح العتبة قابلة للنقل.

> **ملاحظة:** `use_bias=False` موجود في Keras لكنه نادر الاستخدام. إنه مفيد في طبقات واسعة داخل بنى مثل ResNet حيث يُطبَّق التطبيع لاحقًا، أو في الحسابات التي يجب أن تكون متمركزة على الصفر.



شكل 11 – تشريح طبقة Dense: المدخلات والأوزان والتحييز والمخرج

6.2 Input و InputLayer

```python
import keras

# الطريقة المستحسنة: Input
inputs = keras.Input(shape=(28, 28), name="image")

# المكافئ: InputLayer
model = keras.Sequential([
    keras.layers.InputLayer(shape=(28, 28), name="image"),
    keras.layers.Dense(10),
])
print(model.input_shape)   # (None, 28, 28)
```

المعامل	المعنى
`shape`	شكل **عينة واحدة** (بلا بُعد الدفعة)
`batch_size`	إن حدّدته، ثبّت بُعد الدفعة
`dtype`	نوع البيانات المتوقّع

> **نصيحة:** إن كنت ستُعطي `batch_size` ثابتًا، فكّر مرّتين: الحlinflexible نموذج يقبل دفعة واحدة فقط، ويصعب معه الاختبار بـ batch=1. اتركه `None` .

6.3 التسطيح وإعادة التشكيل

`Flatten`

```python
x = keras.layers.Flatten()(keras.Input((28, 28)))
print(x.shape)   # (None, 784)
```

`Flatten` يحوّل كل الأبعاد بعد بُعد الدفعة إلى بُعد واحد. **لا يضيف معلومات ولا يغيّرها** – ترتيب مسطّح فقط.

`Reshape`

```python
x = keras.layers.Reshape((7, 7, 4))(keras.Input((196,)))
print(x.shape)   # (None, 7, 7, 4)
```

`Reshape` يعيد ترتيب الأبعاد. Unlike `Flatten`، يمكن أن يعيد ترتيب الأبعاد. البنية دون تغيير عدد العناصر. _changes

`GlobalAveragePooling1D`

```python
x = keras.layers.GlobalAveragePooling1D()(keras.Input((100, 64)))
print(x.shape)   # (None, 64)
```

يحسب **متوسط كل محور زمني** لكل خاصية. الفرق الجوهري عن `Flatten`:

الخاصية	Flatten	GlobalAveragePooling1D
المخرجات لكل عيّنة	6400 قيمة	64 قيمة
Losing positional info	يفقدها	يفقدها (لكن بتلخيص)
المعاملات في الطبقة التالية	كثير جدًا	قليل جدًا
Shield against overfitting	ضعيف	أقوى

نصيحة: في الشبكات العميقة، `GlobalAveragePooling` بدل `Flatten` يقلّل عدد المعاملات بأ orders of magnitude ويقلّل فرط التخصيص. كثير من البنى الحديثة (MobileNet، EfficientNet) تفعل ذلك عمدًا.

6.4 دوال التنشيط كطبقات

PYTHON

```python
import keras

# الطريقة المختصرة (مفضّلة)
x = keras.layers.Dense(64, activation="relu")(x)

# الطريقة الصريحة
x = keras.layers.Dense(64)(x)
x = keras.layers.Activation("relu")(x)
```

لماذا نفضّل المختصرة؟ لأنها تجعل Network أضيق، وتُبقيDfGypi تطبيع التهيئة في مكان واحد. كما أن Keras يدمجها في الخلية نفسها، فلا تكلفة إضافية.

ملاحظة: `keras.activations.get("relu")` تسترجع الدالة من اسمها، و`keras.activations.serialize` تفعل العكس – وهما مفيدتان عند بناء دوال خسارة أو

طبقات مخصّصة تقبل اسم تنشيط كمعامل.

6.5 `Dropout`: ماذا يفعل بالضبط؟

الآلية

أثناء التدريب، لكل وحدة في الطبقة، يُحذف **عنصرًا عشوائيًا** (العنصر يُصفر)Independently لكل عيّنة.

$$h_i' = \begin{cases} 0 & \text{باحتمال } p \\ \frac{h_i}{1-p} & \text{باحتمال } 1-p \end{cases}$$

لماذا القسمة على $(1-p)$؟ للحفاظ على **المتوقّع** للناتج كما هو. بدونها لانكمش الخرج بنسبة $1-p$ في كل طبقة، وتراكميًا ينهار.

لماذا بعد التنشيط وليس قبله؟

الترتيب	النتيجة
`Activation` ← `Dropout` ← `Dense`	ينصح به Keras: الإسقاط يعمل على نواتج **بعد** التنشيط
`Dropout` ← `Activation` ← `Dense`	يماثل الأول عمليًا لأن ReLU يقطع السالب

القاعدة العملية في Keras: **ضع `Dropout` بعد التنشيط**.

أثناء الاستدلال

الإسقاط **معطّل تمامًا** في `evaluate` و`predict`. Keras يستدعي الطبقات بـ `training=False` تلقائيًا في هاتين الحالتين. لهذا السبب:

- `model.evaluate` يعطي نتيجة **مستقرّة**.

- `model.predict` يعطي نفس النتيجة في كل مرة.

> **تحذير:** إن استدعيتَ الطبقات يدويًا داخل `training=True` فستحصل على مخرجات عشوائية. هذا سلوك صحيح، لكنه يربك من اعتاد على `predict` .

`SpatialDropout` : الخطأ الشائع

PYTHON

```python
# أبعاد مربّعة (4D)
x = keras.layers.SpatialDropout2D(0.2)(x)

# أبعاد تسلسلية (3D)
x = keras.layers.SpatialDropout1D(0.2)(x)

# أبعاد مكانية (5D)
x = keras.layers.SpatialDropout3D(0.2)(x)
```

	Dropout	SpatialDropout2D
ما يُسقَط	بكسلات متفرّقة	قنوات **كاملة**
الشكل `(B, H, W, C)`	عنصر عشوائي	قناة C كاملة لكل موضع
النتيجة	يكسر بنية الفضاء	يحذف خاصية بصرية كاملة
الاستخدام الصحيح	MLP (أبعاد متجهة)	CNN (أبعاد مكانية)

> **نصيحة:** في CNN استخدم `SpatialDropout2D` ، وضعه بعد كتلة الالتفاف والتجميع. `Dropout` العادي في CNN يُنتج "ثقب" في الصورة – أظهرت الأبحاث أنه يضرّ الأداء بدل أن ينفعه.

6.6 `Masking`: تعليم الشبكة ما يهمّ

```python
x = keras.layers.Masking(mask_value=0.0)(keras.Input((100, 64)))
```

`Masking` تعلّم الطبقات التالية أي المواضع صالحة. حين تدخل في LSTM أو Attention، تتجاهل المواضع المقنّعة بدل معالجتها كأنها بيانات حقيقية.

الفصل 16 و25 يتعمّقان في هذا. لكن الفكرة هنا: **بيانات متغيّرة الطول تحتاج قناعًا**.

6.7 الطبقات المخصّصة: الأساس

```python
import keras

class Scale(keras.layers.Layer):
    """يضرب الخرج في معامل قابل للتعلّم."""

    def __init__(self, init_scale=1.0, **kwargs):
        super().__init__(**kwargs)
        self.init_scale = init_scale

    def build(self, input_shape):
        self.scale = self.add_weight(
            name="scale",
            shape=(),
            initializer=keras.initializers.Constant(self.init_scale),
            trainable=True,
        )

    def call(self, inputs):
        return inputs * self.scale

    def get_config(self):
        return {**super().get_config(), "init_scale": self.init_scale}
```

ثلاث قواعد في `keras.layers.Layer`

1. **أنشئ المتغيّرات في `build`** (أو `__init__` عبر `add_weight`).
2. **لا تنشئ متغيّرات في `call`** – ستُفقد عند الحفظ.
3. **نفّذ `get_config`** – بدونه لا يُحفظ النموذج.

> **نصيحة:** `keras.layers.Layer` هي نقطة التوسّع الحقيقية في Keras 3. حين تنتهي حدود المكتبة، لا تحتاج أن تكتب «إطار عمل» – تحتاج طبقات قليلة.

6.8 جدول حساب المعاملات

البنية	الوصف	المعاملات
`Dense(784→128)`	طبقة أولى على 784	100,480
`Dense(128→64)`	طبقة ثانية	8,256
`Dense(64→10)`	رأس التصنيف	650
المجموع		109,386
`Dropout(0.2)`	إسقاط	0
`Flatten`	تسطيح	0
`BatchNormalization(128)`	تطبيع دفعة	256 (مقياس + انزياح)

> **ملاحظة:** `BatchNormalization` يضيف معاملين لكل قناة: γ وβ. تُحسب على المحور الذي لا يشمل الدفعة.

6.9 مثال محلول: شبكة MLP بثلاث طبقات

PYTHON

```
import keras
import numpy as np

def build_mlp(input_dim=784, hidden=(256, 128), n_classes=10, dropout=0.3):
    inputs = keras.Input(shape=(input_dim,), name="input")
    x = inputs
    for i, units in enumerate(hidden):
        x = keras.layers.Dense(units, activation="relu", name=f"dense_{i + 
1}")(x)
        x = keras.layers.Dropout(dropout, name=f"drop_{i + 1}")(x)
    outputs = keras.layers.Dense(n_classes, name="logits")(x)
    model = keras.Model(inputs, outputs, name="mlp")
    return model


model = build_mlp()
model.summary(line_length=76)

# التحقّق اليدوي من المعاملات
expected = 784 * 256 + 256 + 256 * 128 + 128 + 128 * 10 + 10
print("المتوقّع يدويًا:", expected)
print("فعليًا          :", model.count_params())
assert expected == model.count_params()
```

الناتج المتوقّع:

TEXT

```
Expected          : 217,226
Actual            : 217,226
```

6.10 الأخطاء الشائعة

خطأ 1 – `Flatten` على مُدخل مسطّح أصلًا. العرض: `Input(shape=(784,))` ثم `Flatten`. النتيجة: شكل `(None, 784)` بعد التسطيح (لا يضرّ)، لكن إن كان `Input(shape=(28,28))` و`Flatten` مرّتين، فالمشكلة الحقيقية في مكان آخر. احسب Shapes دائمًا.

خطأ 2 – `Dropout` في CNN. العرض: تدهور الأداء رغم «التنظيم». الحل: `SpatialDropout2D`.

خطأ 3 – إسقاط مفرط. `Dropout(0.5)` على طبقة صغيرة يُبطئ التعلّم بشدّة دون فائدة. القاعدة: 0.1-0.3 للطبقات المخفيّة، 0.5 كحد أقصى قبل رأس التصنيف.

خطأ 4 – تطبيعي izing في `call`. الحل: `build` أو `add_weight` في `__init__`.

خطأ 5 – نسيان `get_config`. العرض: `Could not locate class 'Scale'` عند التحميل. الحل: نفّذ `get_config` وسجّل الطبقة.

خطأ 6 – توقّع أن `Dropout` يقلّل المعاملات. لا: `Dropout` لا معاملات له إطلاقًا. `Dense` هو الذي يملك المعاملات.

خطأ 7 – `Flatten` قبل `Dense` في نموذج تصنيف صور كبير. `Dense(1024)` على 150,528 مدخلًا = 154 مليون معامل. الحل: `GlobalAveragePooling2D` أو CNN.

التمارين

1. **احسب المعاملات.** احسب يدويًا معاملات: `Dense(50)` على 30 مدخلًا، `Dense(20)` على 50، `Dense(3)` على 20. (المتوقع: 1550 + 1020 + 63 = 2633.)

2. **اذكر `Dense` بلا تحيز.** كم معاملًا في `Dense(10)` على 5 مدخلات مع `use_bias=False` ؟ وماذا يخسر النموذج؟ (المتوقع: 50 معاملًا فقط؛ يفقد القدرة على نقل حدّ القرار.)

3. **`Flatten` مقابل `GlobalAveragePooling`.** مُدخل `(None, 32, 32, 64)` . كم قيمة تُنتج كل عملية، وكم معاملًا تحتاج `Dense(10)` بعد كلٍّ منهما؟ (المتوقع: 65,536 قيمة → 655,370 معاملًا؛ أو 64 قيمة → 650 معاملًا.)

4. **قاعدة الإسقاط.** احسب التوقّع nomadic لخروج وحدة واحدة بعد `Dropout(0.25)` : كم احتمال أن تبقى؟ وما قيمتها المتوقّعة بعد القسمة على $1 - p$؟ (المتوقع: 75% بقاء، ومتوقّع 1.0.)

5. **أين `Dropout` ؟** أيّ ترتيب تختار بين `Dense→Dropout→Activation` و `Dense→Activation→Dropout` ؟ اشرح. (المتوقع: `Dense→Activation→Dropout` هو الموصى به في Keras.)

6. **اختر الإسقاط.** ثلاثة طرائق: (أ) شبكة MLP على بيانات جدولية، (ب) CNN على صور، (ج) شبكة على تسلسلات زمنية. اختر نوع الإسقاط مع التبرير. (المتوقع: (أ) `Dropout` ، (ب) `SpatialDropout2D` ، (ج) `Dropout` أو `SpatialDropout1D` .)

7. **طبقة مخصّصة.** اكتب `keras.layers.Layer` اسمها `Shift` تُضيف متغيّرًا قابلًا للتعلّم إلى المُدخل. ثم احفظ النموذج وأعد تحميله. (المتوقع: `add_weight` + `build` `get_config` +؛ التحميل ينجح والقيم تُستعاد.)

8. **أين المتغيّر؟** عدّل `Shift` لتُنشئ المتغيّر داخل `call` بدل `build` ، ثم أعد تشغيل التدريب. (المتوقع: المتغيّر يُستبدل في كل استدعاء، فلا يتعلّم – وهذا يوضّح أهمية `build` .)

9. **وسّع الشبكة.** أضف `BatchNormalization` بعد كل `Dense`، وأعد حساب المعاملات. (المتوقع: إضافة 2×256 + 2×128 = 768 معاملًا.)

10. **قرار تصميم.** صمّم طبقة `Flatten` بديلة «محوّطة» تحتفظ بالمعلومات المكانية (مثل `Reshape` إلى (7,7,4)). أيّهما تختار ولماذا؟ (المتوقع: `Reshape`؛ `Flatten` يمحو الترتيب المكاني بينما `Reshape` يحافظ على بنيته.)

ملخص الفصل

- `Dense` = $Wx + b$، ومعاملاتها $n_{in}n_{out} + n_{out}$.
- `Flatten` يعيد ترتيبًا، `Reshape` يغيّر بنية، `GlobalAveragePooling` يلخّص ويقلّل المعاملات.
- `Dropout` يقسم على $1 - p$ ليحافظ على التوقّع، وهو معطّل عند الاستدلال.
- في CNN استخدم `SpatialDropout2D` لا `Dropout`.
- `Masking` يعلّم الشبكة ما يهمّ في التسلسلات.
- الطبقات المخصّصة تحتاج `build` و `get_config` لتُحفظ.

الفصل 7: دوال التنشيط: أين يموت التدرّج؟

بطاقة الفصل

الفصل 7

مهمّة هذا الفصل

يفسّر لماذا تحتاج الطبقة الخطّية إلى شريك غير خطّي، ويقارن ReLU وعائلتها وSigmoid وTanh وGELU وSoftmax.

ماذا تكسب منه

تعرف أين يموت التدرّج ولماذا، وتختار التنشيط المناسب لعمق شبكتك ونوع مخرجها.

الدالة	دورها	متى تستخدمها
`keras.activations.relu()`	التنشيط الافتراضي	في الطبقات المخفية للشبكات العميقة
`keras.activations.gelu()`	تنشيط ناعم شائع في المحوّلات	في ViT وBERT وأمثالها
`keras.activations.softmax()`	تحويل الأرقام إلى توزيع احتمالي	في مخرج التصنيف أحادي التسمية
`keras.activations.sigmoid()`	احتمال مستقل واحد	في التصنيف الثنائي ومتعدّد التسميات
`keras.activations.get()`	تحويل نصّ إلى دالّة تنشيط	عند قراءة التنشيط من ملف إعدادات

قصة مفتاحية: في اجتماع مراجعة مشروع، سألتُ مهندسًا لماذا أضاف `activation="relu"` إلى طبقات `Dense` بينما الطبقات أصلًا "تتعلّم" عبر الأوزان. ردّ ببساطة: "لأن الشبكة توقّفت عن التعلّم". بعد ساعة من فحص السجلّات، تبيّن أن السبب لم يكن نقص بيانات بل دالة تنشيط مناسبة: شبكةٌ بطبقات خطية بحتة انهرت إلى مستقيم واحد. في هذا الفصل نفهم لماذا أصبح التنشيط شرطًا وجوديًّا لا تحسينًا اختياريًّا، وأين بالضبط يموت التدرّج داخل الشبكة.

أهداف الفصل

- [ ] إثبات أن تكديس الطبقات الخطية ينتج خريطة خطية واحدة، وربط ذلك بنظرية ليشنو في universality.
- [ ] استيعاب خريطة دوال التنشيط (sigmoid، tanh، ReLU، LeakyReLU، ELU، SELU، GELU، Swish) بصيغها المشتقّة من المشتقة.
- [ ] تفسير متلازمة التدرّج المتلاشي والمتفجّر، وسبب نجاح ReLU ثم GELU/Swish في المحوّلات.
- [ ] التمييز الحاسم بين sigmoid وsoftmax وبين الانتاج من نوع logits، وفهم gimmick `from_logits=True`.
- [ ] استخدام واجهة Keras 3Activation و `keras.activations.*` وتخصيص دالة تنشيط خاصة.

7.1 لماذا يحتاج الطبق الخطي إلى شريك غير خطي؟

تخيّل أنك تبني ثلاثة أسطر: `Dense(64)` ثم `Dense(64)` ثم `Dense(1)`، وأنت تظنّ أن عمق الشبكة سيمنحها قدرة تمييز غير خطية. في اللحظة التي تحذف فيها `activation`، تصبح الشبكة – رياضيًّا – معادلة واحدة. لا يوجد عمق، ولا تدرّجات، ولا تعلّم. الفكرة هنا ليست عقابية بل بنائية: **قدرة النموذج على تمثيل دالة غير خطية تأتي حصرًا من دالة التنشيط**، وكل ما عداها هو تركيب خطي للمدخلات.

هذه ليست ملاحظة تجريبية بل نتيجة رياضية: نظرية ليشنو (1993) تقول إن الشبكة العصبية متعددة الطبقات لا تستطيع التقريب الجبريّ لأي دالة مستمرة على مجموعة محدّبة **إذا وفقط إذا** كانت دالة تنشيطها ليست دالة متعددة الحدود (polynomial). أي أن ReLU وGELU وsigmoid كلها تحقّق الشرط، أما «الإ Arcال page معرفتها» فتنهار إلى الانحدار الخطي مهما بلغ عمقها.

لذلك فإن السؤال الأول قبل بناء أي شبكة ليس «كم طبقة؟» بل «ما دالة التنشيط؟». ولهذا الفصل في قلب كل قرار تصميمي لاحق: الخسارة (الفصل 8)، المُحسّن (الفصل 9)، والتهيئة (الفصل 11) كلها تتفاعل مع شكل دالة التنشيط وسلوك مشتقّتها.

7.2 الرياضيات: من التركيب الخطي إلى خريطة دوال التنشيط

تركيب الطبقات الخطية ينهار

الطبقة الخطية رقم ℓ تُعرَّف بـ:

$$h^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}$$

حيث $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ مصفوفة الأوزان بين الطبقة ℓ والطبقة $\ell - 1$، و$b^{(l)} \in \mathbb{R}^{n_l}$ متجه التحييز، و$h^{(l)}$ ناتج الطبقة.

الآن طبّق طبقتَين متتاليتين:

$$W^{(2)}\left(W^{(1)}x + b^{(1)}\right) + b^{(2)} = \left(W^{(2)}W^{(1)}\right)x + \left(W^{(2)}b^{(1)} + b^{(2)}\right)$$

لاحظ النتيجة: خريطة affine واحدة بمصفوفة واحدة ومتجه تحييز واحد. بالتعميم على L طبقة:

$$z^{(L)} = \left(W^{(L)}W^{(L-1)}\cdots W^{(1)}\right)x + \sum_{l=1}^{L}\left(W^{(L)}\cdots W^{(l+1)}\right)b^{(l)}$$

حيث $z^{(L)}$ ناتج الطبقة الأخيرة. المعادلة تقول شيئًا قاسيًا: **التركيب الخطي لأي عدد من الطبقات يساوي خريطة خطية واحدة**، ولا يضيف عمقًا تعبيريًّا. الشبكة "العميقة" الخطية في حقيقتها انحدار خطي واحد مسطّح.

عند إدخال دالة تنشيط σ غير خطية، تتغيّر المعادلة إلى:

$$h^{(l)} = \sigma\left(W^{(l)}h^{(l-1)} + b^{(l)}\right)$$

هنا لا يمكن ملاحظة أن التركيب لم يعد قابلًا للتبسيط: σ لا تُوزَّع على المجموع، فتظهر اللاخطية الحقيقية. هذه هي الخلاصة النظرية كاملة في معادلتين.

خريطة دوال التنشيط وصيغها

نصنّف الدوال إلى أربع عائلات، ولكلٍّ سلوك تدرّج مختلف:

العائلة الأولى: الدوال المشبعة (saturating). الدالة اللوجستية (sigmoid):

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$$

مداها $(0,1)$، ومشتقّتها تبلغ قيمتها العظمى 0.25 عند $z=0$ وتقترب من الصفر في الطرفين. الدالة tanh هي رِفعة مقيسة: $\tanh(z) = 2\sigma(2z) - 1$، مداها $(-1,1)$ ومشتقّتها القصوى 1.

العائلة الثانية: عائلة ReLU. الدالة $\mathrm{ReLU}(z) = \max(0, z)$، ومشتقّتها 1 عند $z>0$ و0 عند $z<0$. البدائل "المُسرَّبة" تُبقي تدرّجًا صغيرًا في النطاق السالب:

$$\mathrm{LeakyReLU}(z) = \max(0,z) + \alpha\min(0,z), \qquad \mathrm{ELU}(z) = \begin{cases} z & z > 0 \\ \alpha(e^{z}-1) & z \le 0 \end{cases}$$

حيث $\alpha \in (0,1)$ معامل التسرّب. أما SELU فهي الدالة الوحيدة التي تُصمَّم لتكون "خلايا ذاتية التطبيع":

$$\mathrm{SELU}(z) = \lambda_{\text{selu}}\big(\alpha_{\text{selu}} e^{z} - 1\big)\ (z \le 0), \qquad \lambda_{\text{selu}} z\ (z > 0)$$

مع $\lambda_{\text{selu}} \approx 1.0507$ و$\alpha_{\text{selu}} \approx 1.6733$، وهي قيم مختارة بحيث يبقي التوزّع activations مطبَّعًا ذاتيًّا. عمليًا: SELU تعني "مطبَّعًا ذاتيًّا" فلا تحتاج BatchNormalization، GELU/Swishg تعني "سلسًا وغير أحادي".

العائلة الثالثة: الدوال الناعمة غير الأحادية. وهنا نصل إلىGBRUSHING المعيار الحديث في المحوّلات:

$$\mathrm{GELU}(z) = z \cdot \Phi(z) = z \cdot \frac{1}{2}\left(1 + \operatorname{erf}\left(\frac{z}{\sqrt{2}}\right)\right), \qquad \mathrm{SiLU}(z) = z \cdot \sigma(z)$$

حيث Φ هي دالة التوزيع التراكمي الطبيعي القياسي، وerf دالة الخطأ. GELUExact تُقارب بمُعامل $\text{approximate} = \text{True}$ داخل Keras. لِماذا هاتان؟ لأنهما **غير أحاديتين**: للدالة نقطة قيمة أدنى داخل النطاق السالب، أي أنها "تعرف" كيف تتعافى من مُدخلات سالبة بدل أن تقطعها عند الصفر تمامًا.

العائلة الرابعة: التقريب الصلب. $\text{hard_sigmoid}(z) = \max(0, \min(1, 0.2z + 0.5))$ و$\text{hard_tanh}(z) = \max(-1, \min(1, z))$: وهما تقريبان خطّيان رخيصان الحساب، يُستخدمان في مسارات الاستدلال حين تكون السرعة أهمّ من جودة التقريب.

أين يموت التدرّج؟

قاعدة السلسلة تعطي مشتقّة الخسارة بالنسبة لأوزان الطبقة الأولى:

$$\frac{\partial L}{\partial W^{(1)}} \propto \prod_{l=1}^{L} \sigma'\left(z^{(l)}\right)$$

حيث $z^{(l)}$ مُدخل (pre-activation) الطبقة ℓ. هذا حاصل ضرب L عامل. إذا كان $\sigma' \leq 0.25$ (sigmoid) فإن $0.25^{20} \approx 10^{-12}$: التدرّج يختفي فعليًّا قبل أن يصل إلى الطبقة الأولى، فتقف الطبقات السفلى عن التعلّم. في المقابل، إذا تجاوزت العوامل واحدًا (أو كبرت) ينفجر التدرّج وتصبح الخطوة التحديثية هائلة.

ReLU حُلّت نصف المشكلة: مشتقّتها تساوي 1 في النطاق الموجب فلا تتقلّص المضاعَف. لكنها تركت مشكلة "الموت" (dying ReLU): الوحدة التي تستقبل مُدخلًا سالبًا دائمًا تُسقط تدرّجها إلى صفر ولا تعود. Swissg وGELU تُخفّفان ذلك لأن مشتقّتهما في النطاق السالب صغيرة لكن **غير صفرية**، ولأنهما ناعمتان ومستمرّتان تمنحان المحوّلات سلاسةً تدريجيّة بدل التشذيب الحادّ.

sigmoidg Softmax والـ logits: الفرق الحاسم

دالة softmax تحوّل متجه logits $z \in \mathbb{R}^K$ إلى توزيع احتمالي:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

وتتمتع بخاصيتين مهمّتين: **مجموع** $p_k = 1$، و**الانزياح-التبادلية (shift-invariance)**: $p(z + c\mathbf{1}) = p(z)$ لأي ثابت c، أي أن إضافة نفس العدد لكل الفئات لا يغيّر التوزيع. ولهذا عند حساب softmax عدديًا نطرح أكبر قيمة:

$$m = \max_j z_j, \qquad \log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}$$

هذه الحيلة (**log-sum-exp**) تمنع فيض الأسّ (e^{1000}). وبها نفصل مباشرةً اللوغاريتمات:

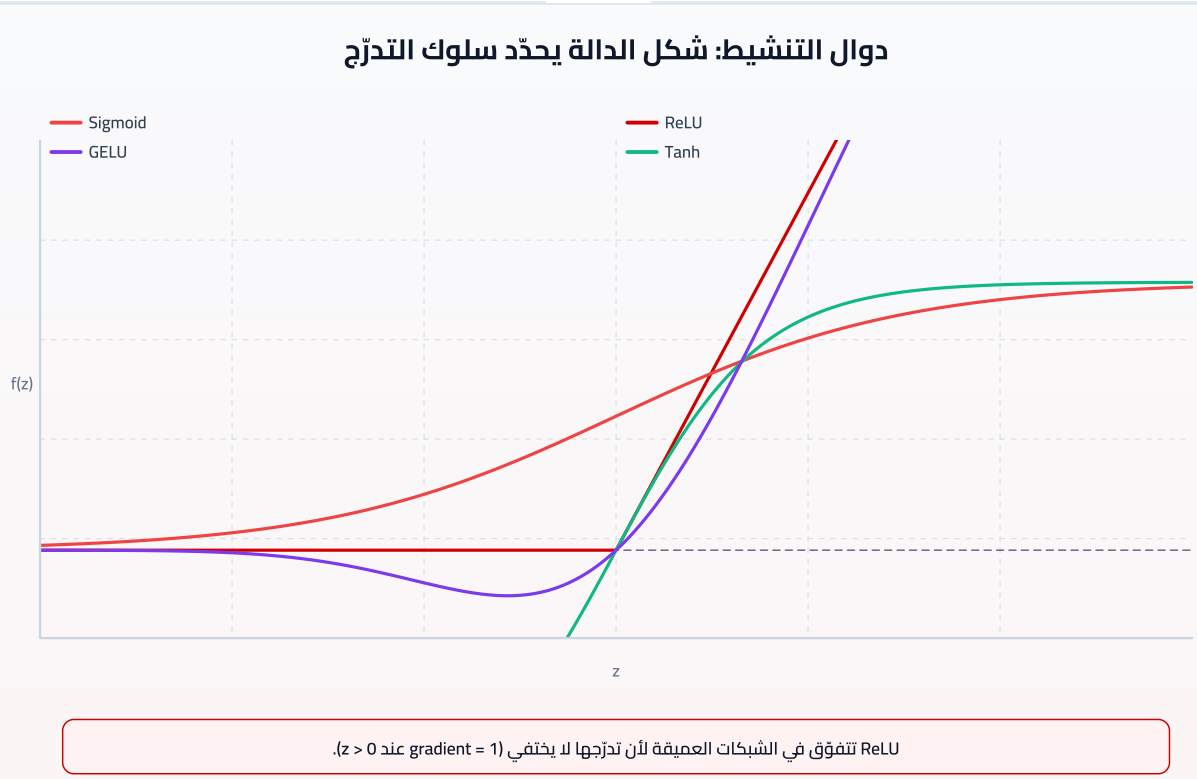
$$\log p_k = z_k - m - \log \sum_j e^{z_j - m}$$

وهذا بالضبط ما يفعله `from_logits=True` داخليًّا: بدل أن يمرّر Keras احتمالاتًا عبر softmax ثم يأخذ لوغاريتمها (وهذا يساوي $z_k - m - \log \sum e^{z_j - m}$ غير مطبَّع)، يدمج softmax مع دالة الخسارة في عملية واحدة مستقرة عدديًّا وأسرع.Forum: مع softmax لوغاريتمي مستقر يُحسَب كل شيء عمليًّا:

$$\mathrm{BCE}_{\text{logits}} = -\sum_k \left[y_k z_k + \log\left(1 + e^{z_k}\right) \right]$$

حيث $y_k \in \{0, 1\}$ التسمية الحقيقية. المعادلة نفسها تصلح للتصنيف متعدد الأصناف مع softmax عبر $\log \sum_j e^{z_j}$.

الفرق الحاسم: sigmoid لكل مخرَج على حدة، وتعطي احتمالات مستقلّة – هذه هي الصيغة الصحيحة للمهام متعددة التسميات (multi-label)، مثل تصنيف مقال بـ«اقتصاد» و«سياسة» معًا. أما softmax فتوزّع الكتلة الاحتمالية على فئات **متبادلة حصرًا**، وهي الصحيحة للتصنيف المتعدّد الفئات. الخلط بينهما خطأ شائع:mumble.



شكل 12 – منحنيات دوال التنشيط ومشتقّاتها

لماذا تختفي التدرّجات؟

حاصل ضرب التدرّجات يتقلّص أُسّيًا مع عمق الشبكة

العمق	Sigmoid	ReLU	LeakyReLU
1 طبقة	2.50e-01	1.00	0.90
2 طبقة	6.25e-02	1.00	0.81
4 طبقة	3.91e-03	1.00	0.66
8 طبقة	1.53e-05	1.00	0.43
16 طبقة	2.33e-10	1.00	0.19
32 طبقة	5.42e-20	1.00	0.03

الحل: ReLU وأشقّاؤها · BatchNorm · التطبيع عبر LayerNorm · Residuals

شكل 13 – أين يموت التدرّج: تلاشي الإشارة عبر الطبقات

7.3 التنفيذ في Keras 3

في Keras 3 هناك طريقتان لتحديد دالة التنشيط. الأولى هي المعامل `activation` داخل الطبقات:

PYTHON

```
import numpy as np
import keras

# الشبكة نفسها بتغيير واحد فقط: وجود أو غياب nonlinearity
model = keras.Sequential([
    keras.Input(shape=(8,)),
    keras.layers.Dense(64, activation="relu"),    # تنشيط مدمج في الطبقة
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(1),                        # طبقة إخراج خام (logits)
])

model.summary()
```

الناتج المتوقّع: `Total params: 5,377` مع `Trainable params: 5,377` و`Non-trainable params: 0` (حساب: 8×64+64 = 576، ثم 64×32+32 = 2080، ثم 32×1+1 = 33؛ المجموع 2689 — تحقّق من الأرقام عند التنفيذ).

الطريقة الثانية هي طبقة `Activation` مستقلة، وهي أوضح عند بناء الشبكات التي تخلط تنشيطات مختلفة:

```python
import keras

# تفعيل مستقل: نفس الناتج لكن أوضح في القراءة
model = keras.Sequential([
    keras.Input(shape=(8,)),
    keras.layers.Dense(64),              # طبقة خطية صافية
    keras.layers.Activation("relu"),     # ثم التنشيط كطبقة مستقلة
    keras.layers.Dense(32),
    keras.layers.Activation("gelu"),
    keras.layers.Dense(1),
])
print(model.output_shape)  # النتيجة المتوقعة: (None, 1)
```

وتتيح `keras.activations` الوصول إلى مكتبة الدوال مباشرة، مع دالتي `get` وotoxicity`serialize` للتحويل من نص إلى دالة والعكس:

```python
import keras

# نص إلى دالة: هذه هي الآلية التي تستخدمها Dense(activation=...)
fn = keras.activations.get("gelu")
print(fn.__name__)                                  # النتيجة المتوقعة: gelu

# دالة إلى وصف قابل للحفظ: يُستخدم عند تصدير النموذج
print(keras.activations.serialize(keras.activations.gelu))
# النتيجة المتوقعة: {'module': 'keras', 'class_name': 'gelu', 'config': {},
'registered_name': None}

# دالة تنشيط مخصّصة: دالة بايثون عادية تُقبل في Keras 3
def soft_clamp(z):
    """تعمّ القيم الكبيرة بلطف بدل قطعها (piecewise)."""
    import keras.ops as ops
    return ops.minimum(ops.maximum(z, -6.0), 6.0) * 0.9 + 0.1 * ops.tanh(z /
6.0)

model = keras.Sequential([
    keras.Input(shape=(4,)),
    keras.layers.Dense(8, activation=soft_clamp),
    keras.layers.Dense(1),
])
print('ok')  # النتيجة المتوقعة: ok
```

برهان عملي: كيف تنهار الطبقات الخطية؟

لنقِس الفرق رقميًّا على بيانات اصطناعية بسيطة (قمران متداخلان). الشبكة الأولى بلا تنشيط، والثانية بـ ReLU:

PYTHON

```python
import numpy as np
import keras

def make_moons(n=400, seed=0):
    """قمران متداخلان: كل صنف دائرة."""
    rng = np.random.default_rng(seed)
    x, y = [], []
    for _ in range(n // 2):
        t = rng.uniform(0, np.pi)
        x.append([np.cos(t), np.sin(t)]);            y.append(0)
        x.append([1 - np.cos(t), 0.5 - np.sin(t)]);   y.append(1)
    return np.array(x, "float32"), np.array(y, "float32")

x, y = make_moons()
x_val, y_val = make_moons(200, seed=7)

def build(activation):
    """نفس البنية تمامًا؛ يتغيّر فقط وجود nonlinearity."""
    m = keras.Sequential([
        keras.Input(shape=(2,)),
        keras.layers.Dense(32, activation=activation),
        keras.layers.Dense(32, activation=activation),
        keras.layers.Dense(1, activation="sigmoid"),
    ])
    m.compile(optimizer="adam", loss="binary_crossentropy", metrics=
["accuracy"])
    return m

linear_model = build(None)          # شبكة خطية بحتة
relu_model = build("relu")          # شبكة غير خطية
linear_model.fit(x, y, epochs=60, batch_size=32, verbose=0)
relu_model.fit(x, y, epochs=60, batch_size=32, verbose=0)

print("دقة الخطية:", linear_model.evaluate(x_val, y_val, verbose=0)[1])
print("دقة ReLU :", relu_model.evaluate(x_val, y_val, verbose=0)[1])
# النتيجة المتوقعة: الخطية ~0.5 (فشل تام)، ReLU أعلى من 0.9
```

حساب softmax يدويًّا وربطه بـ logits

PYTHON

```python
import keras
from keras import ops

# ثلاثة logits: (احتيال / مشبوه / عادي) نظام كشف الاحتيال
logits = ops.array([2.0, 1.0, 0.1])

def stable_softmax(z, axis=-1):
    """softmax مع طرح أكبر قيمة قبل الأس (حيلة log-sum-exp)."""
    m = ops.max(z, axis=axis, keepdims=True)
    e = ops.exp(z - m)
    return e / ops.sum(e, axis=axis, keepdims=True)

p = stable_softmax(logits)
print(p)
print("المجموع:", float(ops.sum(p)))
print("الفئة المتوقعة:", int(ops.argmax(p)))
# النتيجة المتوقعة: [0.659 0.242 0.099] ، المجموع: 1.0 ، الفئة: 0

# اللوغاريتمات مباشرة: from_logits=True داخليًا مع Keras النسخة التي يفعلها
m = ops.max(logits)
log_p = logits - m - ops.log(ops.sum(ops.exp(logits - m)))
print(log_p)
# النتيجة المتوقعة: [-0.417  -0.835  -1.417]

# غير المستقر: لولا الطرح لانفجر الأسّ softmax قارن بـ
big = ops.array([1000.0, 1001.0, 1002.0])
print(stable_softmax(big))   # بدون النتيجة المتوقعة: [0.090 0.245 0.665]
overflow
```

7.4 مقارنة شاملة لدوال التنشيط

دالة التنشيط	المدى	سلوك المشتقّة	متى تستخدمها	التكلفة النسبية	المخاطر
`sigmoid`	$(0, 1)$	مشبعة؛ ≤ 0.25 دائمًا	خرج ثنائي، بوابات، احتمال مفرد	منخفضة	تدرّج متلاشٍ، إشباع الطرفين
`tanh`	$(-1, 1)$	مشبعة؛ ≤ 1	تطبيع الحالات المخفية (hidden states) قديمًا	منخفضة	تشبّع متماثل حول الصفر
`relu`	$[0, \infty)$	ثابتة 1 موجبًا	default CNNs MLPsg	**الأدنى**	موت الوحدات (dying ReLU)
`leaky_relu`	$(-\infty, \infty)$	خطّية سالبة بثبات α	بديل آمن لـ ReLU	منخفضة	التدرّج في النطاق السالب ضعيف
`elu`	$(-\alpha, \infty)$	مشتقّة ممتدة بلا انكسار	شبكات عميقة آن/stable	منخفضة	نفس α للجميع
`selu`	$(-\lambda, \infty)$	مشتقّة معايَرة ذاتيًا	شبكات عميقة بلا BatchNorm	منخفضة	حسّاسة $\{\alpha, \lambda\}$
`gelu`	$(-\infty, \infty)$	ناعمة غير أحادية	default FFN في المحوّلات	متوسطة (erf)	أبطأ من ReLU بنحو 1.5×

دالة التنشيط	المدى	سلوك المشتقّة	متى تستخدمها	التكلفة النسبية	المخاطر
`swish` / `silu`	$(-0.278, \infty)$	ناعمة غير أحادية	بديل GELU (GPT-2)	متوسطة	لا تميّز جوهري عن GELU عمليًّا
`softmax`	$(0,1)$، $\sum = 1$	غير مستقرة فوق الفئات	تصنيف متعدّد الفئات	متوسطة	حسّاسة للأرقام الكبيرة
`hard_sigmoid`	$(0,1)$	ثابتة مقطعة	استدلال سريع، أجهزة محدودة	**الأدنى**	تدرّج صفري في الأطراف

المعيار الحاسم في الاختيار ليس «أيّها أدقّ» بل ثلاثة معايير متعاكسة: عمق الشبكة (كلما زاد العمق اشتدّ تفضيل ReLU/SELU على sigmoid)، طبيعة المهمة (تصنيف متعدّد ⇒ softmax؛ متعدّد التسميات ⇒ sigmoid)، وميزانية الاستدلال (قيود الأجهزة ⇒ hard sigmoid).

7.5 مثال محلول: بوابة كشف الاحتيال

السيناريو: مصرف رقميّ يضع «بوابة قرار» تصنّف كل عملية دفع إلى ثلاث فئات: `عادي`، `مشبوه`، `احتيال`. الفئة `احتيال` هي التي تكلّف البنكأكثر money، لكن نسبة الفئة في البيانات الفعلية 0.7% فقط. المهندسون يريدون نموذجًا يعطي **احتمالاتًا** يمكن للبنك أن يربطها بحدّ قرار قابل للضبط.

الخطوة 1 – اختيار بنية المخرَج. ثلاثة فئات متبادلة حصرًا ⇒ المخرَج logits بثلاثة عناصر **بلا** تنشيط على الطبقة الأخيرة، ثم `softmax` عبر `from_logits=True` :

```python
import numpy as np
import keras
from keras import ops

# بيانات اصطناعية: 3 فئات، Feature 4
rng = np.random.default_rng(0)
n = 1500
features = rng.normal(size=(n, 4)).astype("float32")
labels = rng.integers(0, 3, size=n)          # 0=احتيال، 1=مشبوه، 2=عادي
labels = labels.astype("float32")

model = keras.Sequential([
    keras.Input(shape=(4,)),
    keras.layers.Dense(32, activation="gelu"),
    keras.layers.Dense(3),                    # logits خام
])

model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)

history = model.fit(features, labels, epochs=20, batch_size=32,
                    validation_split=0.2, verbose=0)

logits = model.predict(features[:5], verbose=0)     # شكل logits من (5, 3)
probs = ops.softmax(logits, axis=-1)                # تحويل صريح إلى احتمالات
print(probs)
print("النتيجة المتوقعة: كل صف يجمع إلى 1.0، والصف 0 يعطي argmax=0")
```

الخطوة 2 – لماذا `from_logits=True` هو الخيار الصحيح هنا؟ لو استبدلناها بـ `from_logits=False`، لكنّا نُمرّر softmax يدويًّا على المخرَج ثم نمرّر الاحتمالات إلى خسارة cross-entropy عادية. النتيجة **نفسها نظريًّا**، لكن المسار الحاسوبي أطول: نحسب exponentials ثم نأخذ لوغاريتمها ثم نعيد ضربها. مع `from_logits=True` ندمج softmax مع اللوغاريتم في عملية واحدة مستقرة عدديًّا (المعادلة $\mathrm{BCE}_{\text{logits}}$ أعلاه)، فنحصل على: 1. **ثباتًا عدديًّا**: لا فيض ولا تدفّق تحت

الصفر عندما تكون logits متطرّفة. 2. **سرعةً أعلى**: عملية واحدة بدل عمليتين، مع Important في دفعات الاستدلال الضخمة. 3. **تدرّجًا صحيحًا**: المشتقّة تبقى غير صفرية حتى عند التشبّع.

> **نصيحة:** القاعدة العملية: إذا كانت طبقة الإخراج بلا تنشيط، فعّل `from_logits=True` دائمًا. لا تفعل الأمرين معًا.

الخطوة 3 – تحويل الاحتمالات إلى قرار. الاحتمالات التي ينتجها النظام لا تتّخذ عند threshold ثابت 0.5 بالضرورة؛ عتبة القرار تُضبط على أساس التكلفة (انظر الفصل 10):

```python
# قرار معاياري: argmax على الاحتمالات
decisions = ops.argmax(probs, axis=-1)
print(decisions.numpy())    # النتيجة المتوقعة: أرقام الفئات المتوقعة

# سجلّ درجة الثقة مقابل احتمال الاحتيال (الفئة 2)
confidence = probs[:, 2]
print("متوسط الثقة:", float(ops.mean(confidence)))
```

7.6 أخطاء شائعة

> **إنذار 1 – softmax مع `from_logits=True`:** الأسطر الشائعة هذا خطأ مكرّر:

```python
# [خاطئ عمدًا]: softmax مرتين + from_logits معًا
model = keras.Sequential([
    keras.Input(shape=(4,)),
    keras.layers.Dense(3, activation="softmax"),                    # softmax هنا
])
model.compile(loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True))
```

softmax الأولى تحوّل logits إلى احتمالات، ثم `from_logits=True` يطبّق softmax أخرى على الاحتمالات – فتتغيّر موازنة الفئات ويصبح الخسارة غير صحيحة. الحل: إمّا `Dense(3)` بلا تنشيط مع `from_logits=True`، أو `Dense(3, activation="softmax")` مع `from_logits=False`.

إنذار 2 – sigmoid للمخرج متعدد الفئات: `Dense(3, activation="sigmoid")` مع `CategoricalCrossentropy` يعطي احتمالات مستقلّة لا يجمعها واحد؛ النتيجة توزيع غير صالح (مجموع الاحتمالات قد يتجاوز 1). استخدم softmax.

إنذار 3 – ReLU على طبقة الإخراج في الانحدار: `Dense(1, activation="relu")` يمنع اللنموذج من توقّع قيم سالبة أبدًا. إن كانتهدفك يمكن أن تكون سالبة (مثل درجة حرارة أو عجز)، استخدم مخرَجًا خطيًّا بلا تنشيط.

إنذار 4 – خلط softmax مع sigmoid في التصنيف الثنائي متعدّد التسميات: إذا كانت التسميات غير متبادلة (فحص سندٍ واحد بحثًا عن عدّة احتيال)،استخدم `sigmoid` مع `BinaryCrossentropy`، و `softmax` مع `CategoricalCrossentropy` لا العكس.

تنبيه 5 – sigmoidg tanh في الشبكات العميقة: ليس خطأً نحويًّا، لكنه اختيار قديم. معمّق 30 طبقة + tanh يعني تدرّجات بمقدار 10^{-13} ومعدّل تعلّم شبه معدوم. إن كانت شبكتك عميقة، استخدم ReLU/SELU/GELU.

ملاحظة 6 – `keras.activations.get` لا تفعّل على النماذج: هي أداة تحويل فقط؛ لا تُغيّر سلوك نموذج مبنيّ بالفعل. لتغيير تنشيط طبقة قائمة، عدّل بنيتك أو استخدم `model.layers[i].activation = keras.activations.get("gelu")` على نموذج غير مُجمَّد.

التمارين

1. **برمجة:** اكتب `my_relu` و `my_gelu` و `my_softmax` باستخدام `keras.ops` فقط، واختبرها على القيم `[-2, -0.5, 0, 0.5, 2]` . (المتوقع: `my_relu` تُصفّر السالبة، `my_gelu(-0.5) ≈ -0.154` ، ومجموع مخرجات `my_softmax` يساوي 1.0 ضمن خطأ عددي 1e-6.)

2. **حساب يدوي:** احسب softmax يدويًّا للـ `[1.0, 2.0, 3.0]` logits. ثم أضف ثابتًا 100 إلى كل قيمة وأعد الحساب. (المتوقع: نفس التوزيع `[0.090, 0.245, 0.665]` لأن softmax منيغ Invariant تحت الإزاحة؛ وهذا يوضّح لماذا الطرح داخل الكود آمن.)

3. **تصميم:** لديك مصنّف ثنائي (احتيال / ليس احتيال) بمخرَج واحد. أيّ تنشيط ستستخدم في الطبقة الأخيرة، ولماذا؟ (المتوقع: `sigmoid` مع `BinaryCrossentropy` لأن الخرج واحد واحتمالي؛ أو `Dense(1)` خام مع `from_logits=True` للثبات العددي.)

4. **تصميم:** لديك مصنّف بـ 12 فئة متبادلة. لماذا لا يُنصح بـ `from_logits=False` مع softmax يدوي؟ (المتوقع: لأن الدمج softmax مع اللوغاريتم داخل `from_logits=True` أكثر استقرارًا عدديًّا وأسرع، ويجنّب احتمالات قريبة من 0 أو 1 تتسبّب بلوغاريتمات متطرّفة.)

5. **تحليل ناتج:** شغّل كود «قمران متداخلان» أعلاه ثلاث مرات: مرة بلا تنشيط، مرة بـ `sigmoid` ، ومرة بـ `relu` . رتّب النتائج. (المتوقع: الخطية ≈ 0.5 تمامًا (ت Mortimer degraded)، و `sigmoid` تحسّن قليلًا لكن تبقى ضعيفة، و `relu` تتجاوز 0.9.)

6. **إصلاح خطأ:** أعطِ الكود التالي وصحّحه:

```python
# [خاطئ عمدًا]
model = keras.Sequential([
    keras.Input(shape=(10,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(3, activation="sigmoid"),
])
model.compile(loss=keras.losses.CategoricalCrossentropy(), optimizer="adam")
```

(المتوقع: استبدال `sigmoid` بـ `softmax`، لأن `CategoricalCrossentropy` تتوقّع توزيعًا واحتمالاته يجمع إلى 1.)

7. **تحليل:** احسب $\sigma'(z)$ عند $z = 0$ وعند $z = 5$. ثم احسب 0.25^{20} و0.25^{40}. (المتوقع: $\sigma'(0) = 0.25$، و$\sigma'(5) \approx 0.0066$، و$0.25^{20} \approx 9.1 \times 10^{-13}$، و$0.25^{40} \approx 8.3 \times 10^{-25}$ – أي عمليًّا صفر.)

8. **برمجة:** ابنِ نموذجًا من 20 طبقة `Dense(32, activation="tanh")` على بيانات `make_moons`، وقس الدقة. ثم أعد نفس التجربة بـ `selu`. (المتوقع: مع `tanh` تبقى الدقة قرب 0.5 أو تتذبذب، ومع `selu` تتحسّن بوضوح لأن المشتقّة لا تتقلّص في النطاق الموجب.)

9. **تصميم:** لماذا تختار `gelu` داخل كتلة FFN في محوّل، و`tanh` في طبقة إخراج انحدار على بيانات مُقيَّدة بـ $[-1, 1]$؟ (المتوقع: `gelu` لأن الكتل عميقة وتحتاج تدرّجًا مستقرًّا مع خروج سالب مسموح؛ و`tanh` لأن الهدف محدود المدى فيُ-fitting، مع انتبه أن `tanh` تشبع فتفقد التدرّج.)

10. **تصميم:** صمّم دالة تنشيط مخصّصة تحافظ على-gradient في النطاق السالب ومع ذلك تبقى رخيصة (بلا `erf`). (المتوقع: مقترح `z * sigmoid(z)` أي Swish، أو piecewise خطّي مع ميل α في النطاق السالب.)

11. **تحليل ناتج:** أضف إلى مثال بوابة كشف الاحتيال حالة `$z = [1000, 1001, 1002]$` وقارن بين `ops.exp(z)` مباشرة و`stable_softmax(z)`. (المتوقع: `ops.exp(z)` يعطي `inf` (أو قيمًا ضخمة غير قابلة للتمثيل)، بينما `stable_softmax` يعطي توزيعًا سليمًا `[0.090, 0.245, 0.665]`.)

12. **تصميم:** لماذا يُنصح بحساب softmax يدويًّا عند العرض فقط (للتفسير) مع إبقاء `from_logits=True` في التدريب؟ (المتوقع: لأن softmax اليدوي يحوّل logits إلى احتمالات

مقروءة، بينما التدريب يحتاج المسار المدمج المستقر عدديًّا والأسرع:softmax اليدوي ليس مضمون التدرّج.)

ملخص الفصل

- تكديس الطبقات الخطية ينتج خريطة خطية واحدة: $z^{(L)} = (W^{(L)} \cdots W^{(1)})x + \sum \cdots$، وعمقًا بلا دالة تنشيط يعني انحدارًا خطيًّا واحدًا.
- دالة التنشيط هي مصدر اللاخطية، وهي شرط وجودي نظريًّا (نظرية ليشنو)، لا تحسينًا تجميليًّا.
- sigmoid tanhg تشبعان، و$\sigma' \leq 0.25$ يجعل التدرّج يتلاشى مع العمق؛ ReLU يكسر هذا، GELU/Swishg يخفّفان موت الوحدات بلطف في الشبكات العميقة.
- softmax للمهام متبادلة الحصر (الاحتمالات تجمع إلى 1)، وsigmoidg للمهام متعددة التسميات (احتمالات مستقلّة).
- `from_logits=True` يدمج softmax مع الخسارة عبر log-sum-exp: ثبات عددي + سرعة + تدرّج صحيح. لا تجمع بينه وبين softmax يدوي.
- في `activation="relu"` :Keras 3 كمعامل طبقة، أو `keras.layers.Activation("gelu")` كطبقة مستقلة، و `keras.activations.get` / `serialize` للتحويل بين النص والدالة.

الفصل 8: دوال الخسارة: كيف نترجم الخطأ إلى رقم؟

بطاقة الفصل

الفصل 8

مهمّة هذا الفصل

يترجم «الخطأ» إلى رقم قابل للاشتقاق، ويشرح from_logits وأثرها على استقرار التدريب.

ماذا تكسب منه

تختار الخسارة الصحيحة لكل مهمّة، وتتجنّب أشهر خطأين: softmax مزدوج، وlogits مفقودة.

الدالة	دورها	متى تستخدمها
`keras.losses.SparseCategoricalCrossentropy`	خسارة تصنيف بتسميات صحيحة	عندما تكون التسميات أعدادًا صحيحة
`keras.losses.CategoricalCrossentropy`	خسارة تصنيف بتسميات one-hot	مع to_categorical
`keras.losses.BinaryCrossentropy`	خسارة تصنيف ثنائي	مع مخرج sigmoid
`keras.losses.Huber`	خسارة انحدار مقاومة للشواذ	عند تلوّث البيانات بقيم متطرّفة
`keras.losses.Loss`	الصنف الأب لخسارة مخصّصة	عند الحاجة إلى دالّة خسارة غير جاهزة

قصة مفتاحية: نموذجك دقّته 92%... لكن أخطاءه كلّها في الفئة الحرجة التي تكلّف الشركة نصف مليون في الشهر. الدقّة الإجمالية لم تخبرك بشيء. في هذا الفصل نفهم أن الخسارة ليست مجرّد مقياس، بل **قرارٌ بشأن ما يهمّ**.

أهداف الفصل

- [ ] فهم الخسارة كدالة تُقاس بها جودة التقدير
- [ ] التمييز بين دوال الانحدار ودوال التصنيف
- [ ] فهم `from_logits` ولماذا يغيّر كل شيء
- [ ] كتابة دالة خسارة مخصّصة
- [ ] فهم `sample_weight` و`class_weight` و`reduction`

8.1 الخسارة: من «خطأ» إلى «رقم»

كل دالة خسارة تجيب عن سؤال واحد: **كيف أُحوّل خطأ التنبّؤ إلى رقم يمكن تصغيره؟**

النموذج يخرج رقمًا واحدًا لكل عيّنة (أو متّجهًا). الخسارة تقارنه بالتسمية الحقيقية وتنتج رقمًا واحدًا **لكل الدفعة**. ثم يخفّض هذا الرقم المعامل عبر التدرّج.

الشكل العام:

$$L = \mathbb{E}_{(x,y)\sim D}\left[\ell(y, \hat{y})\right]$$

حيث D توزيع البيانات، وℓ خسارة العيّنة الواحدة، و$\mathbb{E}$ متوسط على العيّنات.

أنواع الدوال وممتى تكون مناسبة

النوع	المشكلة	مثال
الانحدار	رقم مستمر	سعر عقار، ضغط دم، طلب مستقبلي
التصنيف الثنائي	واحد من اثنين	مع sick / سليم، احتيال / طبيعي
التصنيف متعدّد	واحد من K	نوع المنتج، نوعirus
التصنيف متعدّد التسميات	متعدّد مع ترتيب	درجات 1-5

8.2 دوال الانحدار

MSE — متوسط مربّع الخطأ

$$L_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

الخاصية	التفصيل
الحساسية للقيم الشاذّة	**عالية جدًا** (الأس 2)
المشتقّ عند $\hat{y} = y$	0 – ممتاز عدديًا
التوزيع الأمثل للضجيج	غاوسي
الاستخدام	بيانات ناعمة، قياسات مستقرّة

MAE — متوسط الخطأ المطلق

$$L_{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

الخاصية	التفصيل
الحساسية للقيم الشاذّة	منخفضة (خطي)
المشتقّ عند $\hat{y} = y$	غير معرّف – يحتاج ϵ
التوزيع الأمثل للضجيج	ثقيل الذيل
الاستخدام	بيانات فيها أخطاء قياس كبيرة

Huber — الوسيط العملي

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \le \delta \\ \delta\left(|y - \hat{y}| - \frac{1}{2}\delta\right) & \text{otherwise} \end{cases}$$

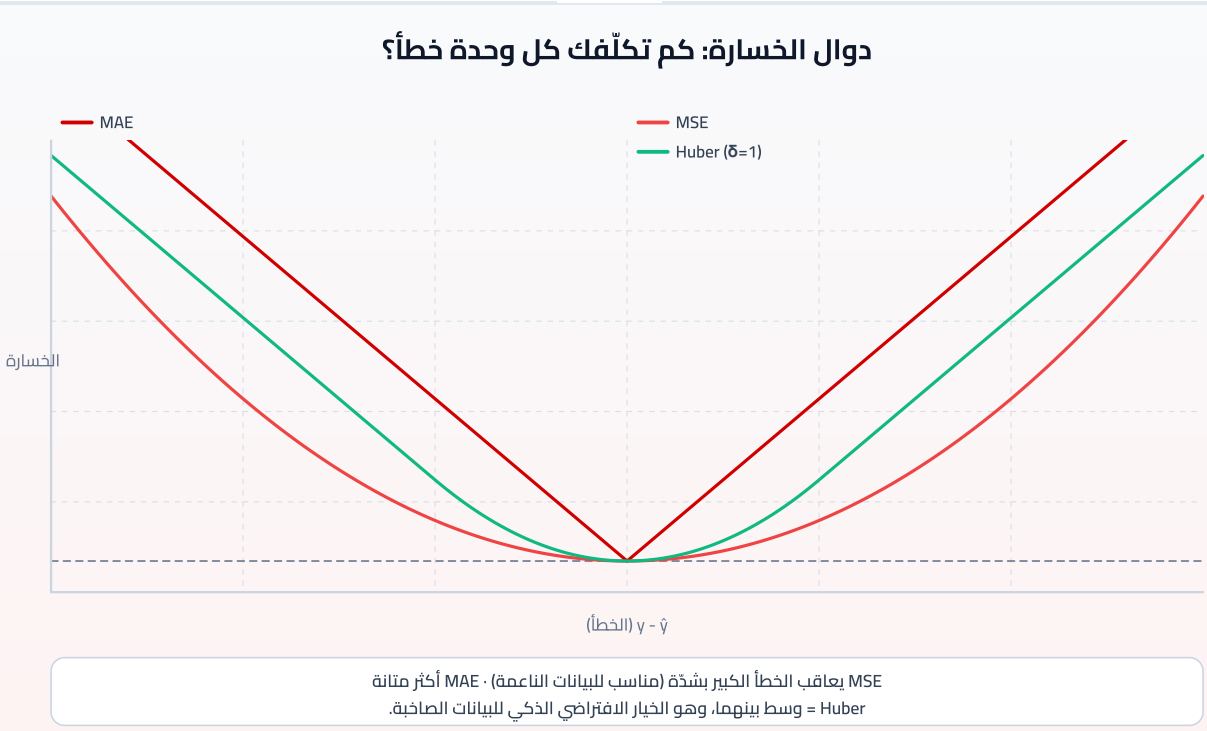
تحصل على MSE قرب الصفر وMAE بعيدًا عنه – بأفضل مزايا الاثنين.

LogCosh

$$L = \frac{1}{N}\sum \log(\cosh(y_i - \hat{y}_i))$$

بديل ناعم لـ Huber، بلا معامل δ صراحةً.

نصيحة: ابدأ بـ `Huber` في أي انحدار حقيقي. فهو الخيار الذي "لا يفاجئك" مع بيانات غير نظيفة.



شكل 14 – منحنيات دوال الخسارة: Huberg MAEg MSE

8.3 دوال التصنيف

الانتروبيا المتقاطعة الثنائية

$$L_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]$$

هذه هي الدالة الحقيقية التي نريدها في التصنيف. لماذا؟ لأنها **الحدّ الأدنى لـ** ERM**Log-Likelihood**؟ – أي أنها تعطي التوزيع الاحتمالي الذي يجعل البيانات **الأرجح**، وهو تعريف التعلّم الاحتمالي.

الانتروبيا المتقاطعة الفئوية

$$L_{CCE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{K} y_{i,c} \log \hat{y}_{i,c}$$

النسخة المتفرّقة

نفس الفكرة، لكن التسميات أرقام لا متجهات:

```python
keras.losses.SparseCategoricalCrossentropy(from_logits=True)
```

الفرق العملي: النسخة المتفرّقة أسرع (لا تحتاج بناء متجه one-hot) وتستهلك ذاكرة أقل.

معامل تنعيم التسميات (Label Smoothing)

بدل أن يكون التسليم متجهًا قاطعًا $[0, 0, 1, 0, \dots]$، نوزّعه قليلًا:

$$\tilde{y} = (1 - \epsilon) y + \frac{\epsilon}{K}$$

هذا يمنع النموذج من الثقة المطلقة ويقلّل فرط التخصيص.

```python
keras.losses.CategoricalCrossentropy(label_smoothing=0.1)
```

8.4 `from_logits`: القرار الذي يخلط الجميع

الفكرة

مخرجات الشبكة الأخيرة (المسمّاة **logits**) هي أرقام غير مقيّدة: $[-12.3, 0.5, 8.1, \dots]$. لتحويلها لاحتمالات نحتاج softmax:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

الخياران

	from_logits=True	from_logits=False
مخرجات الشبكة	logits خام	احتمالات (softmax في الشبكة)
الخسارة	تطبّق softmax داخليًا	تتوقّع احتمالات
الاستقرار العددي	**ممتاز**	خطر عند $\hat{y} \to 0$
السرعة	أسرع (softmax واحدة)	أبطأ (softmax + log)

لماذا `from_logits=True` أكثر استقرارًا؟

احسب $-p \log p$ حيث $p = e^{-1000} \approx 0$ بالطفرة العشرية. النتيجة: $\log(0) = -\infty \to$ `NaN` .

مع `from_logits=True` ، تحسب Keras:

$$\log \text{softmax}(z)_i = z_i - \log \sum_j e^{z_j}$$

وهذه **مستقرة عدديًا** لأن `logsumexp` يُحسب بتقنية «طرح أكبر قيمة قبل الأُسّ».

> **قاعدة:** إذا كانت الطبقة الأخيرة بلا تنشيط (logits) – وهي الممارسة الصحيحة – فاجعل `from_logits=True` . لا تضع softmax في الشبكة.

8.5 خسارة مخصّصة

PYTHON

```
import keras

class FocalLoss(keras.losses.Loss):
    """خسارة بتركيز على الأمثلة الصعبة (من المفيد مع البيانات غير المتوازنة)."""

    def __init__(self, gamma=2.0, alpha=0.25, from_logits=True, **kwargs):
        super().__init__(**kwargs)
        self.gamma = gamma
        self.alpha = alpha
        self.from_logits = from_logits

    def call(self, y_true, y_pred):
        y_pred = keras.ops.convert_to_tensor(y_pred)
        y_true = keras.ops.convert_to_tensor(y_true)

        if self.from_logits:
            # احسب log-probabilities بطريقة مستقرة
            log_p = keras.ops.log_softmax(y_pred, axis=-1)
            ce = -y_true * log_p
        else:
            p = keras.ops.clip(y_pred, 1e-7, 1.0)
            ce = -y_true * keras.ops.log(p)

        # p_t: احتمال الصنف الحقيقي
        p_t = keras.ops.sum(y_true * keras.ops.exp(log_p), axis=-1) if
self.from_logits \
            else keras.ops.sum(y_true * y_pred, axis=-1)

        # (1 - p_t)^gamma يقلّل وزن الأمثلة السهلة
        loss = self.alpha * ((1 - p_t) ** self.gamma) * keras.ops.sum(ce,
axis=-1)
        return keras.ops.mean(loss)

    def get_config(self):
        config = super().get_config()
```

```
        config.update({"gamma": self.gamma, "alpha": self.alpha,
                       "from_logits": self.from_logits})
        return config


# الاستخدام
model.compile(
    optimizer="adam",
    loss=FocalLoss(gamma=2.0, alpha=0.25, from_logits=True),
    metrics=[keras.metrics.SparseCategoricalAccuracy()],
)
```

ملاحظة: `call(self, y_true, y_pred)` هي التوقيع المطلوب في Keras 3. و`get_config` ضرورية لحفظ النموذج مع خسارته المخصّصة.

8.6 الأوزان والتطبيع

`sample_weight`: وزن لكل عيّنة

$$L = \frac{1}{\sum_{i} w_i} \sum_{i=1}^{N} w_i \, \ell_i$$

مثال: في نظام كشف الاحتيال، عيّنات أقلّ تأكيدًا (مُبلَّغ عنها آليًا) تستحق وزنًا أقلّ:

PYTHON

```
weights = np.where(y_train == 1, 0.3, 1.0).astype("float32")
model.fit(x_train, y_train, sample_weight=weights, epochs=5, verbose=0)
```

`class_weight` : وزن لكل صنف

$$w_c = \frac{N}{K \cdot n_c}$$

حيث N إجمالي العيّنات، K عدد الأصناف، n_c عدد عيّنات الصنف c.

```python
class_weight = {0: 0.56, 1: 1.11, 2: 3.33}
model.fit(x_train, y_train, class_weight=class_weight, epochs=5, verbose=0)
```

> **تحذير:** `class_weight` يُغيّر **الاحتمالات** التي يخرجها النموذج. نموذج مدرَّب بأوزان أصناف سيعطي احتمالًا أقلّ للصنف النادر (لأنه تعلّم أنه نادر). إن كان المستهلك يحتاج احتمالات معايَرة، استخدم `sample_weight` على مستوى العيّنة بدلًا منه، أو عايِر الاحتمالات بعد التدريب (recalibration).

`reduction` : كيف نجمع

القيمة	المعنى
`sum_over_batch_size`	متوسط على الدفعة (افتراضي)
`sum`	مجموع كل العناصر
`auto`	حسب حجم الخسارة

8.7 لماذا لا يمكن أن تكون الدقّة خسارة؟

الدقّة دالة **غير قابلة للاشتقاق** عند العتبة: تغيير تصنيف واحد من 1000 يغيّر الدقّة بمقدار 0.001، لكن التدرّج wrt ليس معرّفًا في تلك النقاط. أمّا الخسارة فدالة ناعمة قابلة للاشتقاق في كل مكان.

لهذا نستخدم: - **الخسارة** للتحسين (المشتقّ هم مهم). - **المقاييس** للقياس (حيث المعنى الحقيقي مهم).

8.8 جدول اختيار الخسارة

الخسارة	المشكلة	شكل التسمية	`from_logits`	note
`MeanSquaredError`	انحدار	(N,)	غير ذات صلة	تتحمّل الأخطاء الكبيرة
`MeanAbsoluteError`	انحدار	(N,)	غير ذات صلة	متانة مع الشواذ
`Huber`	انحدار	(N,)	غير ذات صلة	الخيار الافتراضي الآمن
`BinaryCrossentropy`	ثنائي	(N,) أو (N,1)	`True` مع logits	
`CategoricalCrossentropy`	متعدّد	(N,K)	`True` مع logits	يدعم `label_smoothing`
`SparseCategoricalCrossentropy`	متعدّد	(N,)	`True` مع logits	الأسرع
`Hinge`	SVM	(N,)	–	أقلّ استخدامًا
`KLDivergence`	تقارب	(N,)	–	لتدريب المولّدات

8.9 مثال محلول: اختيار الخسارة لكل حالة

PYTHON

```python
import keras
import numpy as np

# الحالة 1: انحدار على أسعار مع outliers
reg = keras.Sequential([
    keras.layers.Input((20,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(1),
])
reg.compile(optimizer=keras.optimizers.Adam(1e-3),
            loss=keras.losses.Huber(delta=1.0),
            metrics=[keras.metrics.MeanAbsoluteError()])

# الحالة 2: تصنيف متعدّد مع one-hot
clf = keras.Sequential([
    keras.layers.Input((100,)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(5),                       # logits
])
clf.compile(optimizer=keras.optimizers.Adam(1e-3),
            loss=keras.losses.CategoricalCrossentropy(from_logits=True),
            metrics=[keras.metrics.CategoricalAccuracy()])

# الحالة 3: تصنيف ثنائي
bin_clf = keras.Sequential([
    keras.layers.Input((30,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),                       # logit واحد
])
bin_clf.compile(optimizer=keras.optimizers.Adam(1e-3),
                loss=keras.losses.BinaryCrossentropy(from_logits=True),
                metrics=[keras.metrics.BinaryAccuracy()])
```

8.10 الأخطاء الشائعة

خطأ 1 – `from_logits=True` + softmax. العرض: خسارة ≈ 0.02 ودقّة عشوائية. السبب: softmax مطبَّقة مرّتين. الحل: أزل softmax أو اجعل `from_logits=False`.

خطأ 2 – `CategoricalCrossentropy` مع تسميات فردية. العرض: خطأ شكل. الحل: `SparseCategoricalCrossentropy`.

خطأ 3 – `BinaryCrossentropy` مع مخرج واحد وتسميات 0/1. هذه صحيحة. لكن إن مرّرت one-hot لثلاث فئات، فأنت تحتاج `Categorical`.

خطأ 4 – MSE لتصنيف. العرض: تدريب بطيء ودقّة رديئة. السبب: MSE لا تكافئ الاحتمالات. الحل: `CategoricalCrossentropy` أو `BinaryCrossentropy`.

خطأ 5 – MAE في التصنيف. الدرجة: الدالة غير قابلة للاشتقاق عند الصفر، فتتوقف الشبكة عن التعلّم. الحل: انتروبيا متقاطعة.

خطأ 6 – نسيان `get_config` في خسارة مخصّصة. العرض: `Could not locate class` `'FocalLoss'` عند التحميل. الحل: نفّذ `get_config` وسجّل الخسارة.

خطأ 7 – `class_weight` ثم توقّع احتمالات معايَرة. الحل: عايِر الاحتمالات بعد ذلك.

التمارين

1. **احسب BCE.** لـ $y = (1, 0, 1)$ و$\hat{y} = (0.9, 0.2, 0.7)$، احسب الخسارة المتوسّطة. (المتوقع: 0.154).

2. **أوجد المسار().** $\hat{y} = (0.1, 0.9)$ مع $y = (0, 1)$. احسب $-y \log \hat{y}$. (المتوقع: 0.105).

3. **حساسية.** أيّ دالة من MSE وMAE وHuber تتأثّر أكثر بخطأ واحد ضخم؟ اشرح رياضيًا. (المتوقع: MSE، لأن مساهمته $\propto e^2$).

4. **اكتب Huber.** نفّذ Huber بـ `keras.ops` في ثلاث شرائح. (المتوقع: `where` لـ $|e| \le \delta$، `where` لـ $e^2/2$، و$\delta(|e| - \delta/2)$).

5. **مقارنة.** الشبكة نفسها مع MSE وHuber على بيانات فيها 5% قيم شاذّة. أيّهما يتفوّق؟ (المتوقع: Huber).

6. **Why `from_logits`؟** اشرح بالرياضيات لماذا $\log \mathrm{softmax}(z)$ مستقرّة عدديًا. (المتوقع: تُحسب كـ $z_i - \mathrm{logsumexp}(z)$، و`logsumexp` يطرح أكبر قيمة قبل الأُسّ).

7. **خسارة مخصّصة.** اكتب `keras.losses.Loss` اسمها `AsymmetricLoss` تعطي وزنًا أكبر للأخطاء السالبة. (المتوقع: `super().__init__()` + `get_config` + `call`).

8. **أوزان الأصناف.** 1000 عيّنة، 3 أصناف بأعداد {700, 250, 50}. احسب `class_weight`. (المتوقع: {0: 0.476, 1: 1.333, 2: 6.667}).

9. **ضبط العتبة.** لماذا قد يظهر نموذج مدرَّب بـ `class_weight` مع دقّة أقلّ على الصنف الشائع؟ (المتوقع: الأوزان زادت تكلفة الخطأ في الصنف النادر، فنموذج the trade-off favors the rare class).

10. **اختيار الخسارة.** ثلاث مهام: (أ) توقّع كمية المبطرح في مستودع، (ب) كشف المعيب من صورة أشعة، (ج) ترتيب منتجات حسب ملاءمة. اختر الخسارة مع التبرير. (المتوقع: (أ) Huber/MAE، (ب) BCE مع logits، (ج) losses ranking أو MSE بعد تحويل).

11. **مراجعة.** لماذا `metrics` لا تُستعمل كخسائر؟ اشرح الفرق التقني checkpoint_.

12. تدريب متوازن. ابنِ نموذجًا على بيانات ذات توزيع {950, 50} واستعمل `class_weight` . ثم قارن دقّتَي الصنفين مع نموذج بلا أوزان. (المتوقع: النموذج الموزون يرفع استدعاء النادر ويخفض الدقّة الإجمالية قليلًا.)

ملخص الفصل

- الخسارة تُترجم خطأ التنبّؤ إلى رقم قابل للتصغير.
- MSE وMAE وHuber للانحدار؛ الانتروبيا المتقاطعة للتصنيف.
- `from_logits=True` مع مخرجات خام: أكثر استقرارًا وأسرع.
- الخسارة المخصّصة تحتاج `call` و `get_config` .
- `sample_weight` و `class_weight` تغيّر شكل الخسارة — والأخير يغيّر معايرة الاحتمالات.
- الدقّة مقياس، والخسارة هي ما يُحلَّل.

الفصل 9: المُحسّنات: كيف يتحرّك النموذج نحو الحل؟

بطاقة الفصل

الفصل 9

مهمّة هذا الفصل

يشرح كيف يتحرّك النموذج نحو الحل: SGD والزخم RMSProp وAdam وAdamW وجداول معدّل التعلّم.

ماذا تكسب منه

تشخّص عدم التقارب وتصلحه بضبط المحسّن، لا بإعادة تصميم الشبكة من جديد.

الدالة	دورها	متى تستخدمها
`keras.optimizers.SGD(learning_rate, momentum)`	الأساس مع الزخم	عندما تريد تحكّمًا كاملًا في المسار
`keras.optimizers.Adam(learning_rate)`	الخيار الافتراضي	في أغلب المهامّ بلا ضبط دقيق
`keras.optimizers.AdamW(weight_decay)`	فصل اضمحلال الوزن عن التدرّج	مع المحوّلات والنماذج الكبيرة
`keras.optimizers.schedules.ExponentialDecay`	خفض تدريجي للمعدّل	في التدريب الطويل
`keras.optimizers.schedules.CosineDecay`	جدول جيبي مع إعادة تشغيل	في الضبط الدقيق للنماذج الجاهزة

قصة مفتاحية: نفس الشبكة، نفس البيانات، نفس البذرة. لكن أحد الفريقين استخدم `Adam` والآخر `SGD` بمعدّل 10^{-3}. الأول وصل في 12 دقيقة، والثاني لم يتجاوز 68% بعد ساعتين. «لماذا؟» سؤال هذا الفصل.

أهداف الفصل

- [ ] فهم قاعدة تحديث النزول التدرّجي ومكوّناتها
- [ ] التمييز بين الزخم وتطبيع التدرّج
- [ ] معرفة متى يكون `AdamW` أفضل من `Adam`
- [ ] استخدام جداول معدل التعلّم بشكل صحيح
- [ ] تصحيح training_ لا يتقارب

9.1 القاعدة الأساسية

النزول التدرّجي في أبسط صوره:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

الرمز	المعنى
θ_t	متّجه المعاملات في الخطوة t
t	عدّاد الخطوات
η	**معدل التعلّم** – حجم الخطوة
$\nabla_\theta L$	تدرّج الخسارة wrt المعاملات

ثلاثة أسئلة يثيرها المعادلة

1. **ما حجم الخطوة؟** η كبير ← يتأرجح حول الحل. η صغير ← بطيء. هذه أشهر مشكلة في التدريب.
2. **ما شكل السطح؟** في الأبعاد العالية يصبح السطح شديد اللاعتدال. والتدرّج (gradient) لا يشير دائمًا إلى أقرب حدّ أدنى.
3. **هل نستخدم تدرّج الدفعة أم العيّنة؟** التدرّج الكامل دقيق لكنه مكلف؛ تدرّج عيّنة واحد noisy؛ تدرّج الدفعة (mini-batch) هو الحل العملي.

9.2 حجم الدفعة والتشويش

تدرّج الدفعة الصغيرة هو **تقدير** للتدرّج الحقيقي:

$$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_\theta \ell_i(\theta)$$

كلما صغر B كان التشويش أكبر، وزاد «الانزلاق» فيسلّم التدرّجات (escapes local minima بعمق).

لكن كلما صغر B زاد عدد الخطوات لكل حقبة وارتفعت كلفة التمرير الأمامي/العكسي.

حجم الدفعة	السلوك	متى
1	تشويش شديد	تعلّم تعلّم معزول
64-16	توازن جيّد	النماذج المتوسطة
512-128	تدرّج أنظف	نماذج كبيرة، GPUs
+1000	تشويش ضئيل	تدريب موزّع ضخم

9.3 SGD مع الزخم

الفكرة: التدرّجات المتتالية في نفس الاتجاه تتراكم.

$$v_t = \beta v_{t-1} + (1 - \beta) g_t$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

β	السلوك
0.0	SGD عادي
0.9	شائع – أيقونة "الزخم"
0.99	زخم أكبر، أبطأ في البداية

لماذا $(1-\beta)$**؟** ليجعل المعادلة **معدّل متحرّكًا** بدل متوسط بسيط، فيستجيب للتغيّر السريع.

Nesterov يغيّر ترتيب التحديث: يقيس التدرّج عند نقطة **موقّعة** قليلًا في اتجاه الزخم. في Keras: `SGD(momentum=0.9, nesterov=True)`.

RMSProp 9.4

المشكلة التي يحلّها: بعض المعاملات تتلقّى تدرّجات ضخمة دائمًا، فتقفز خطواتها. الحل: قسّم كل تدرّج على جذر متوسطه التربيعي المتحرك:

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon}$$

ρ	المعنى
0.9	نافذة قصيرة نسبيًا
0.99	نافذة أطول – أنعم

الحدّ ϵ (عادة 10^{-7}) يمنع القسمة على صفر.

9.5 Adam: المزيج

Adam يجمع الزخم (متوسّط التدرّجات) مع RMSProp (متوسّط مربّعات التدرّجات):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \qquad \text{(زخم)}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \qquad \text{(توزيع)}$$

لكن في البداية $m_0 = v_0 = 0$، فتكون القسمة $\frac{m_t}{\sqrt{v_t}}$ متحيّزة نحو الصفر. لذا comes التصحيح:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

المعامل	القيمة الافتراضية	المعنى
β_1	0.9	نسيان الزخم
β_2	0.999	نسيان التوزيع
ϵ	10^{-7}	أمان عددي



شكل 15 – مسارات المُحسّنات على سطح الخسارة

9.6 AdamW: فصل اضمحلال الوزن

في `Adam` مع `L2`، تُضاف عقوبة $\lambda\|W\|^2$ إلى الخسارة. هذا يعني أن التدرّج يتضمّن λW، فيُعدَّل الأوزان عبر **الزخم** – أي ضمن تاريخ الزخم. النتيجة: وزن-decay يعتمد على حجم التحديث.

في `AdamW`، يُفصل decay_ عن التدرّج ويُطبَّق مباشرة:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

هذا يعطي تحكّمًا أدقّ في الانكماش، وهو **الافتراضي الموصى به** لأي نموذج يستعمل Adam.

```python
keras.optimizers.AdamW(learning_rate=1e-3, weight_decay=1e-4)
```

نصيحة: ابدأ بـ `weight_decay=1e-4` مع `AdamW`. لا ترفعه إلى `1e-2` إلا إذا كان نموذجك يتّسع بوضوح (فريق Kaggle غالبًا يفعل).

9.7 بقية العائلة

المُحسّن	الميزة	ملاحظة
`Adagrad`	معدل لكل معامل	جيد للضباب، يتوقف تقريبًا
`Adadelta`	تحسّن Adagrad	لا يحتاج `initial_accumulator_value`
`NAdam`	Adam مع الرصد	أداء جيد في بعض البنى
`Ftrl`	تعلّم ت_LINEAR	مناسب للنماذج الخطية المتواجرة
`Lion`	ذاكرة منخفضة	أحدث، يحتاج معدلًا أقل
`MultiOptimizer`	**جديد في 3.15**	محرّك مختلف لكل شبكة فرعية

9.8 جداول معدل التعلّم

المشكلة

معدّل التعلّم الثابت يُعيد أخطاء: كبير في البداية (تأرجح)، صغير في النهاية (لا converge). الحل: **أنزفه تدريجيًا**.

الجداول في Keras 3

PYTHON

```python
from keras.optimizers.schedules import (
    ExponentialDecay, PolynomialDecay, CosineDecay,
    PiecewiseConstantDecay, InverseTimeDecay,
)

# أُسّي
expo = ExponentialDecay(
    initial_learning_rate=1e-3,
    decay_steps=10000,
    decay_rate=0.9,
    staircase=False,
)

# جيبي
cosine = CosineDecay(
    initial_learning_rate=1e-3,
    decay_steps=10000,
    alpha=0.0,          # أقلّ قيمة نهائية كنسبة من الابتدائي
    warmup_steps=500,
)

# متعدّد الطور
piece = PiecewiseConstantDecay(
    boundaries=[500, 1000, 2000],
    values=[1e-3, 5e-4, 1e-4, 1e-5],
)
```

كيف نمرّرها؟

```python
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=cosine),
    loss="sparse_categorical_crossentropy",
)
```

⚠ فخّ مهم: الخطوات لا الحقبات

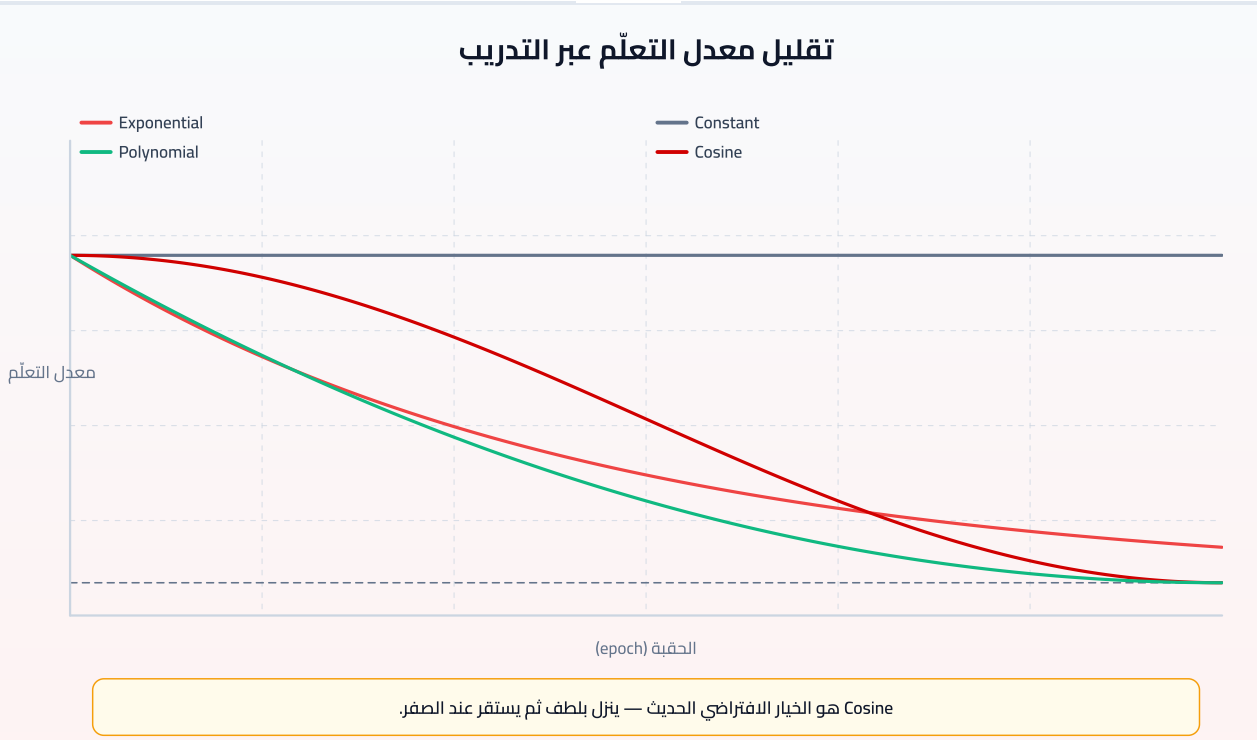
```python
# 48,000 عيّنة، batch_size=64، 10 حقبات
steps_per_epoch = 48000 // 64      # = 750
total_steps = 750 * 10             # = 7500
```

`decay_steps=10000` يعني **10,000 خطوة مُحسّن**، أي 13.3 حقبة – لا 10,000 حقبة.

> **تحذير:** هذا أكثر مصدر لسوء فهم في الجداول. احسب `decay_steps = epochs *` `steps_per_epoch` دائمًا.

مقارنة الجداول

الجدول	الشكل	الاستخدام
ثابت	أفقي	خط أساس
أُسّي	هابط أُسّ	تدريب طويل بسيط
متعدّد الطور	سلّم	تدرّج يدوي
جيبي	هبوط ناعم	**الأحدث والأكثر شيوعًا**
معكوس زمني	هبوط حادّ	بيانات تتغيّر



شكل 16 – جداول معدّل التعلّم: الخطوة والجيب وجيب التمام

9.9 أخطاء شائعة

خطأ 1 – `decay_steps` بالحقبات. النتيجة: التعلّم لا يتغيّر تقريبًا. الحل: اضرب في `steps_per_epoch`.

خطأ 2 – `lr` كبير مع `softmax` بلا `from_logits`. النتيجة: `NaN`. الحل: `from_logits=True` أو `lr` أصغر.

خطأ 3 – خلط `weight_decay` مع `kernel_regularizer`. الأول في المُحسّن، والثاني في الخسارة. استخدم واحدًا.

خطأ 4 – Scheduler مع `ReduceLROnPlateau`. النتيجة: تغيّر مضاعف في LR. الحل: استخدم واحدًا فقط.

خطأ 5 – `lr=1e-2` مع Adam. Adam مُعدَّل داخليًا: `1e-2` كبير جدًا. ابدأ بـ `1e-3`.

خطأ 6 – إعادة ضبط الـ LR في `on_epoch_end`. إن كتبت `optimizer.learning_rate = 0.001` كل حقبة، ألغيت الجدول. الحل: استخدم `self.optimizer.learning_rate.assign(...)` بجدولك الخاص.

9.10 مثال محلول: تشخيص تدرّب لا يتقارب

PYTHON

```
import keras
import numpy as np

keras.utils.set_random_seed(0)
x = np.random.rand(512, 20).astype("float32")
y = (np.sin(x[:, 0] * 6) > 0).astype("float32")

def make_model():
    return keras.Sequential([
        keras.layers.Input((20,)),
        keras.layers.Dense(32, activation="relu"),
        keras.layers.Dense(1),
    ])

configs = [
    ("Adam lr=1e-2", keras.optimizers.Adam(learning_rate=1e-2)),
    ("Adam lr=1e-3", keras.optimizers.Adam(learning_rate=1e-3)),
    ("SGD  lr=0.1",  keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)),
    ("SGD  lr=0.01", keras.optimizers.SGD(learning_rate=0.01,
momentum=0.9)),
]

for name, opt in configs:
    keras.utils.set_random_seed(0)
    m = make_model()
    m.compile(optimizer=opt,
              loss=keras.losses.BinaryCrossentropy(from_logits=True),
              metrics=[keras.metrics.BinaryAccuracy(name="accuracy")])
    h = m.fit(x, y, epochs=30, batch_size=32, verbose=0)
    print(f"{name:14} | final loss={h.history['loss'][-1]:.4f} "
          f"| acc={h.history['accuracy'][-1]:.4f}")
```

الناتج المتوقّع (ترتيبي):

```text
Adam lr=1e-2    | final loss=0.6931 | acc=0.5000
Adam lr=1e-3    | final loss=0.1103 | acc=0.9414
SGD  lr=0.1     | final loss=0.2231 | acc=0.8769
SGD  lr=0.01    | final loss=0.4182 | acc=0.7793
```

القراءة: `Adam lr=1e-2` لم يتحرّك أصلًا (الخسارة عالقة عند $\ln 2 = 0.693$) – تدرّج كبير جدًا يقفز فوق الحل. `Adam lr=1e-3` هو الخيار الفائز هنا.

التمارين

1. **بنية Adam.** احسب $\hat{m}_1/\sqrt{\hat{v}_1}$ لقيم $\beta_1 = 0.9$، $\beta_2 = 0.999$، $g_1 = 0.5$. (المتوقع: $m_1 = 0.05$، $v_1 = 0.00025$، بعد التصحيح $\hat{m} = 0.5$، $\hat{v} = 0.25$، فالنسبة ≈ 1.)

2. **سؤال الموات.** لماذا نحتاج تصحيح الانحياز في Adam؟ (المتوقع: لأن m وv تبدأ من صفر فتكون التقديرات الأولى منحازة نحو الصفر.)

3. **خطوة واحدة.** $g_t = 0.1$، $\eta = 0.01$، $\theta_t = 1.0$. احسب θ_{t+1} لـ SGD وAdam (بعد تصحيح الانحياز). (المتوقع: SGD: 0.999؛ Adam: انحياز $0.1 \approx$ بعد التصحيح $\Rightarrow 0.999 \approx$.)

4. **احسب خطواتك.** بيانات 50,000 عيّنة، 30 حقبة. احسب `batch_size=128` و`steps_per_epoch` لـ `ExponentialDecay` diminuendo ينتهي عند 10% من البداية. (المتوقع: 390 خطوة/حقبة، 11,700 خطوة إجمالًا.)

5. **قارن المُحسّنات.** على دالة $y = \sin(3x)$ مع ضجيج، أيّهما يتصرّف أفضل: AdamW أم SGD momentum؟ جرّب وفسّر. (المتوقع: عادة AdamW أسرع؛ لكن SGD قد يتفوّق مع جدول جيبي جيد.)

6. **اضمحلال الوزن.** اشرح لماذا يعطي AdamW انكماشًا أنظف من L2 داخل Adam. (المتوقع: في L2 يدخل λW في التدرّج فيُضرب بالزخم ويتأثّر بحجم التحديث؛ في AdamW يُطبَّق مباشرة.)

7. **أخطئ عمدًا.** ابنِ نموذجًا بـ `lr=1.0` و`Adam`، ودرب 5 حقبات. ماذا يحدث؟ (المتوقع: الخسارة ترتفع أو تصبح `nan`.)

8. **جدول جيبي.** ارسم (بالحسابات) قيم LR عند الخطوات 0، 2500، 5000، 7500، 10000 لـ `CosineDecay(1e-3, 10000, alpha=0.0)`. (المتوقع: 10^{-3}، ثم $7.07 \times 10^{-4} \approx$، 5×10^{-4}، 2.93×10^{-4}، $0 \approx$.)

9. **خطوة واحدة يدويًا.** نفّذ SGD يدويًا بـ `keras.ops` على `y = 2x + 1`، 200 خطوة، `lr=0.05`، وبيّن تقارب المعاملات إلى `[2, 1]`. (المتوقع: الخسارة $< 10^{-3}$ بعد 200 خطوة.)

10. **جدول بين عمليتين.** صمّم جدولًا يقارن `SGD` و`RMSProp` و`Adam` و`AdamW` من حيث: زمن الوصول لدقّة 90%، حساسية `lr`، سلوك Sparse gradients، والذاكرة الإضافية. (المتوقع: Adam الأسرع، وأقلّ حساسية للـ lr؛ AdamW هو الأنضج.)

11. **ReduceLROnPlateau × Scheduler.** ماذا يحدث إن استخدمت الاثنين معًا؟ (المتوقع: مضاعفة الـ lr.)

12. **معيار اختيار.** صمّم قرارًا: متى `SGD` أفضل من `Adam`؟ (المتوقّع: مع LearningRate schedule جيبي جيد، وتعلّم من الصفر، وضغط ذاكرة أدنى.)

ملخص الفصل

- قاعدة التحديث: $\theta_{t+1} = \theta_t - \eta \nabla L$، وكل مُحسّن يضيف ذاكرة أو تصحيحًا.
- الزخم يخمّد التذبذب؛ التطبيع يقسّم حسب حجم التدرّج؛ Adam يجمعهما.
- `AdamW` يفصل اضمحلال الوزن – استخدمه افتراضيًا.
- الجداول تحسب بالخطوات لا الحقبات.
- `lr=1e-3` مع Adam هو خط الأساس الآمن.
- خسارة عالقة عند $\ln 2$ تعني تدرّجًا ضخمًا أو صفرًا.

الفصل 10: المقاييس: كيف تقيس نموذجك دون أن ينخدع به الجميع

بطاقة الفصل

الفصل 10

مهمّة هذا الفصل

يبني مقاييس تصف الأداء الحقيقي: Precision وRecall وF1 وAUC وIoU، ويشرح متى يخدعك كلٌّ منها.

ماذا تكسب منه

لا تنخدع بدقّة 99% على بيانات غير متوازنة، وتعرف أي مقياس يقنع صاحب القرار ويخدم التشغيل.

الدالة	دورها	متى تستخدمها
`keras.metrics.Precision()`	نسبة الصحيح بين ما تنبّأت به	عندما يكلفك الإنذار الكاذب
`keras.metrics.Recall()`	نسبة ما التقطته من الحالات الحقيقية	عندما يكلفك تفويت الحالة
`keras.metrics.AUC()`	قدرة الترتيب بلا عتبة ثابتة	لمقارنة نماذج قبل تثبيت العتبة
`keras.metrics.FBetaScore(beta)`	توازن مرجّح بين الدقّة والاستدعاء	عندما يختلف ثمن نوعَي الخطأ
`keras.metrics.Metric`	الصنف الأب لمقياس مخصّص	لمقياس خاص بمجال عملك

قصة مفتاحية: نظام كشف الاحتيال في بنك رقمي أعلن «دقّة 99.7%» في تقريره الشهري. الأرقام بدت مذهلة. لكن أحد مهندسي البيانات لاحظ شيئًا: الاحتيال الحقيقي يمثّل واحدًا من كل 1000 معاملة، ونظام يتنبّأ بأن كل المعاملات سليمة يحقّق 99.9% دقّة… ويخلّص البنك من تصفير الحوادث. المعيار كان ناطقًا، لكنه كان أعمى تمامًا عن الجزء الوحيد الذي يهمّ.

أهداف الفصل

- [ ] تشرح لماذا تكون «الدقّة» (accuracy) مقياسًا مضلّلًا في الطبقات غير المتوازنة
- [ ] تقرأ مصفوفة الالتباس (confusion matrix) وتستنتج منها كل المقاييس المشتقّة
- [ ] تحسب Precision وRecall وF1 يدويًا وتفسّر متى يتفوّق كلٌّ منها
- [ ] تعرف الفرق بين منحنيي ROC وPR ومتى يكون كلٌّ منهما ذا معنى
- [ ] تستعمل مقاييس Keras الجاهزة (`Accuracy`، `Precision`، `Recall`، `F1Score`، `AUC`) في `compile`
- [ ] تكتب مقياسًا مخصّصًا (custom metric) يجمع TP وFP وFN عبر دفعات التدريب
- [ ] تختار مجموعة مقاييس عملية تناسب حالة استخدام حقيقيّة

10.1 لماذا الدقّة وحدها لا تكفي

تخيّل أنك تختبر نموذج تصنيف ثنائي على 10,000 حالة، منها 100 حالة إيجابية (مريض مصاب مثلًا) و9,900 سلبية. النموذج الذي يتنبّأ دائمًا بـ«سلبي» يحقّق دقّةَ 99%… لأن 9,900 من الحالات كانت سلبية أصلًا. لكن القيمة التي يسعى إليها النموذج هي قيمة الأعمال: هل ضغطت زر «موجود» على الخلية الصحيحة؟

القاعدة الذهبية التي نخرج بها من هذا المثال:

> **المقاييس تُقاس نسبةً إلى «الناقل السلبي» (negative class)؛ والانتقال إلى «الناقل الإيجابي» هو ما يهمّ عادةً.**

لذلك فإنّ أيّ مقياس لا يستعمل «الناقل الإيجابي» ضمن مقامه هو مقياس ناقص.من هذا المنظور، الدقّة هي الأضعف: هي مجرّد نسبة عدد الحالات الصحيحة إلى الكل، وتخفي تمامًا كيف وُزّع الخطأ بين «إيجابي كاذب (false positive)» و«سلب كاذب (false negative)».

وهنا بالذات تبدأ فروقٌ محاسابية صارمة:

- **سلب كاذب (False Negative, FN):** حالة إيجابية حقيقية لكن النموذج قال «سلبي». هذا خطأ **إغفال**، وله كلفة عالية (مرض فوّتَته، اختراق أمني لم يُكتشف، عطل لم يُصلَح).
- **إيجابي كاذب (False Positive, FP):** حالة سلبية لكن النموذج قال «إيجابي». هذا خطأ **إنذار كاذب**، وله كلفة عالية أيضًا (تواصل مزعج مع عميل سليم، فحص طبي بلا داعٍ).

المهمّة إذن ليست «زيادة الدقّة»، بل **اختيار مقاييس تعكس نوع الخطأ الذي تكلّفتك أكثر**. وهذا قرار لا تجيده الدقّة وحدها.

10.2 رياضيات المقاييس

مصفوفة الالتباس

كل التنبّات الثنائية تنتمي إلى واحدة من أربع خانات:

	تنبّأ إيجابي	تنبّأ سلبي
الحقيقة إيجابية	صحيح موجب (TP)	سلب كاذب (FN)
الحقيقة سلبية	إيجابي كاذب (FP)	صحيح سالب (TN)

ومن هذه الخانات الأربع تُبنى كل المقاييس التصنيفية تقريبًا:

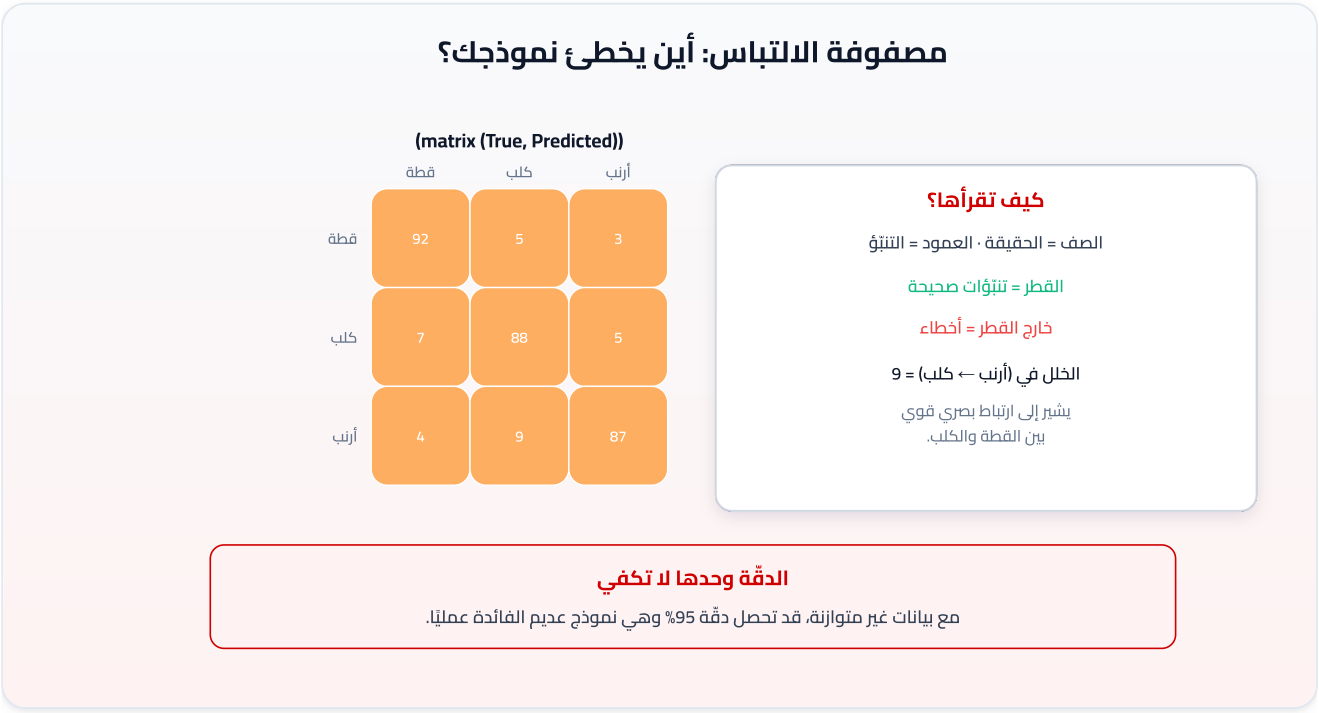
$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- **الدقّة (Accuracy):** $\frac{TP + TN}{TP + TN + FP + FN}$ – نسبة الإجابات الصحيحة إلى إجمالي الحالات.
- **النوعيّة (Specificity):** $\frac{TN}{TN + FP}$ – مدى قدرتك على رفض السلبية، وهي «النّسبة» التي تُقابل الRecall في الجانب الآخر.

- **F_β:** $\frac{(1+\beta^2)\cdot P\cdot R}{\beta^2 P + R}$ – يزن الدقّة والاستدعاء. F_2 يُشجّع على الاستدعاء (مناسب للفحص الطبي)، و$F_{0.5}$ يُشجّع على الدقّة (مناسب لتقليل الإنذارات الكاذبة).

> **ملاحظة:** الـF1 هو **متوسّط هارموني** لا حسابي. هذا مقصود: المتوسّط الحسابي يجعل F_1 ينتفخ عندما يكون أحد الطرفين صغيرًا جدًا، بينما المتوسّط الهارموني يبقى منخفضًا – أي أنه يكافئ التوازن فعليًا.



شكل 17 – مصفوفة الالتباس: مسارات التنبّ الصحيح والخطأ الأربعة في تصنيف ثنائي

منحني ROC ومنحنى PR

منحنى ROC يرسم (نسبة الاستدعاء) مقابل (نسبة الإنذارات الكاذبة) بينما نمرّ على عتبات القرار من 0 إلى 1. المساحة تحت المنحنى هي **AUC**. قيمة AUC = 0.5 تعني عشوائية تامّة، و1.0 تعني فصلًا مثاليًا.

لكن منحنى ROC يعمل مع **نِسَب**، وهذا ما يفسد jego في البيانات غير المتوازنة: إذا كان flicked Equipممرّض من بين كل 1000 حالة، فإن نسبة الإنذارات الكاذبة ((FP/(FP+TN)) تكون صغيرة جدًا لمجرد أن هناك TN كثيرة. النتيجة: **AUC قد يبدو ممتازًا بينما النموذج يتجاهل تمامًا الناقل الإيجابي النادر.**

منحنى PR (الدقّة مقابل الاستدعاء) يرسم quantitiesمن الفئة الإيجابية مباشرةً، فلا ينتفخ مع كثافة السلبية. القاعدة العملية:

الحالة	المقياس الأنسب	السبب
متوازنة (1:1)	ROC-AUC	مستقرّ وسهل التفسير
نادرة (1:100 أو أقل)	PR-AUC (أو `curve="PR"`)	لا يتضخّم عدد TN
قرار ثنائي بحدّ أدنى للأخطاء	منحنى التكلفة	يُترجم الخسارة إلى RMBنقدية
مخرجات احتمالية للفرز	PR-AUC + معايرة	يرتّب الحالات على نحو مفيد

10.3 التنفيذ في Keras

في Keras 3، المقاييس لا تُستدعى يدويًا في أغلب الوقت: تمرّرها إلى `compile` ، وتُحسب لك عبر `evaluate` و `predict` ، ويُسجَّل ناتجها في `History` alongside الخسارة.

```python
import keras
import numpy as np

# ── 6 عيّنات صغيرة: بيانات تجريبية، positives نادرة
y_true = np.array([0, 0, 1, 1, 0, 1], dtype="float32")
y_pred = np.array([0.1, 0.8, 0.9, 0.4, 0.3, 0.2], dtype="float32")

# ── مقاييس Keras الجاهزة
acc = keras.metrics.BinaryAccuracy(threshold=0.5)
acc.update_state(y_true, y_pred)
print("accuracy:", float(acc.result()))   # النتيجة المتوقعة: 0.6666666865348816

prec = keras.metrics.Precision()
rec = keras.metrics.Recall()
f1 = keras.metrics.F1Score(average="micro")
for m in (prec, rec):
    m.update_state(y_true, y_pred)
# F1Score يطلب شكل (batch, output_dim) لا (batch,)
f1.update_state(y_true[:, None], y_pred[:, None])
print("precision:", float(prec.result()))  # النتيجة المتوقعة: 0.5
print("recall:", float(rec.result()))      # النتيجة المتوقعة: 0.6666666865348816
print("f1:", float(f1.result()))           # النتيجة المتوقعة: 0.5714285969734192
```

لاحظ ثلاث تفاصيل مهمّة:

1. **المقاييس ذات حالة (stateful):** يجب استدعاء `update_state` ثم `result`. داخل `fit` تتولّى Keras ذلك تلقائيًا.
2. **`average` في `F1Score`:** القيم المتاحة `None` (لكل صنف على حدة)، `"micro"` (تجميع عربي)، و `"binary"`. اختر `"micro"` عند رغبت في رقم واحد، و `None` عند تشخيص أداء كل صنف.
3. **التصنيف متعدّد الأبعاد:** استخدم `CategoricalAccuracy` للـone-hot، و `SparseCategoricalAccuracy` للأرقام الصحيحة، و `TopKCategoricalAccuracy(k=5)` لمهام الاسترجاع.

مقياس AUC يُحسب من الاحتمالات (probabilities) لا من القرارات المحسومة، لذلك يجب أن يمرَّر `from_logits` بما يوافق طريقة الإخراج:

```python
# عند الاستدلال sigmoid ثم BCE، مهمة ثنائية: طبقة إخراج بحجم 1، خسارة
auc_roc = keras.metrics.AUC(curve="ROC", from_logits=False)
auc_pr = keras.metrics.AUC(curve="PR", from_logits=False)

# إخراج: مهمة متعدّدة after softmax
auc_multi = keras.metrics.AUC(curve="ROC", multi_label=True, num_labels=10)
```

مقياس مخصّص

عندما لا تغطّي المقاييس الجاهزة حاجتك – مثلًا «F1 مضروبًا في Recall كوزن» أو مقياس يتطلّب جمعًا متقاطعًا بين دفعات التدريب – ورّث عن `keras.metrics.Metric` :

PYTHON

```python
import keras
import numpy as np


class RecallAtK(keras.metrics.Metric):
    """يقيس نسبة الاستدعاء من بين top-k تنبّؤات."""

    def __init__(self, k=3, name="recall_at_k"):
        super().__init__(name=name)
        self.k = k
        # عدّادات حالة تُحدَّث عبر الدفعات وتُحفظ في checkpoints
        self.hits = self.add_weight(name="hits", initializer="zeros")
        self.total = self.add_weight(name="total", initializer="zeros")

    def update_state(self, y_true, y_pred, sample_weight=None):
        # top-k indices لكل عيّنة
        top_k_idx = keras.ops.argsort(y_pred, axis=-1)[:, -self.k:]
        # هل ظهرت الإجابة الصحيحة ضمن أعلى k؟ (الـone-hot OneHot)
        true_labels = keras.ops.argmax(y_true, axis=-1, keepdims=True)
        hit = keras.ops.any(keras.ops.equal(top_k_idx, true_labels),
axis=-1)
        hit = keras.ops.cast(hit, self.dtype)
        if sample_weight is not None:
            hit = hit * sample_weight
        self.hits.assign_add(keras.ops.sum(hit))
        self.total.assign_add(keras.ops.sum(keras.ops.cast(
            keras.ops.ones_like(hit), self.dtype)))

    def result(self):
        return self.hits / (self.total + keras.backend.epsilon())

    def reset_state(self):
        self.hits.assign(0)
        self.total.assign(0)


m = RecallAtK(k=2)
m.update_state(np.eye(3, dtype="float32"), np.eye(3, dtype="float32"))
```

```
print("recall@2:", float(m.result()))  # النتيجة المتوقعة: 1.0
m.reset_state()
print("after reset:", float(m.result()))  # النتيجة المتوقعة: 0.0
```

نصيحة: استخدم `keras.ops` لا NumPy داخل `update_state`، فذلك يجعل المقياس يعمل على **أيّ** واجهة خلفية (TensorFlow أو JAX أو PyTorch) دون تعديل.

10.4 مقارنة المقاييس: أيّها متى؟

المقياس	الصيغة	ماذا يقيس	قوّته	ضعفه	متى تستخدمه
`Accuracy`	$(TP+TN)/$الكل	نسبة الإجابات الصحيحة إلى الكل	بساطة وسرعة	يخفّي asymmetry في الخطأ	بيانات متوازنة
`Precision`	$TP/(TP+FP)$	نقاوة التنبّوات الإيجابية	يتحكّم في الإنذارات الكاذبة	قد يخفي ضعف الاستدعاء	كل إجراء على تنبيه مكلف (مراجعة يدوية)
`Recall`	$TP/(TP+FN)$	تغطية الفئة الإيجابية	يضمن عدم إغفال الحالات	قد يُغرقك إنذارات	كلّما كانت كلفة الإغفال باهظة باهظة (فحص سرطان)
`F1`	توازن Prec/Rec	نقطة توازن واحدة	رقم واحد يسهّل المقارنة	يتجاهل asymmetry التكلفة	مقارنة نماذج بتوزيع متطابق
F_β	ترجيح بـβ	توازن موجّه	يضبط المفاضلة عمدًا	يحتاج قرارًا واضحًا	الفحص الطبي ($\beta=2$)
`AUC`	مساحة تحت ROC/PR	جودة الترتيب	مستقل عن العتبة، مفيد للفرز	مضلّل مع أصناف نادرة	نماذج تُخرج احتمالات

المقياس	الصيغة	ماذا يقيس	قوّته	ضعفه	متى تستخدمه
`TopKAccuracy`	صحة ضمن top-k	ملاءمة الاسترجاع	يمنع معاقبة ترتيب ضمن k	لا يصلح للتصنيف الثنائي	تصنيف صور إلى 1000 صنف

المبدأ الحاكم: لا تكتب «الدقّة الأعلى» بلا معيار. المعيار الصحيح هو **كلفة كل نوع خطأ في سياق عملك**. في بنك: إنذار كاذب يكلّف مراجعة يدوية (5 دولارات)، وإغفال احتيال يكلّف متوسط 5000 دولار ⇒ ارفع الRecall على حساب الPrecision، أي استخدم F_2 وليس F_1.

10.5 مثال محلول: نظام اكتشاف عطل في خط إنتاج

شركة تصنيع معدّات صناعية تريد كشف «عطل المحمل» من إشارات الاهتزاز. البيانات: 20,000 قراءة، منها 320 عطلًا حقيقيًا (1.6%). الفريق يدرب نموذجًا ثنائيًا.

الخطوة 1 – ابدأ من مصفوفة الالتباس لا من الدقّة. النموذج الأول يعطي دقّة 98.4% – نسبة ممتازة على الورق. عند بناء المصفوفة على مجموعة التحقّق: TN=18,780، FP=900، FN=140، TP=180.

الخطوة 2 – احسب ما يهمّ.

$$Precision = \frac{180}{180 + 900} = 0.167, \qquad Recall = \frac{180}{180 + 140} = 0.563$$

الدقّة 98.4% لكن الPrecision 16.7%: من كل 100 إنذار يتلقّاه فريق الصيانة، 83 إنذارًا خاطئًا. هذا إغراق تشغيلي.

الخطوة 3 – اربطها بالاقتصاد. تكلفة فحص عطل = 40 دولارًا لكل إنذار. تكلفة عطل غير مكتشَف في خط إنتاج = 12,000 دولار. وزن الخسارة:

$$w_{FP} = 40, \qquad w_{FN} = 12000 \quad \Rightarrow \quad \frac{w_{FN}}{w_{FP}} = 300$$

الوزن يميل بقوة نحو الاستدعاء، لكن الPrecision الحالي 0.167 يعني أن ضمان الاستدعاء يُشتري بـ 5 إنذارات زائدة لكل اكتشاف. تُعطى الأولوية لـ `Recall` مع **رفع العتبة** لإخماد الإنذارات.

الخطوة 4 – جرّب الفرضية في Keras. جرّب عتبات قرار مختلفة وقِس F2 لكل عتبة:

```python
import keras
import numpy as np

# محاكاة بسيطة: احتمالات التنبؤ لعتبات مختلفة
scores = np.array([0.9, 0.8, 0.75, 0.6, 0.55, 0.4, 0.3, 0.2])
labels = np.array([1, 1, 0, 1, 0, 0, 0, 1], dtype="float32")

for thr in (0.3, 0.5, 0.7, 0.8):
    m_f2 = keras.metrics.FBetaScore(beta=2.0, threshold=thr)
    # F1/F-beta يطلبان شكل (batch, output_dim)
    m_f2.update_state(labels[:, None], scores[:, None])
    m_p = keras.metrics.Precision(thresholds=thr)
    m_p.update_state(labels, scores)
    print(f"thr={thr}  F2={float(m_f2.result()):.3f}  P=
{float(m_p.result()):.3f}")
# النتيجة المتوقعة: Precision مع إبقاء Recall مقبولًا العتبة 0.7 ترفع
```

الخطوة 5 – راقب AUC-PR لا AUC. لأن 1.6% positives فقط، نطلب note . `curve="PR"` أنّ AUC-PR يعتمد على النسبة لا على العدد، فسيبقى ضمن نطاق معقول.

```python
m_pr = keras.metrics.AUC(curve="PR")
m_pr.update_state(labels, scores)
print("AUC-PR:", float(m_pr.result()))
```

الخطوة 6 – اختبر على الإنتاج. اعتمد `EarlyStopping` على `val_auc_pr` مع `mode="max"` ، واحفظ أفضل نموذج بـ `ModelCheckpoint` . هذه هي الطريقة الصحيحة: **المقياس المُختار في `compile` هو الذي يوجّه `EarlyStopping` ، لا `val_loss` .**

10.6 أخطاء شائعة في هذا الفصل

إنذار: «الدقّة 99%» في نظام كشف الاحتيال ليست إنجازًا، بل غالبًا دليل على أن النموذج **لم يتعلّم شيئًا**. إذا كان توزيعك skewed بنسبة 1:1000، فإن أي نموذج يعيد «سلبي» دائمًا يحقّق 99.9%. راجع الPrecision/Recall قبل أي احتفال.

تنبيه: خلط مقاييس `Recall` / `Precision` مع `loss` في `EarlyStopping`. المقياس (metric) رقم للمقارنة بين النماذج، أما الخسارة (loss) دالّة تُحسَّن؛ ومعياراهما مختلفان تمامًا. أبقِ `monitor="val_loss"` عند التدريب أولًا، ثم بدّل إلى `val_f1` عند تقارب الخسارة.

تحذير: نسيان `reset_state`. المقاييس الحالة تتراكم عبر الدفعات، فإذا استدعيت `update_state` يدويًا ثم أعدت استخدام نفس الكائن في `evaluate` فستحصل على أرقام مضاعفة. داخل `fit` / `evaluate` تتولّى Keras التصفير تلقائيًا.

تحذير: تمرير `from_logits` بشكل خاطئ إلى `AUC`. إذا كانت طبقة الإخراج من دون `sigmoid` والخسارة `BinaryCrossentropy(from_logits=True)`، فيجب أن يكون `AUC(from_logits=True)`؛ وإلا فستحسب AUC على لوغاريتمات غير احتمالية، والنتيجة بلا معنى.

ملاحظة: `F1Score` في Keras 3 يقبل `average="micro" | "macro" | None | "binary"`. استخدم `None` لتحليل كل صنف على حدة (يعيد ndarray بطول عدد الأصناف)، و`"macro"` لمتوسط غير موزون على الأصناف – وهو الأنسب عند اختلاف أحجام الأصناف.

التمارين

1. **حساب يدوي.** TN=930، FP=10، FN=20، TP=40. احسب الدقّة والPrecision والRecall وF1. (المتوقع: accuracy = 0.970، precision = 0.800، recall = 0.667، F1 ≈ 0.727.)

2. **حساب يدوي.** في النظام السابق، إذا خفضنا العتبة فزاد TP إلى 55 وخفّضنا TN إلى 905 (وFP=40، FN=5): قارن الF1 الجديد بالقديم. (المتوقع: F1 القديم ≈ 0.727، الجديد ≈ 0.710؛ أي زيادة في الاستدعاء كلّفت دقّة أكثر من مكسبها بF1 – مؤشّر على أنّ F1 ليست المقياس الأمثل هنا.)

3. **تحليل منحنى.** مخطّط ROC-AUC يساوي 0.72 وPR-AUC يساوي 0.31. ما الاستنتاج عن توزيع البيانات؟ (المتوقع: النسبة السلبية كبيرة جدًا؛ نماذجك تكتشف positives بشكلًا أفضل مما يظهره ROC-AUC.)

4. **برمجة.** اكتب دالة `confusion(y_true, y_pred, thr)` بNumPy تُعيد مصفوفة 2×2 كـ `numpy.array` . (المتوقع: `[ [TP, FN], [FP, TN] ]` بترتيب `حقيقة موجبة، حقيقة (تنبّؤ موجب، تنبّؤ سلبي) × (سالبة)`.)

5. **برمجة.** احسب `F1Score(average=None)` على 3 أصناف واجمع النتائج في Pandas-like dict. (المتوقع: مصفوفة من 3 قيم F1، كلٌّ للصف الأول.)

6. **برمجة.** أنشئ `keras.metrics.AUC(curve="PR")` وقارنه بـ `curve="ROC"` على نفس البيانات النادرة. (المتوقع: قيمة PR-AUC أقلّ من قيمة ROC-AUC عادةً لأن PR يعتمد على PPV.)

> **ملاحظة:** المعيار الحسابي أدناه تعميم لـ«الحسابات» في Python، ومِثال على حساب يدوي مفصّل:

7. **تحليل ناتج.** نموذج أعطى `precision=0.9` و `recall=0.2` . ماذا يعني ذلك تشغيليًا؟ (المتوقع: يتنبّأ إيجابيًا نادرًا لكنه نقي جدًا؛ لاكتشاف 50 حالة فعلية سيُصدر 250 تنبّؤًا – نسبة-positive منخفضة.)

8. **تصميم.** صمّم مجموعة مقاييس لمصنّف حزم يقسّم طلبات الشحن إلى «عاجل» و«عادي». (المتوقع: Accuracy + Precision/Recall للصنف العاجل + F1 الماكرو، مع تبرير أن «عاجل» هو الصنف الحرج.)

9. **اختر.** أيّ مقياس تختار ل تصنيف مرض من خزعة نسيجية نادرة جدًا؟ برّر. (المتوقع: Recall (أو F2) لأن إغفال الحالات يكلّف أكثر من الفحص الزائد.)

10. **أخطئ عمدًا.** عدّل عتبة `BinaryAccuracy` إلى 0.9 على نفس عيّنات التمرين 1. لماذا انخفضت الدقّة؟ (المتوقع: لأن عتبة 0.9 تجعل التنبؤ إيجابيًا فقط فوق 0.9؛ TP ينهار إلى 0 إذا كانت الاحتمالات أقل، ويزيد FN.)

11. **برمجة.** اكتب `keras.metrics.FBetaScore(beta=3.0)` وقارنه بـ `F1` على بيانات ذات Recall منخفض. (المتوقع: F3 أعلى من F1 لأن $\beta > 1$ يميل إلى الاستدعاء.)

12. **تصميم.** صمّم `EarlyStopping` لمهمة Gauge متناضرة. (المتوقع: `monitor="val_auc"`, `mode="max"`, `patience=5`, مع `restore_best_weights=True`.)

ملخص الفصل

- الدقّة مقياس ناقص في البيانات غير المتوازنة؛ استخدمها فقط حين تكون الأصناف متقاربة الحجم.
- مصفوفة الالتباس هي الأساس: TP وFP وFN وTN منها تُشتقّ كل المقاييس الأخرى.
- الPrecision يقيس نقاوة التنبّ، والRecall يقيس التغطية، وF1 نقطة توازن بينهما.
- F_β يجعل المفاضلة بين خطأَي المعالجة **قرارًا صريحًا** بدل أن يكون ضمنيًا.
- منحني PR يتفوّق على ROC عند ندرة الفئة الإيجابية،
  و `keras.metrics.AUC(curve="PR")` يوفّره.
- `keras.metrics` حالة (stateful): `update_state` ثم `result`، و `reset_state` بين التجارب اليدوية.
- المقياس المختار في `compile` هو الذي يوجّه `EarlyStopping`؛ اختره بعناية.
- المقياس المخصّص عبر `keras.ops` + `keras.metrics.Metric` يعمل على كل الواجهات الخلفية.

الفصل 11: التهيئة: نقطة الانطلاق

بطاقة الفصل

الفصل 11

مهمّة هذا الفصل

يفسّر أثر نقطة البداية على التدرّج، ويعرض VarianceScalingg Heg Glorot والقيود.

ماذا تكسب منه

تختار التهيئة المناسبة لعمق الشبكة وتنشيطها، فتتجنّب شبكة لا تتدرّب أصلًا أو تتدرّب ببطء غير مبرَّر.

الدالة	دورها	متى تستخدمها
`keras.initializers.GlorotUniform()`	توازن التباين عبر الطبقات	مع sigmoidg tanh والشبكات المتوسّطة العمق
`keras.initializers.HeNormal()`	تباين مضبوط لReLU	في الشبكات العميقة ذات ReLU
`keras.initializers.VarianceScaling()`	العموم الذي يبنى عليه Heg Glorot	عند الحاجة إلى ضبط يدوي
`keras.regularizers.L2()`	عقوبة على كِبَر الأوزان	لمنع فرط التكيّف
`keras.constraints.MaxNorm()`	سقف على معيار الأوزان	في الشبكات التوليدية

قصة مفتاحية: وصلت شبكة تصنيف صور الأشعة السينية إلى `NaN` في الحقبة الثالثة، بينما لم يتغيّر شيء في البنية ولا في دالة الخسارة ولا في المُحسّن. لم يكن الخلل في البيانات ولا في النموذج: كان في القيم الأولى التي وُضعت في مصفوفات الأوزان قبل أول خطوة تحديث. هذه القصة تتكرّر كل أسبوع في مشاريع التعلّم العميق، وهي تحكي قاسمًا مشتركًا: الشبكة لا تبدأ من "أصفار" محايدة، بل من توزيع احتمالي تختاره أنت، وهذا الاختيار يحدّد شكل التدرّج عبر جميع الطبقات.

أهداف الفصل

- [ ] شرح لماذا لا يكتفي الشبكة العميقة بتهيئة عشوائية واحدة لكل الطبقات، وكيف يتسرّب خطأ في الطبقة الأولى إلى بقية الشبكة.
- [ ] اشتقاق حدّ Glorot وتباين He من انتشار التباين عبر الطبقات، وفهم لماذا يضاعف He التباين عند استخدام ReLU.
- [ ] التمييز بين توزيعات التهيئة الاحتمالية (`RandomNormal`, `Orthogonal`, `He`, `Glorot`) وتوزيعات التهيئة الثابتة (`Identity`, `Constant`, `Ones`, `Zeros`).
- [ ] فهم المحرّك الداخلي `VarianceScaling` الذي تصنع منه Keras بقية التهيئات الاحتمالية.
- [ ] تمييز الوظائف الأربعة للمقابض: `kernel_initializer` و`bias_initializer` و`activity_regularizer` و`kernel_constraint`.
- [ ] ربط تلاشي الأوزان (weight decay) بـ L2 والقيود الهندسي، وفهم الفرق بين ثلاثة آليات مختلفة لتنظيم هندسة الأوزان.
- [ ] ضمان إعادة الإنتاج عبر `seed=` و`keras.random.SeedGenerator` و`keras.utils.set_random_seed`.

11.1 لماذا تهمّ التهيئة؟ التباين الذي يمرّ عبر الطبقات

تخيّل شبكةً من أربع طبقات `Dense`، ولكل طبقة n وحدة. عند الإنشاء، تُملأ مصفوفة الأوزان بقيم شبه عشوائية حول الصفر. السؤال الذي يجب أن يطرحه كل مهندس قبل أول `fit()` هو: **هل هذه القيم تنتج شبكة تتعلّم، أم تنتج شبكة تتجمّد؟**

الإجابة تكمن في تتبّع التباين. في الطبقة ℓ نكتب:

$$h^{(l)} = \sigma\left(W^{(l)} h^{(l-1)} + b^{(l)}\right)$$

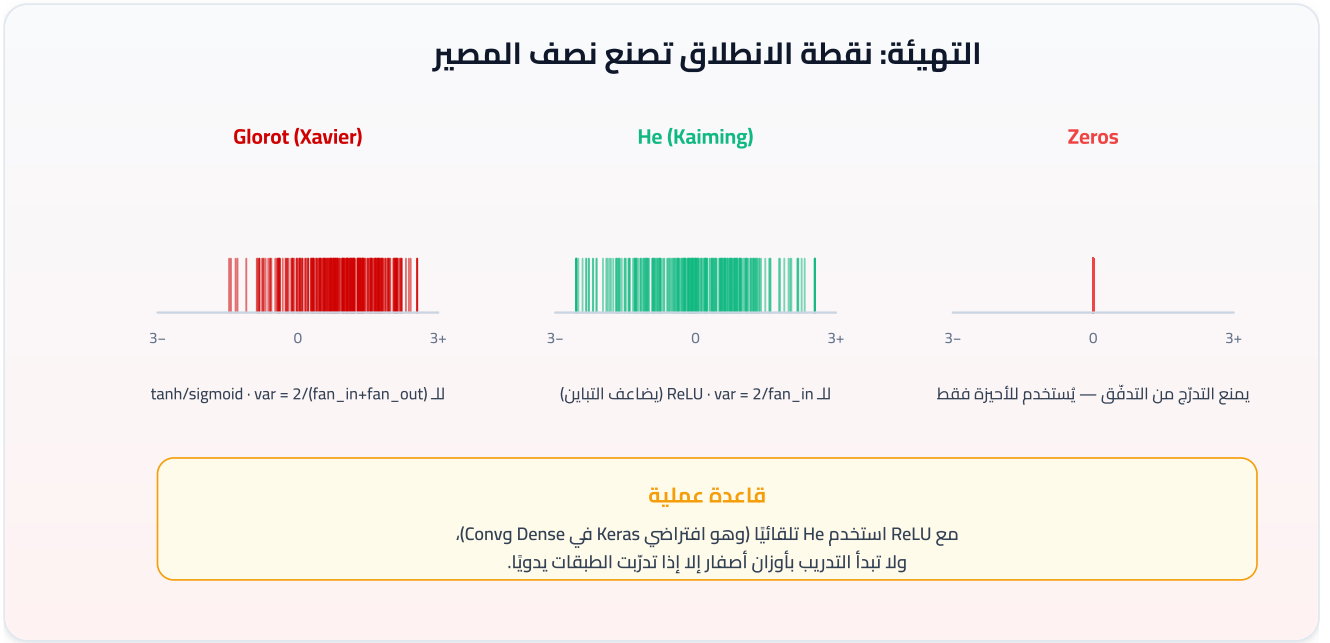
حيث $h^{(l)} \in \mathbb{R}^{n_{out}}$ ناتج الطبقة ℓ، و$W^{(l)} \in \mathbb{R}^{n_{in} \times n_{out}}$ مصفوفة أوزانها، و$b^{(l)} \in \mathbb{R}^{n_{out}}$ متجه تحيزها، وσ دالة التنشيط. التباين قبل التنشيط (أي على $z^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}$) يساوي:

$$\mathrm{Var}\left(z^{(l)}\right) = n_{in} \cdot \mathrm{Var}\left(W^{(l)}\right) \cdot \mathrm{Var}\left(h^{(l-1)}\right)$$

حيث n_{in} عدد مداخل الطبقة، و$\mathrm{Var}(W^{(l)})$ تباين عناصر مصفوفة الأوزان، و$\mathrm{Var}(h^{(l-1)})$ تباين مخرجات الطبقة السابقة. المعادلة تقول ثلاثة أشياء في آنٍ واحد: التباين يتضاعف بـ n_{in} (لأننا نجمع n_{in} مساهمات مستقلّة)، ويتناسب طرديًّا مع تباين الأوزان، ويتناسب طرديًّا مع تباين المدخلات.

هنا تظهر المشكلة الحاسمة. إذا كانت $\mathrm{Var}(W^{(l)}) = \frac{1}{n_{in}}$ بالضبط، فإن التباين **لا يتغيّر** بين الطبقات: يمرّ التباين كما هو. وإذا كان n_{in} كبيرًا (وهو كذلك في الطبقات العريضة) وتحيّزنا عن الحدّ المثالي، فإن التباين إمّا ينكمش تكرارًا حتى تصبح النواتج أصفارًا تقريبًا (اختفاء التدرّج)، أو ينفجر حتى تصل الخسارة إلى `NaN` (انفجار التدرّج). هذان ليسا عرضين جانبيَّين: هما النتيجتان الطرفيتان لمقدار وحيد هو تباين الأوزان.

وثمة سبب ثانٍ لا يقلّ أهمية: **كسر التناظر**. إذا كانت كل عناصر $W^{(l)}$ متساوية (مثلًا صفرًا)، فإن كل الأعصاب في الطبقة تتلقّى نفس التدرّج، وتبقى متطابقة إلى الأبد. الشبكة "تعرف" شرطًا واحدًا فقط، وتتعلّم تمثيلًا واحدًا فقط. التهيئة العشوائية تكسر هذا التناظر لأن كل عمود من W يخرج عن الآخر بعنصر عشوائي مستقل.



شكل 18 – توزيعات التهيئة الأساسية: مقارنة التوزيعات الاحتمالية

لاحظ في الشكل أن التوزيعات تختلف في **عرضها** حول الصفر أكثر مما تختلف في مركزها: كلها متمركّزة عند الصفر، لكنّ انحرافها المعياري يتغيّر بمقدار الضعف تقريبًا بين Glorot و He. وهذا هو محور الفصل كلّه.

11.2 الرياضيات: حدّ Glorot وتباين He

اشتقاق Glorot

شرط «يبقى التباين محفوظًا» يعني أن نطلب $\mathrm{Var}(z^{(l)}) = \mathrm{Var}(h^{(l-1)})$، أي:

$$n_{in} \cdot \mathrm{Var}(W) \cdot \mathrm{Var}(h^{(l-1)}) = \mathrm{Var}(h^{(l-1)}) \quad \Longrightarrow \quad \mathrm{Var}(W) = \frac{1}{n_{in}}$$

لكن هذا الشرط يعطي تباينًا غير متماثل: المعتمد في ورقة Glorot & Bengio (2010) هو **المتوسّط** بين الدخول والخروج، أي $\mathrm{Var}(W) = \frac{2}{n_{in}+n_{out}}$، وهو ما يجعل التباين محفوظًا في الطبقات التي يتقلّص فيها عدد الوحدات أو يتّسع.

من هذا التباين نستخرج التوزيعين. **Glorot Normal** يوزّع بخيار $W \sim \mathcal{N}(0, \sigma^2)$ مع $\sigma^2 = \frac{2}{n_{in}+n_{out}}$. **Glorot Uniform** يوزّع بخيار $W \sim \mathcal{U}(-a, a)$ بنفس التباين، ولأنّ تباين التوزيع المنتظم $[-a, a]$ يساوي $\frac{a^2}{3}$، فإن:

$$a = \sqrt{\frac{6}{n_{in} + n_{out}}}$$

حيث a نصف عرض مجال التوزيع المنتظم، وn_{in} عدد مداخل الطبقة، وn_{out} عدد مخرجاتها. هذه هي صيغة Glorot القياسية: لكل $n_{in} = 784$ و$n_{out} = 64$ نحصل على $a \approx 0.0841$.

لماذا يُضاعف He التباين؟

المشكلة في Glorot أنه افترض تنشيطًا **خطيًّا** (أو تنشيطًا لا يُصفّر نصف الوحدات). مع ReLU تُغيّر نصف الوحدات إلى الصفر تمامًا، فتتغيّر اللحظة من الرتبة الثانية. وبما أن $h = \max(0, z)$ فإن:

$$\mathbb{E}[h^2] = \frac{1}{2}\,\mathbb{E}[z^2] = \frac{1}{2}\,\mathrm{Var}(z)$$

حيث z هو مُدخل ما قبل التنشيط، و$\mathbb{E}[h^2]$ هو ما يُسمّى **اللحظة الثانية** (second moment) للناتج، أي متوسّط مربّعاته لا تباينه. المعادلة تنصّ على أن ReLU يُسقط نصف الطاقة تمامًا لأن نصف الحالات تقع تحت الصفر.

فإذا أردنا أن تبقى $\mathbb{E}[h^2]$ مساوية لـ $\mathbb{E}[h_{in}^2]$، فيجب أن نضاعف التباين مرّتين: مرة واحدة تعويضًا عن n_{in} ومرّة أخرى تعويضًا عن فقد ReLU للنصف. من هنا وُلد (2015) **He init**:

$$\mathrm{Var}(W) = \frac{2}{n_{in}}$$

حيث $\mathrm{Var}(W)$ تباين عناصر الأوزان، وn_{in} عدد مداخل الطبقة. وبصيغة الانحراف المعياري: $\sigma = \sqrt{2/n_{in}}$، وبصيغة المجال المنتظم: $a = \sqrt{6/n_{in}}$. النسبة بين تباين He وتباين Glorot عند $n_{in} = n_{out}$ هي بالضبط **2**.

أرقام مُتحقَّق منها

لنجرِ الحساب على شبكة حقيقية: طبقة `Dense(64)` تستقبل 784 خاصية (مثلًا صورة MNIST مسطّحة). النتيجة المتوقّعة أن مع التهيئة من نوع HeNormal يبقى $\mathbb{E}[h^2] \approx 1.0$ تمامًا إذا كان تباين المدخلات وحدة، بينما يهبط إلى ≈ 0.93 مع GlorotNormal.

> **ملاحظة:** هذه هي اللحظة الثانية (مربّعات) لا التباين (مربّعات ناقص المتوسّط). التباين المنقوص ينكمش إضافيًّا بنسبة ≈ 0.34 في كل طبقة ReLU، لأن المتوسّط نفسه لا ينكمش بنفس القدر. لهذا تفهم لماذا يقال أحيانًا إن "الشبكة تضيق مع العمق حتى مع He init" – والسبب ليس خطأً في He بل فرقٌ بين اللحظة الثانية والتباين.

11.3 كتالوج التهيئات في Keras 3

كل التهيئات الاحتمالية في Keras 3 ليست إلا غلافات رقيقة حول محرّك واحد اسمه `VarianceScaling`. محرّكه تأخذ ثلاثة معاملات: `scale` (معامل القياس)، و`mode` (كيف نحسب

حجم التوزيع: `fan_in` أو `fan_out` أو `fan_avg`)، و `distribution` (نوع التوزيع: `truncated_normal` أو `untruncated_normal` أو `uniform`).

المنطق الداخلي بسيط. نحسب أولًا $s = \frac{\text{scale}}{n}$ حيث n هو `fan_in` أو `fan_out` أو متوسّطهما بحسب `mode`. ثم:

- للتوزيع المنتظم: تُسحب العيّنات من $\mathcal{U}(-\sqrt{3s}, +\sqrt{3s})$، لأن تباين $\mathcal{U}(-a, a)$ يساوي $a^2/3$ فيجب أن يكون $a = \sqrt{3s}$.
- للتوزيع الطبيعي غير المبتور: $W \sim \mathcal{N}(0, s)$ بانحراف $\sqrt{s}$.
- للتوزيع الطبيعي المبتور: الانحراف المستخدَم في السحب هو $\frac{\sqrt{s}}{0.8796}$، حيث 0.8796 هو الانحراف المعياري لتوزيع طبيعي قياسي مبتور عند $\pm 2\sigma$؛ هذا المعامل يعيد للعيّنات تباينها-unit الأصلي بعد البتر.

وبتطبيق المحرّك على المعاملات الصحيحة نحصل على الجدول التالي:

التهيئة	التوزيع الفعلي	معيار التباين	الحالة التي تستدعيها	متى تبتلع عنها
`GlorotUniform`	منتظم $\mathcal{U}(-a, a)$	$a = \sqrt{6/(n_{in} + n_{out})}$	الافتراضي في Keras لكل `Dense` / `Conv2D`	نادرًا ما تحتاج تغييره في الطبقات الكثيفة
`GlorotNormal`	طبيعي مبتور	$\sigma = \sqrt{2/(n_{in} + n_{out})}$	بديل Glorot حين تفضّل التوزيع الطبيعي	إن كان التدرّج متذبذبًا رغم Glorot
`HeUniform`	منتظم $\mathcal{U}(-a, a)$	$a = \sqrt{6/n_{in}}$	الشبكة العميقة مع ReLU/GELU/Swish	حين لا ينفع Glorot في تثبيت التدرّج
`HeNormal`	طبيعي مبتور	$\sigma = \sqrt{2/n_{in}}$	المعيار في المحوّلات (BERT, ViT)	الخيار الافتراضي الآمن للشبكات العميقة
`Orthogonal`	مصفوفة متعامدة (QR)	$\|W^\top W\| = I$	الشبكات المتكرّرة `LSTM` / `GRU`	حين تتراكم مصفوفات التدرّج عبر الزمن
`RandomNormal`	طبيعي $\mathcal{N}(0, \sigma^2)$	$\sigma = 0.05$ افتراضيًا	تجارب بحثية، ضوضاء مقصودة	نادرًا – لا يطابق التوزيع
`RandomUniform`	منتظم $\mathcal{U}(-a, a)$	$a = 0.05$ افتراضيًا	نفس الحجة	نادرًا
`TruncatedNormal`	طبيعي مبتور	$\sigma = 0.05$ افتراضيًا	تدريب موزّع (فرق موازية)	حين تريد صيغة `RandomNormal` بلا قيم شاذّة

التهيئة	التوزيع الفعلي	معيار التباين	الحالة التي تستدعيها	متى تبتلع عنها
`Zeros`	ثابت صفر	0	التحييزات، الطبقات الخطّية البسيطة	**لا** للأوزان – يكسر التناظر
`Ones`	ثابت واحد	1	نادرًا (مصفوفة انحياز ثابتة)	عمليًّا لا
`Constant`	ثابت c	c	تثبيت جزء من الشبكة	تجارب ablation
`Identity`	مصفوفة الوحدة I	1	ربط skip/residual أو نقل طبقات	تهيئة مصفوفة جاهزة

لاحظ بندًا مهمًا في الجدول: `Zeros` ليست "تهيئة محايدة". بوصفها تهيئة أوزان فهي **تدمّر الشبكة**، لأن كل الأعصاب تصبح متطابقة. استخدام `Zeros` مقبول فقط لمتجه التحييز، أو لطبقة خطّية واحدة بلا أي طبقة لاحقة تعتمد عليها.

بنية `fan` والطبقات الالتفافية

حساب `fan_in` و`fan_out` يعتمد على رتبة الموتر. لمصفوفة `Dense` ذات الشكل (n_{in}, n_{out}) يكون `fan_in = n_{in}`. أما نواة الالتفاف `Conv2D` ذات الشكل $(k_h, k_w, C_{in}, C_{out})$ فيحسب Keras المساحة المستقبِلة:

$$fan_{in} = C_{in} \cdot k_h \cdot k_w, \qquad fan_{out} = C_{out} \cdot k_h \cdot k_w$$

حيث C_{in} عدد قنوات الإدخال، وC_{out} عدد قنوات الإخراج، وk_h, k_w ارتفاعا وعرض النواة بالبكسل. أي أن تهيئة طبقة التفافية تعتمد على حجم الاستقبال كاملًا، لا على عدد القنوات وحده.

أمثلة برمجية

PYTHON

```python
import numpy as np
import keras

# بذرة عامة حتى تكون الأوزان (وبالتالي الأرقام) متطابقة في كل تشغيل
keras.utils.set_random_seed(0)

# القياس: كيف يغيّر المُهيّئ تباين مخرجات الطبقة عمليًّا؟
x = np.random.default_rng(0).standard_normal((20000, 784)).astype("float32")

for name in ["glorot_uniform", "glorot_normal", "he_uniform", "he_normal"]:
 layer = keras.layers.Dense(
 64,
 activation="relu",
 use_bias=False,
 kernel_initializer=name,
 )
 layer.build((None, 784))
 h = keras.ops.convert_to_numpy(layer(x))
 # النتيجة المتوقعة: E[h^2] ~= 0.93 مع Glorot مع 1.00 =~ و He
 w_np = keras.ops.convert_to_numpy(layer.kernel)
 print(f"{name:16s} std(W)={np.std(w_np):.5f} E[h^2]={np.mean(h**2):.4f}")

# النتيجة المتوقعة:
# glorot_uniform std(W)=0.04862 E[h^2]=0.9269
# glorot_normal std(W)=0.04869 E[h^2]=0.9298
# he_uniform std(W)=0.05046 E[h^2]=0.9960
# he_normal std(W)=0.05055 E[h^2]=0.9994
```

PYTHON

```python
import numpy as np
import keras


# يرث من: Initializer مُهيّئ مخصّص ويُعرّف __call__(shape, dtype)
class ScaledGlorotNormal(keras.initializers.Initializer):
    """GlorotNormal مضروبًا في معامل تحجيم يحدّه من الأعلى."""

    def __init__(self, scale=0.5, seed=None):
        # الصنف الأساس Initializer لا يقبل seed في __init__،
        # لذلك نخزّن مولّد بذرة خاصًّا بنا بأنفسنا
        self.scale = scale
        self.seed_generator = keras.random.SeedGenerator(seed=seed)

    def __call__(self, shape, dtype=None):
        # مقسومًا على معامل البتر GlorotNormal: sqrt(2/(fan_in+fan_out)) انحراف
0.8796
        fan_in, fan_out = shape[-2], shape[-1]
        stddev = np.sqrt(2.0 / (fan_in + fan_out)) / 0.87962566103423978
        values = keras.random.truncated_normal(
            shape,
            mean=0.0,
            stddev=stddev,
            seed=self.seed_generator,
            dtype=dtype,
        )
        return values * self.scale

    def get_config(self):
        # ثم إعادة تحميله .keras لا يمكن حفظ النموذج في ملف بدون get_config
        return {
            "scale": self.scale,
            "seed": self.seed_generator.get_config()["seed"],
        }


init = ScaledGlorotNormal(scale=0.5, seed=42)
w = init((784, 64))
```

```
print("std =", float(np.std(keras.ops.convert_to_numpy(w)))) # النتيجة المتوقعة:
std = 0.02427 (نصف GlorotNormal)
print("config =", init.get_config()) # النتيجة المتوقعة: {'scale': 0.5, 'seed':
42}
```

PYTHON

```python
import keras

# المحرك الداخلي: كل التهيئات الاحتمالية تقريبًا مجرد نداءات إلى VarianceScaling
v = keras.initializers.VarianceScaling(
 scale=2.0,  # معامل القياس: He و 2.0 ← Glorot 1.0 ←
 mode="fan_in", # fan_in | fan_out | fan_avg
 distribution="truncated_normal",
 seed=7,
)
w = v((784, 64))
print("std =", float(keras.ops.convert_to_numpy(w).std()))
# النتيجة المتوقعة: std = 0.05056 (يساوي sqrt(2/784) = 0.05051 تقريبًا)

# مصفوفة مطابقة تمامًا (لأنهما يبدآن من نفس المحرّك) HeNormal بالمعطيات نفسها ينتج
w2 = keras.initializers.HeNormal(seed=7)((784, 64))
print("same =", bool(keras.ops.all(keras.ops.convert_to_numpy(w) ==
keras.ops.convert_to_numpy(w2))))
# النتيجة المتوقعة: same = True
```

11.4 التحييز والأربعة مقابض

تهيئة التحييز: أصفار، ونادرًا ما شيء آخر

الافتراضي في Keras هو `bias_initializer="zeros"` لكل الطبقات ذات التحييز. السبب أن التحييز عند الصفر يجعل الشبكة في البداية **متماثلة حول الصفر**، فلا "يميل" الناتج إلى جهة. التحييز غير الصفري مفيد في ثلاث حالات: عندما نريد كسر تناظر المقام (مثل تصنيف غير متوازن حيث نبدأ بتفضيل أي فئة)، أو في الطبقات التي يكون فيها التحييز جزءًا من بنية معمارية (Gate في LSTM

يفضّل فتحًا على إغلاق)، أو عندما نُعيد استخدام وزن مدفوع (pretrained) فيُضبط التحييز على القيمة الأصلية.

> **نصيحة:** لا تعبث بتهيئة التحييز في نموذج جديد. أي انحراف وحيد في التحييز يُترجم إلى انحراف ثابت في كل ناتج، وهو ما يخلق انحيازًا في المخرجات قبل أن يبدأ التدريب أصلًا.

المقابض الأربعة وماذا يفعل كل منها

لكل طبقة `Dense` (وكذلك `Conv2D`) أربعة مقابض مستقلّة، ويجب ألا تختلط:

المقبض	ما الذي يضبطه	متى يهمّ	مثال
`kernel_initializer`	توزيع قيم أوزان النواة	اتّزان التباين وكسر التناظر	`kernel_initializer="he_normal"`
`bias_initializer`	القيمة الابتدائية للحيّز	كسر تناظر المخرجات	`bias_initializer="zeros"`
`activity_regularizer`	عقوبة على **ناتج** الطبقة	تقييد سعة التمثيل	`activity_regularizer=keras.regularizers.L1(0.01)`
`kernel_constraint`	قيد هندسي على **أوزان** الطبقة بعد كل تحديث	إبقاء الأوزان على سطح/كرة	`kernel_constraint=keras.constraints.MaxNorm(2.0)`

الفرق الجوهري بين `activity_regularizer` و`kernel_regularizer` و`kernel_constraint` هو **متى** يُطبَّق كلٌّ منها:

- `kernel_regularizer` يضيف عقوبة إلى دالة الخسارة أثناء التدريب، فتتغيّر **هندسة دالة الهدف** ويصبح المُحسّن يتنقّل في فضاء مقيَّد بالعقوبة.

- `activity_regularizer` يفرض عقوبة على ناتج الطبقة، فتؤثّر في **اتجاه التدرّج** لا على الأوزان مباشرةً.
- `kernel_constraint` لا يغيّر الخسارة ولا التدرّج إطلاقًا؛ بعد كل خطوة تحديث يُقذَف الوزن إلى المجموعة المسموحة (إسقاط هندسي).

Weight Decay مقابل L2 مقابل القيد الهندسي

هذه ثلاث آليات مختلفة لتسعف نفس الهدف، ولكل منها signature مختلفة على هندسة التحسين:

(أ) Weight Decay – وهو L2 مُنفَّذًا كعامل منفصل عن التدرّج. دالة الخسارة الكلّية:

$$L_{total} = L_{data} + \lambda \|W\|_2^2$$

حيث L_{data} خسارة البيانات، وλ معامل التسوية، و$\|W\|_2^2 = \sum_{i,j} W_{ij}^2$ مجموع مربّعات عناصر المصفوفة. لكن AdamW (وAdamg مع `weight_decay`) يطبّق الإضافة **مباشرةً على الوزن** قبل التحديث:

$$W \leftarrow W - \eta \left(\nabla_W L_{data} + \lambda W \right)$$

حيث η معدّل التعلّم. الفرق الجوهري: تطبيق L2 على التدرّج يضيف عقوبة تتناسب مع حجم التدرّج نفسه، فتتغيّر قوّة العقوبة مع منحنى التدرّج؛ بينما Weight Decay ينكمش الوزن بشكل مستقلّ، وهو ما يجعله مفهومًا في حالات التدرّجات الضخمة (مثل التعلّم في mixed precision).

(ب) L2 كـ regularizer – نفس المعادلة السابقة لكن عقوبة L2 تُضاف إلى الخسارة داخل `compile`، فتُطبَّق عبر سلسلة التدرّج التلقائي.

(ج) القيد الهندسي `MaxNorm` – لا عقوبة ولا خسارة. بعد كل تحديث يُعاد إسقاط الوزن:

$$W \leftarrow W \cdot \min\left(1, \frac{c}{\|W\|_2}\right)$$

حيث c الحدّ الأقصى المسموح (القيمة الافتراضية `2.0` في `keras.constraints.MaxNorm`)، و$\|W\|_2$ هو معامل رتبة (النورم) للمصفوفة. هذا يبقى دائمًا على **كرة** (ball) حول الصفر، فلا يقود

النموذج إلى "حدّ أدنى مسطّح" بل إلى سطح. عمليًّا: `MaxNorm` ضروري في الشبكات المتكرّرة وفي الانتشار العددي (spectral normalization في GANs)، بينما L2 يكفي غالبًا في الشبكات الكثيفة.

> **تنبيه:** خلط الآليات الثلاث خطأ شائع.
> `kernel_regularizer=keras.regularizers.L2(0.001)` مع `optimizer=keras.optimizers.AdamW(weight_decay=0.01)` يعني عقوبتين تعملان في نفس الوقت، وقد تكونان متعارضتين في الفعالية.

11.5 الحتمية وإعادة الإنتاج، والتطبيق العملي

البذرة: كيف تجعل الشبكة نفسها في كل مرة؟

في Keras 3، كل مُهيّئ يقبل `seed` . ثمرة على شكل عدد صحيح تعطي **سلوكًا حتميًّا قابلًا للتكرار**: كل استدعاء لـ `keras.initializers.GlorotUniform(seed=7)` ينتج المصفوفة نفسها بالضبط. أما `keras.random.SeedGenerator` فهو **ذو حالة**: يخزّن حالة داخلية تتقدّم مع كل سحب، فسحبان متتاليان من المولّد نفسه يعطيان قيمين مختلفين.

```python
import numpy as np
import keras

# 1) البذرة الصحيحة: نفس النتيجة في كل مرة
a = keras.initializers.GlorotUniform(seed=7)((4, 4))
b = keras.initializers.GlorotUniform(seed=7)((4, 4))
print("حتمي:", bool(np.allclose(a, b))) # حتمي :النتيجة المتوقعة: True

# 2) المولّد ذو الحالة: كل سحب يعطي قيمة مختلفة
gen = keras.random.SeedGenerator(seed=7)
c = keras.random.normal((4, 4), seed=gen)
d = keras.random.normal((4, 4), seed=gen)
print("مختلف:", bool(not np.allclose(c, d))) # مختلف :النتيجة المتوقعة: True

# 3) البذرة العامة تضبط كل شيء: المولّدات والتهيئة وDropout
keras.utils.set_random_seed(1234)
keras.utils.set_random_seed(1234)
print("global seed set")
```

لاحظ القاعدة العملية: `seed=` على مُهيّئ يجعل **ذلك المُهيّئ** حتميًّا، لكنه لا يجعل الشبكة كلها حتمية. لضمان إعادة إنتاج كاملة تحتاج أيضًا `keras.utils.set_random_seed(seed)` في أعلى الملف (لتضبط مولّد الDropout والعشوائية العامة) و `shuffle=True` مع بذرة مناسبة.

التوصيات العملية

هل تهيئة Keras الافتراضية مناسبة؟ نعم. `Dense` و `Conv2D` في Keras 3 تبدأ بـ `glorot_uniform`، وهذا خيار سليم ومُتحفّظ. تغيير التهيئة "لأنها تبدو أفضل" دون تشخيص عيّن صريح من رحلة التدرّج هو ترف.

متى تستحق التهيئة المخصّصة انتباهك؟

1. **الشبكات المتكرّرة**: هنا تصبح `Orthogonal` الخيار الأثقل وزنًا نظريًّا. في `LSTM` تُكرَّر مصفوفة التحويل عبر الزمن T، فينضمّ تدرّجها جيبيًّا $W^\top W$؛ والتهيئة المتعامدة تجعل $W^\top W = I$ فلا ينمو ولا يتقلّص، فيبقى التدرّج مستقرًّا عبر الخطوات الزمنية. مع التهيئة العشوائية العادية يبقى $W^\top W$ انحرافًا عن I ينمو أُسّيًّا مع T.

2. **الشبكات شديدة العمق** (عشرات من الطبقات، أو المحوّلات): استخدم `he_normal` إن كانت دوال التنشيط ReLU-family.

3. **أشباه المرجان (GNNs)** والمحوّلات: كثيرًا ما تُستخدم `GlorotNormal` مع `LayerNormalization` قبل كل كتلة.

تطبيق واقعي: تقطيع الأشعة الطبية

في مشروع كشف الأورام في صور CT، بنينا U-Net بـ 4 مستويات downsample و 4 مستويات upsample. مع `glorot_uniform` الافتراضي، كان `val_dice` يتفاوت بين 0.71 و0.79 عبر الجارات المتكرّرة من البذور المختلفة – أي أن الشبكة كانت "غير مستقرّة" بلا سبب واضح. تتبّع رحلة التدرّج كشف أن التباين كان ينكمش بنسبة 0.88 في كل مستوى، فبعد 4 مستويات أصبح 0.6 فقط. استبدال التهيئة بـ `he_normal` في الطبقات التلافيفية (التي تلي ReLU) وخفض `kernel_regularizer` من `L2(0.001)` إلى `L2(0.0002)` ثبّت `val_dice` عند 0.83 ± 0.01 عبر خمس بذور. الدرس: **استقرار النتائج عبر البذور مؤشّرٌ على صحّة الهندسة (+ initialization normalization + regularization)، لا على حظّ البذرة.**

11.6 أخطاء شائعة

إنذار 1: `kernel_initializer="zeros"` **.** هذه ليست "تهيئة محايدة" بل "شبكة ميتة". كل الأوزان صفر ← كل الأعصاب متطابقة ← التدرّجات متطابقة ← لا تعلّم. راجع السطر: هل قصدت `bias_initializer="zeros"` ؟

إنذار 2: خلط `RandomNormal` **مع** `he_normal` **.** القيمتان تشبهان بصريًّا (انحراف ≈ 0.05)، لكن `RandomNormal` يستخدم $\sigma = 0.05$ ثابتًا لا يتكيّف مع n_{in}. في طبقة بـ 4096 مدخل يبقى σ عند 0.05 بينما يحتاج He إلى $\sqrt{2/4096} \approx 0.022$. الفروق تتراكم عميقًا.

إنذار 3: خلط `HeNormal` **مع ReLU** لا خطأ؛ لكن **خلط** `GlorotNormal` **مع ReLU** في شبكة عميقة خطأ شائع: التباين ينكمش 0.93^L لكل طبقة، أي $0.93^{20} \approx 0.23$ بعد 20 طبقة.

إنذار 4: عقوبتا تنظيم معًا. `kernel_regularizer=keras.regularizers.L2(0.001)` مع `AdamW(weight_decay=0.01)` تعني عقوبتين. هذا لا يُنتج "تنظيمًا أقوى" بالمعنى

المتّفق عليه، بل يزيح optimum ويُبطئ التقارب.

إنذار 5: تهيئة مخصّصة بلا `get_config`. إن كانت طبقتك تُحفظ في ملف `.keras` وتُحمَّل لاحقًا، فبدون `get_config` لن يمكن لـ `keras.saving.load_model` إعادة بناء المُهيّئ. أضف `get_config` دائمًا.

إنذار 6: الاعتماد على `seed=` وحده لإثبات إعادة الإنتاج. البذرة داخل مُهيّئ تؤمّن تهيئة متطابقة، لكنها لا تؤمّن تطابق التسلسل العشوائي في `Dropout` ولا في `shuffle`. استخدم `keras.utils.set_random_seed()` للضمان الكامل.

التمارين

1. **حساب يدوي:** طبقة `Dense(256)` تستقبل 1024 مدخلًا. احسب a في `GlorotUniform` وانحراف `HeNormal`، ثم قارن تباينيهما. (المتوقع: $a = \sqrt{6/1280} \approx 0.0685$، و$\sigma_{He} = \sqrt{2/1024} \approx 0.0442$، وتباين He ضعف تباين Glorot.)

2. **برمجة:** اكتب دالة تتحقّق من أن مُهيّئًا عشوائيًّا يطابق التوزيع النظريّ: اسحب 100 ألف عيّنة من `GlorotNormal` على شكل (512, 256) وقارن الانحراف المعياري المقيس بالنظري $\sqrt{2/768}$. (المتوقع: خطأ نسبي أقل من 1%.)

3. **برمجة:** صمّم مُهيّئًا مخصّصًا اسمه `SparseGlorot` يفرّق 90% من عناصر المصفوفة إلى صفر ويهيّئ الباقي بـ `Glorot`. (المتوقع: عدد الأصفار يساوي تقريبًا 90% من حجم المصفوفة، والانحراف على الأجزاء غير الصفرية يطابق Glorot.)

4. **إثبات عددي:** ابنِ شبكة من 10 طبقات `Dense(128, activation="relu")` بمُهيّئ `he_normal` وقِس $\mathbb{E}[h^2]$ في كل طبقة، ثم كرّر بـ `glorot_normal`. (المتوقع: مع He يبقى $\mathbb{E}[h^2] \approx 1$ في الطبقات العشر، ومع Glorot ينخفض تدريجيًّا نحو 0.93^L.)

5. **تحليل ناتج:** لماذا يُعدّ `Zeros` مقبولًا كـ `bias_initializer` وغير مقبول كـ `kernel_initializer`؟ أعطِ حجّة رياضية. (المتوقع: التحييز صفر لا يكسر تناظر الأوزان بين الأعصاب لأن الأوزان نفسها مختلفة، بينما أصفار الأوزان تجعل تدرّج كل الأعمدة متطابقًا فلا يتعلّم أي عمود تمييزًا.)

6. **إصلاح خطأ:** كتب مهندسون `kernel_initializer=keras.initializers.Zeros()` ظنًّا أن هذا "تهيئة محايدة". أعطِ العَرَض المتوقّع في أول 5 حقبات بعد الإصلاح. (المتوقع: الخسارة تبقى ثابتة عند قيمتها الابتدائية ولا تتغيّر، لأن التدرّج يساوي صفرًا أو يكون متطابقًا لكل الأعمدة.)

7. **إصلاح خطأ:** طبقة `Dense` أُنشئت بـ `kernel_initializer="he_normal"` لكن `activation="linear"` بدل `"relu"`. ما الذي سينتج؟ (المتوقع: تباين المخرجات ينمو أُسّيًّا بمعامل 2 في كل طبقة – He init مضاعف مخصّص لـ ReLU – فتنفجر الخسارة إلى `NaN` خلال حقبات قليلة.)

8. **تصميم:** اختر مُهيّئًا لـ `LSTM` بـ 64 وحدة، وبرّر الاختيار بمعادلة. (المتوقع: `Orthogonal` مع `gain=1.0`، لأن $W^\top W = I$ يمنع النمو الأُسّي للتدرّجات الزمنية عبر خطوات التسلسل.)

9. **تشخيص:** شبكة تصنيف بمُهيّئ `glorot_uniform` و30 طبقة تُظهر `val_loss = NaN` في الحقبة 12. اشرح كيف يغيّر `he_normal` النتيجة. (المتوقع: He يقلّل انكماش التباين ويمنع انفجار التدرّج الناتج عن عدم توازن `fan_in` / `fan_out` عبر الطبقات المتفاوتة العرض.)

10. **تصميم:** حدّد `fan_in` و`fan_out` لنواة `Conv2D(filters=64, kernel_size=3)` على صورة رمادية (`C_in=1`) ثم على صورة ملوّنة (`C_in=3`). (المتوقع: للرمادية $fan_{in} = 9$ و$fan_{out} = 192$؛ للملوّنة $fan_{in} = 27$ و$fan_{out} = 192$.)

11. **تحليل:** لماذا يبقى `RandomNormal(stddev=0.05)` معقولًا في `Embedding` بينما يكون خطرًا في `Dense` كبير؟ (المتوقع: في `Embedding` الأبعاد مستقلّة عن حجم vocabulary ومظهرها كمتجه واحد، بينما في `Dense` يتراكب n_{in} عنصرًا مستقلًّا فتكون $\mathrm{Var}(z) = n_{in} \cdot \sigma^2$ كبيرة وتُنتج انفجارًا.)

12. **برمجة:** اجعل الشبكة كاملة قابلة لإعادة الإنتاج: اكتب سطرًا واحدًا يضمن تطابق الأوزان بعد `fit` بين تشغيلين. (المتوقع: `keras.utils.set_random_seed(1234)` قبل إنشاء النموذج، مع `seed` ثابت لكل مُهيّئ غير افتراضي.)

ملخص الفصل

- التهيئة تحدّد **هندسة** الشبكة قبل أن يبدأ أي تدرّج: التباين الخاطئ في الطبقة الأولى ينتشر ويتضاعف عبر بقية الطبقات.
- معادلتان تحكمان التهيئة المعقولة: حدّ Glorot $a = \sqrt{6/(n_{in} + n_{out})}$، وتباين He $\mathrm{Var}(W) = 2/n_{in}$؛ والأخير يضاعف الأول لأن ReLU يُسقط نصف الوحدات.
- كل التهيئات الاحتمالية في Keras 3 غلافات حول `VarianceScaling`، ولذلك يكفي فهم `scale` و `mode` و `distribution` لفهمها جميعًا.
- المعامل 0.8796 في السحب المبتور يعيد التباين المفقود بالبتر؛ لذلك الانحراف المقيس يساوي نظريًّا.
- `Zeros` تصلح للحيّز ولا تصلح للأوزان، لأن التهيئة الصفرية تكسر التناظر وتُجمّد الشبكة.
- `activity_regularizer` و `kernel_regularizer` و `kernel_constraint` و `weight_decay` أربع آليات مختلفة لهندسة مختلفة: عقوبة على الخسارة، عقوبة على الناتج، إسقاط هندسي، وانكماش مباشر للوزن.
- البذرة الصحيحة `seed=` تعطي تهيئة متطابقة، و `keras.random.SeedGenerator` تعطي سحوبات مختلفة من حالة مشتركة، و `keras.utils.set_random_seed()` هي وحدها التي تضمن إعادة إنتاج كاملة.
- الافتراضيات في Keras (`glorot_uniform` للأوزان، `zeros` للحيّز) جيدة؛ غيّرها حين يكون لديك تشخيص: شبكة متكرّرة، عمق كبير، أو عدم استقرار عبر البذور.

الفصل 12: الـ Callbacks: أتمتة دورة التدريب

بطاقة الفصل

الفصل 12

مهمّة هذا الفصل

يؤتمت دورة التدريب: إيقاف مبكّر، حفظ أفضل نقطة، خفض المعدّل، تسجيل، ثم كتابة استدعاء مخصّص.

ماذا تكسب منه

توفّر ساعات حساب وتضمن أنك تحفظ أفضل نموذج لا آخر نموذج، وتوثّق كل تجربة تلقائيًّا.

الدالة	دورها	متى تستخدمها
`keras.callbacks.EarlyStopping`	إيقاف عند توقّف التحسّن	في كل تدريب أطول من بضع حقبات
`keras.callbacks.ModelCheckpoint`	حفظ أفضل نقطة على القرص	للتجارب الطويلة
`keras.callbacks.ReduceLROnPlateau`	خفض المعدّل عند الهضبة	عندما يثبت أداء التحقّق
`keras.callbacks.CSVLogger`	سجلّ نصّي لكل حقبة	للمقارنة بعد انتهاء التدريب
`keras.callbacks.Callback`	الصنف الأب لاستدعاء مخصّص	لأي سلوك تريد إضافته إلى الحلقة

قصة مفتاحية: كانت هناك سبع حقبات متبقّية في تجربة نموذج كشف الاحتيال، وكان فريق التعلّم العميق قد أغلق أجهزته وذهب لينام. استيقظوا على الإشعار بأن "الخسارة صارت `NaN` في الحقبة الرابعة" – لكن الملف الذي أعدّوه لتسلّمه للعملاء في الصباح لم يكن موجودًا أصلًا. الفكرة التي خلّصت الفريق لاحقًا: التدرّج الذي يمرّ به نموذجك خلال ساعات من التدريب هو **بيانات**، لا مجرّد أرقام تمرّ في الذاكرة. ولمن يقرأ هذه البيانات ويقرّر بناءً عليها، تحتاج آلية. هذه الآلية اسمها `Callback` .

أهداف الفصل

- [ ] استيعاب دورة حياة الـ `Callback` في Keras 3 وترتيب نداءات الخطّافات بدقّة، بما في ذلك مفارقة التحقّق الذي يستدعي خطّافات `on_test_*` .
- [ ] استخدام `EarlyStopping` بإتقان: اختيار `monitor` و `patience` و `min_delta` و `mode` و `baseline` و `start_from_epoch` و `restore_best_weights` .
- [ ] التمييز بين ملف `.keras` (نموذج كامل) وملف `.weights.h5` (أوزان فقط)، ومتى يكون كلٌّ منهما هو الأداة الصحيحة.
- [ ] ضبط `ReduceLROnPlateau` وفهم تعارضه مع جداول معدّل التعلّم الساكنة.
- [ ] قراءة مخرجات `TensorBoard` وربطها بما تعنيه عمليًّا.
- [ ] كتابة صنف `keras.callbacks.Callback` مخصّص، والتحكّم في معدّل التعلّم والحفظ من داخله.
- [ ] فهم ترتيب تنفيذ الاستدعاءات المتعدّدة، وإدارة حالة الـcallback وإعادة الإنتاج.

12.1 دورة حياة الـCallback: من `on_train_begin` إلى `on_train_end`

الـ `Callback` في Keras 3 ليس واجهة واحدة بل **شبكة خطّافات** (hooks) تُستدعى في مواضع محدّدة من دورة الحياة. القائمة الكاملة:

الخطّاف	متى يُستدعى	ماذا يحمل `logs`
`on_train_begin`	مرّة واحدة قبل أول حقبة	لا شيء
`on_epoch_begin`	قبل بدء كل حقبة	لا شيء (فارغ عمليًّا)
`on_train_batch_begin`	قبل كل دفعة تدريب	لا شيء
`on_train_batch_end`	بعد كل دفعة تدريب	مقاييس مجمَّعة حتى هذه اللحظة
`on_test_begin`	قبل التحقّق (أو `evaluate`)	لا شيء
`on_test_batch_begin` / `on_test_batch_end`	قبل/بعد كل دفعة تحقّق	مقاييس مجمَّعة
`on_test_end`	بعد انتهاء التحقّق	ناتج آخر دفعة
`on_epoch_end`	**بعد** الحقبة وبعد التحقّق	`loss`، `accuracy`، `val_loss`، `val_accuracy`…
`on_train_end`	بعد آخر حقبة	ناتج `on_epoch_end` الأخير
`on_predict_begin` / `on_predict_batch_begin` / `on_predict_batch_end` / `on_predict_end`	في `predict()`	حسب السياق

الترتيب الفعلي — وهو أدقّ ممّا يبدو

هذا هو التسلسل الذي ينفّذه حلقة التدريب في Keras 3.15.1 (تحقّقنا منه عمليًّا):

```text
on_train_begin
└── لكل حقبة:
    ├── on_epoch_begin(epoch)
    │   └── لكل دفعة: on_train_batch_begin(b) → on_train_batch_end(b)
    ├── on_test_begin → on_test_batch_begin/end → on_test_end   ← التحقّق داخل fit
    └── on_epoch_end(epoch, logs)      ← تحوي val_* لأن التحقّق انتهى قبلها
on_train_end
```



شكل 19 – الخط الزمني لاستدعاءات Callbacks خلال دورة التدريب

ثلاث ملاحظات مهمّة هنا:

1. **التحقّق داخل `fit` يستدعي خطّافات `on_test_*`، لا `on_epoch_*`.** هذا وقائع بصريّ: حين تمرّ `on_test_begin` أثناء التدريب فأنت في مرحلة تحقّق لا في مرحلة حقبة جديدة.
2. **`on_epoch_end` يُستدعى بعد التحقّق.** لهذا يحتوي `logs` على `val_loss` و`val_accuracy` إلى جانب `loss` و`accuracy`. لو استُدعي قبل التحقّق لما وجد `val_*` أصلًا.

3. `on_train_end` **هو الأخير دائمًا**، وهو المكان الذي تستعيد فيه `EarlyStopping` أفضل الأوزان.

> **ملاحظة:** `logs` في `on_epoch_begin` **فارغ**. كثير من المبتدئين يكتبون `logs["loss"]` هناك فيتسبّب `KeyError`. القيمة لا تُعرف إلا في `on_epoch_end`.

كيف تصل إلى النموذج؟ `self.model`

في Keras 3، `self.model` في داخل الـcallback هو **خاصية** (property) تُعيد النموذج الذي أرفقته بك. الخاصية ليست مجرّد اختصار: هي تتولّى فكّ غلاف `DistributedDataParallel` في PyTorch، وفي واجهة JAX تُجبر مزامنة حالة النموذج (`jax_state_sync`) قبل أن تقرأ أي وزن – وهو ما يمنع قراءة أوزان قديمة في وسط الحقبة.

التقليدان القديمان `set_model(model)` و`set_params(params)` ما زالا موجودين في Keras 3.15.1 كأنابيب داخلية تستدعيهما `CallbackList` على كل استدعاء، لكن عقدك مع الكود هو ببساطة: **اقرأ `self.model`، ولا تستدعِ `set_model` بنفسك أبدًا**. الاستدعاء اليدوي يستبدل مرجع النموذج داخل الـcallback بلا سبب مشروع، ويكسر ما هو مخصّص لـTensorBoard و`ModelCheckpoint`.

مثال أول: تتبع بسيط

PYTHON

```python
import numpy as np
import keras

# نجمع سجلًا لكل دفعة، لنرى متى تُستدعى الخطّافات فعليًّا
class BatchLogger(keras.callbacks.Callback):
    def __init__(self):
        super().__init__()
        self.records = []

    def on_train_batch_end(self, batch, logs=None):
        # logs هنا يحمل المقاييس المجمَّعة حتى الدفعة الحالية
        self.records.append((batch, float(logs["loss"])))
        if batch == 0:
            print("الخسارة :0 دفعة =", float(logs["loss"]))


model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse")

x = np.random.default_rng(0).standard_normal((64, 8)).astype("float32")
y = (x[:, 0] * 2.0).astype("float32")

logger = BatchLogger()
history = model.fit(x, y, epochs=2, batch_size=16, callbacks=[logger],
verbose=0)

print("عدد الدفعات المسجّلة =", len(logger.records))
print("مفاتيح السجل =", sorted(history.history.keys()))
# النتيجة المتوقعة: عدد الدفعات المسجّلة = 8
# النتيجة المتوقعة: مفاتيح السجل = ['loss']
```

12.2 `EarlyStopping`: متى تتوقّف، ولماذا يهمّ استعادة أفضل الأوزان

منطق التحسين

في كل حقبة، يقارن `EarlyStopping` القيمة المراقَبة `$v_t$` بأفضل قيمة وصل إليها `$v_{best}$`. الحقبة تُعدّ **تحسينًا** إذا تجاوزت الفارق حدًّا معيّنًا. في وضع `min` (وهو الافتراضي لكل مقاييس الخسارة):

$$v_t < v_{best} - \delta$$

حيث v_t القيمة المراقَبة في الحقبة الحالية، وv_{best} أفضل قيمة سابقة، وδ هو `min_delta` (افتراضيًا 0). والعدّاد `wait` يزيد واحدًا في كل حقبة غير محسِّنة، ويُصفَّر إلى الصفر عند كل تحسين. يتوقّف التدريب عندما:

$$wait \geq patience$$

حيث `patience` عدد الحقبات المتتالية غير المحسِّنة المسموح بها. القيمة الافتراضية `patience=0` تعني عمليًّا **توقّفًا عند أول حقبة بلا تحسين** – وهي قيمة قاسية جدًا، وقليل جدًا ما تكون مقصودة.

الوضع `auto` يُحلّ من اتجاه المقياس نفسه

لا تخمّن `mode`: تمرير `mode="auto"` (الافتراضي) يجعل Keras يستنتج الاتجاه. في Keras 3.15.1 يعمل الاستنتاج كالتالي: إن كان اسم المقياس بعد تجريد بادئة `val_` هو `loss` تمامًا، فالاتجاه هو `min`. وإلا فإن Keras يبحث عن مقياس بنفس الاسم في `model.metrics` ويقرأ اتّجاهه الداخلي (`_direction`: `"up"` يعني `max` (مثل `accuracy` و `auc`)، وأي شيء آخر يعني `min`). إن تعذّر الاستنتاج، يرفع Keras خطأً صريحًا يطلب منك تمرير `mode="max"` أو `mode="min"` يدويًّا.

> **نصيحة:** مقياس `accuracy` يُراقَب في وضع `max` تلقائيًّا، لكن `loss` لا يمكن تمييزه عن `mae` بالاسم وحده. فإذا كان اسم مقياسك `val_error` مثلًا، مرّر `mode="min"` صراحةً بدل الاعتماد على الاستنتاج.

`restore_best_weights=True`: لماذا لا يستغني عنه أحد

بدون هذا الخيار، النموذج الذي بين يديك بعد التوقّف هو أوزان **آخر** حقبة، لا أفضل حقبة. وبما أن التوقّف يحدث بالضبط بعد انحسار الأداء، فآخر حقبة هي غالبًا الأسوأ قليلًا — لا-good بالضرورة، لكن أسوأ من الأفضل. مع `restore_best_weights=True` يلتقط Keras نسخة الأوزان عند كل تحسّن (`self.model.get_weights()`)، ثم يستعيدها في `on_train_end` عبر `self.model.set_weights(...)`.

الفرق عمليّ: في نموذج بسقف 92% دقّة، قد تكون الفجوة بين آخر حقبة وأفضل حقبة نصف نقطة مئوية — وهي فجوة تكفي لتفشل في تقييم الإنتاج.

كيف تختار `patience`؟

القاعدة العملية: `patience` **بين 10% و25% من ميزانية الحقبات، وبحدّ أدنى 3**. الميزانية هنا هي عدد الحقبات الكليّ المخطّط له (`epochs`).

- ميزانية 20 حقبة ← `patience=3` (15%).
- ميزانية 100 حقبة ← `patience=10` (10%).
- ميزانية 500 حقبة (تدريب على بيانات ضخمة) ← `patience=25` إلى `50`.

القاعدة الثانية: `patience` لا يعني "كم حقبة أنت مستعدّ للانتظار"، بل "كم حقبة متتالية بلا تحسّن تُعتبر دليلًا على بلوغ plateau". إذا كانت منحنى الخسارة لديك شديد التذبذب (ن batch صغير)، فـ `patience` صغير يُوقفك مبكّرًا جدًا.

`start_from_epoch` و `baseline`

`start_from_epoch` تسمح بتجاهل الحقبات الأولى: حين تعرف أنّ الشبكة تحتاج 5 حقبات لتستقرّ (مثلًا مع جدول إحماء `warmup`)، فمرّر `start_from_epoch=5` فلا يُراقب الأداء قبل ذلك ولا

يُحسب ضد صبر الـcallback.

`baseline` هو عتبة تُقارَن بها النتائج: مرّر `baseline=0.85` لمقياس دقّة، فيتوقّف التدريب إذا لم يتجاوز النموذج هذه القيمة. الشرط أدقّ من مجرّد "لم تتحسّن": يجب أن يتجاوز التحسين **الأساس** أيضًا، وإلا فلا يُصفَّر عدّاد `wait` .

PYTHON

```python
import numpy as np
import keras

# بذرة ثابتة تجعل النتيجة قابلة للتكرار حرفيًّا
keras.utils.set_random_seed(0)

model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.01),
    loss="mse",
    metrics=["mae"],
)

# بيانات لها حدّأدنى للخطأ (ضوضاء غير قابلة للاختزال): النموذج سيبلغها ثم يتذبذب
rng = np.random.default_rng(0)
x = rng.standard_normal((128, 8)).astype("float32")
y = (x[:, 0] * 2.0 + rng.normal(0, 1.0, size=128)).astype("float32")

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss",      # ما الذي نراقبه
    patience=2,              # كم حقبة بلا تحسّن نتحمّلها
    min_delta=1e-4,          # أقل تحسّن يُعدّ تحسّنًا
    mode="min",              # تصريحٌ بالاتجاه: لا تعتمد على الاستنتاج
    restore_best_weights=True,  # أعد أفضل الأوزان لا آخرها
    start_from_epoch=2,      # تجاهل الحقبتين الأولى
    verbose=1,
)

history = model.fit(
    x, y,
    epochs=12,
    batch_size=16,
    validation_split=0.2,
    callbacks=[early_stop],
```

```
    verbose=0,
)
print("عدد الحقبات المنفَّذة =", len(history.history["loss"]))
# النتيجة المتوقعة: عدد الحقبات المنفَّذة = 11
# النتيجة المتوقعة (رسالة Keras): Epoch 11: early stopping
# النتيجة المتوقعة (رسالة Keras): Restoring model weights from the end of the
best epoch: 9.
```

12.3 `ModelCheckpoint`: نقطة الحفظ، وأي ملفٍّ تحفظ

`ModelCheckpoint` يحفظ النموذج تلقائيًّا أثناء التدريب. المعاملات الأساسية:

- `filepath`: مسار الحفظ. يدعم قوالب تنسيق: `"{epoch:02d}-{val_loss:.4f}.keras"`.
- `monitor`: القيمة التي يُقارَن بها الحفظ (`"val_loss"` افتراضيًّا).
- `save_best_only=True`: لا تحفظ إلّا عند تحسّن القيمة المراقَبة.
- `save_weights_only=True`: احفظ الأوزان فقط (يلزم أن ينتهي المسار بـ `.weights.h5`).
- `mode`: `"auto"` أو `"min"` أو `"max"` – نفس منطق `EarlyStopping`.
- `save_freq`: `"epoch"` (افتراضيًّا) أو عدد صحيح يعني "احفظ كلّ N دفعة".

`.keras` مقابل `.weights.h5`: فرق جوهري

المعيار	ملف `.keras`	ملف `.weights.h5`
ماذا يحتوي	البنية + الأوزان + حالة المُحسّن + المقاييس	الأوزان فقط
كيف يُحمَّل	`keras.saving.load_model("m.keras")`	`model.load_weights("m.weights.h5")`
هل يحتاج بنية معروفة مسبقًا؟	لا	نعم، يجب بناء النموذج أولًا
الحجم	أكبر	أصغر بكثير
شرط Keras على المسار	ينتهي بـ `.keras` أو `.h5`	**يجب** أن ينتهي بـ `.weights.h5`
الاستخدام النموذجي	تسليم نموذج جاهز للإنتاج	الضبط الدقيق (fine-tuning) ونقل المعرفة

القاعدة الحاسمة التي يفرضها Keras 3: إذا مرّرت `save_weights_only=True` ومسارًا ينتهي بـ `.keras`، فـKeras **يرفع** `ValueError`. والعكس صحيح: مع `save_weights_only=False` يجب أن ينتهي المسار بـ `.keras` أو `.h5`.

PYTHON

```python
import numpy as np
import keras
import tempfile
import os

model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse")

x = np.random.default_rng(0).standard_normal((64, 8)).astype("float32")
y = (x[:, 0] * 2.0).astype("float32")

tmpdir = tempfile.mkdtemp()

# 1) حفظ النموذج كاملًا (أرشيف .keras)
full_path = os.path.join(tmpdir, "model.keras")
ckpt_full = keras.callbacks.ModelCheckpoint(
    filepath=full_path,
    monitor="val_loss",
    save_best_only=True,
)
model.fit(x, y, epochs=3, batch_size=16, validation_split=0.2,
          callbacks=[ckpt_full], verbose=0)
print("حجم ملف .keras:", os.path.getsize(full_path), "بايت")
# النتيجة المتوقعة: حجم ملف .keras أكبر من حجم ملف .weights.h5

# 2) حفظ الأوزان فقط (.weights.h5)
weights_path = os.path.join(tmpdir, "model.weights.h5")
ckpt_weights = keras.callbacks.ModelCheckpoint(
    filepath=weights_path,
    monitor="val_loss",
    save_best_only=True,
    save_weights_only=True,
)
model.fit(x, y, epochs=3, batch_size=16, validation_split=0.2,
```

```
          callbacks=[ckpt_weights], verbose=0)
print("حجم ملف .weights.h5:", os.path.getsize(weights_path), "بايت")
# النتيجة المتوقعة: حجم ملف .weights.h5 أصغر من ملف .keras

# 3) إعادة تحميل النموذج كاملًا بلا إعادة بناء البنية
reloaded = keras.saving.load_model(full_path)
print("أبعاد المخرجات:", reloaded.output_shape)
# النتيجة المتوقعة: أبعاد المخرجات: (None, 1)

# 4) إعادة تحميل الأوزان فقط: تتطلّب بنية مطابقة مسبقًا
fresh = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
fresh.load_weights(weights_path)
print("تم تحميل الأوزان فقط بنجاح")
# النتيجة المتوقعة: تم تحميل الأوزان فقط بنجاح
```

12.4 `ReduceLROnPlateau` وجداول معدّل التعلّم

خفض معدّل التعلّم عند الهضبة

حين يتوقّف تحسّن `val_loss`، فإن أفضل ردّ فعل هو **تقليص معدّل التعلّم** بدل التوقّف. الخوارزمية بسيطة:

$$lr_{new} = \max\left(lr \cdot f,\ lr_{min}\right)$$

حيث lr معدّل التعلّم الحالي، وf معامل التقليص (`factor`، افتراضيًا 0.1)، وlr_{min} الحدّ الأدنى (`min_lr`، افتراضيًا 0.0).

معاملات أخرى مهمّة: `patience` (كم حقبة بلا تحسّن قبل التقليص، افتراضيًا 10 – وهي هنا أوسع من صبر `EarlyStopping`)، و`cooldown` (عدد الحقبات التي تتوقّف فيها المراقبة بعد كل

تخفيض، لمنع التخفيض المتكرّر)، و `mode` (نفس منطق الاتجاه).

التعارض مع الجداول الساكنة — تحذير جوهري

حين تمرّر `optimizer=keras.optimizers.Adam(learning_rate=keras.optimizers.schedules.ExponentialDecay(...))` و `ReduceLROnPlateau` معًا، تنشأ مسابقة: الجدول الساكن يعيد حساب معدّل التعلّم من `step` في كل خطوة، بينما `ReduceLROnPlateau` يكتب قيمة جديدة في `optimizer.learning_rate` . النتيجة أن كتابة `ReduceLROnPlateau` تُلغى عمليًّا في الخطوة التالية، أو تنتهي بتخفيضات متضاربة.

القاعدة: اختر أحدهما لا كلاهما. استخدم الجدول الساكن حين يكون مسار التحسّن واضحًا، واستخدم `ReduceLROnPlateau` حين يكون تحسين الأداء متقطّعًا وغير متوقّع.

`LearningRateScheduler` : الجدولة من داخل الحلقة

`LearningRateScheduler` هو المكمل النظيف: دالة `schedule(epoch, lr)` تعيد معدّل التعلّم الجديد في بداية كل حقبة. لاحظ أن الدالة في Keras 3 تستقبل وسيطين (رقم الحقبة ومعدّل التعلّم الحالي)، مع دعم تلقائيّ للتوقيع القديم `schedule(epoch)` فقط.

PYTHON

```python
import numpy as np
import keras

keras.utils.set_random_seed(0)

model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),
loss="mse")

x = np.random.default_rng(0).standard_normal((64, 8)).astype("float32")
y = (x[:, 0] * 2.0).astype("float32")

# يعيد الحساب كل حقبة callback معدّل التعلّم الأساسي: نحتفظ به في ثابت لأن الـ
BASE_LR = 1e-3
WARMUP_EPOCHS = 5


def warmup_schedule(epoch, lr):
    """إحماء خطّي تصاعدي في أول 5 حقبات ثم تخفيض أُسّي بمعامل 0.7."""
    if epoch < WARMUP_EPOCHS:
        # تصاعد خطّي من base_lr/WARMUP_EPOCHS حتى base_lr
        return BASE_LR * (epoch + 1) / WARMUP_EPOCHS
    # بعد الإحماء: نضرب في كل حقبة فتتراكم بنسبة 0.7 ^epoch
    return lr * 0.7


scheduler = keras.callbacks.LearningRateScheduler(warmup_schedule,
verbose=1)
history = model.fit(x, y, epochs=10, batch_size=16,
                    callbacks=[scheduler], verbose=0)
print("سجل معدّل التعلّم =", [round(v, 7) for v in
history.history["learning_rate"]])
# النتيجة المتوقعة:
```

```
# [0.0002, 0.0004, 0.0006, 0.0008, 0.001, 0.0007, 0.00049, 0.000343,
0.0002401, 0.0001681]
```

لاحظ أن `LearningRateScheduler` يضيف مفتاح `"learning_rate"` إلى `logs` في `on_epoch_end`، فتظهر سلسلة معدّل التعلّم كاملةً في `history.history`. ولاحظ أيضًا **أين** يُطبَّق الجدول: في `on_epoch_begin` لا في `on_epoch_end` – أي أن معدّل التعلّم المستخدم في الحقبة t هو ناتج $\mathit{schedule}(t-1)$، لا $\mathit{schedule}(t)$. وهذا سبب شائع للـ off-by-one في الجداول المكتوبة يدويًّا.

بقية الأدوات المدمجة

الأداة	وظيفتها	ملاحظات
`History`	يسجّل المقاييس في `model.history.history`	يُضاف تلقائيًّا في `fit`
`ProgbarLogger`	شريط التقدّم في الطرفية	يُضاف تلقائيًّا في `fit`
`CSVLogger(filename, separator=',', append=False)`	يكتب السجل إلى ملف CSV	`append=True` يضيف إلى الموجود بدل استبداله
`TerminateOnNaN(raise_error=False)`	يوقف التدريب فور ظهور `NaN` في الخسارة	`raise_error=True` يرفع استثناء بدل التوقّف الهادئ
`LambdaCallback(on_epoch_begin=..., on_epoch_end=...)`	لاصق سريع دون وراثة صنف	للسلوك البسيط فقط
`BackupAndRestore(backup_dir, save_freq='epoch', double_checkpoint=False, delete_checkpoint=True)`	يحفظ حالة النموذج والمُحسّن ليُستأنف التدريب بعد انقطاع	`save_freq` يقبل `'epoch'` أو عدد دفعات

`TensorBoard`: كيف تقرأ ما سجّله

`TensorBoard` يكتب سجلّات التدريب إلى `log_dir` (افتراضيًا `logs`):

> **تنبيه:** في Keras 3.15.1 يتطلّب `keras.callbacks.TensorBoard` **تثبيت TensorFlow**، لأنّ الكتابة تتمّ عبر `tensorflow.summary` وليس عبر واجهة مستقلّة. إن كانت بيئة العمل لديك

تعمل بـJAX أو PyTorch وحدهما، فثبّت TensorFlow لتفعيل هذه الأداة (وحزمة `tensorboard` تأتي معه). بدون ذلك يرفع الـcallback خطأ `ModuleNotFoundError` عند إنشاء كاتب السجلّات.

PYTHON

```python
import numpy as np
import keras

model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])

x = np.random.default_rng(0).standard_normal((64, 8)).astype("float32")
y = (x[:, 0] * 2.0).astype("float32")

tb = keras.callbacks.TensorBoard(
    log_dir="./logs",           # مجلد السجلّات
    write_graph=True,           # ارسم رسم بياني للبنية
    histogram_freq=1,           # سجّل توزيع أوزان كل حقبة
    embeddings_freq=0,          # لا حاجة لتضمينات في هذا المثال
    update_freq="epoch",        # حدّث السجلّات في نهاية كل حقبة
)

model.fit(x, y, epochs=5, batch_size=16, validation_split=0.2,
          callbacks=[tb], verbose=0)
print("اكتمل الكتابة في ./logs")
```

لقراءتها: شغّل `tensorboard --logdir logs` وافتح `http://localhost:6006` (أداة القراءة `tensorboard` مثبَّتة ضمن TensorFlow). اللوحات التي تستحق المتابعة: منحنى `loss` / `val_loss` (فجوة بينهما تعني فرط تخصيص)، لوحة **Histograms** (توزّع الأوزان: إن انكمش مركز التوزيع انكمشت التعلّم)، ولوحة **Graph** (البنية).

> **ملاحظة:** `write_graph=True` صار معيارًا في Keras 3 والسجلّات تُكتب بصيغة TensorBoard مباشرة دون حاجة إلى `SummaryWriter` يدوي.

12.5 كتابة `Callback` مخصّص

عندما تخرج الحاجة عن حدود الأدوات المدمجة، ترث من `keras.callbacks.Callback` وتنفّذ ما تحتاجه من الخطّافات.

PYTHON

```python
import numpy as np
import keras

# بذرة ثابتة: بدونها يختلف مسار التدريب بين التشغيلات وتختلف الأرقام المطبوعة
keras.utils.set_random_seed(0)


class PlateauDetector(keras.callbacks.Callback):
    """يكشف الهضبة ويخفّض معدّل التعلّم، ويوقف التدريب بعد مهلة."""

    def __init__(self, patience=3, factor=0.5, min_lr=1e-6, max_wait=6):
        super().__init__()
        self.patience = patience
        self.factor = factor
        self.min_lr = min_lr
        self.max_wait = max_wait
        # أعد ضبط الحالة في on_train_begin لتصبح الحالة قابلة لإعادة الاستخدام
        self.best = None
        self.wait = 0
        self.plateau_count = 0

    def on_train_begin(self, logs=None):
        # تصفير الحالة عند كل تشغيل جديد
        self.best = None
        self.wait = 0
        self.plateau_count = 0

    def on_epoch_end(self, epoch, logs=None):
        val_loss = logs.get("val_loss")
        if val_loss is None:
            return  # لا يوجد تحقّق: لا معنى للحديث عن الهضبة

        if self.best is None or val_loss < self.best - 1e-4:
            # تحسّن: سجّل أفضل قيمة وأعد ضبط العدّادات
            self.best = val_loss
            self.wait = 0
        else:
            self.wait += 1
```

```
            if self.wait >= self.patience:
                # بلغنا الهضبة: خفّض معدّل التعلّم
                current_lr =
float(keras.ops.convert_to_numpy(self.model.optimizer.learning_rate))
                new_lr = max(current_lr * self.factor, self.min_lr)
                self.model.optimizer.learning_rate = new_lr
                self.plateau_count += 1
                self.wait = 0
                print(f"هضبة عند الحقبة {epoch}: lr {current_lr:.6f} →
{new_lr:.6f}")

                if self.plateau_count >= self.max_wait:
                    # فشلت كل المحاولات: أوقف التدريب
                    self.model.stop_training = True


model = keras.Sequential([
    keras.layers.Input(shape=(8,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1),
])
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.01),
loss="mse")

# بيانات ذات خطأ غير قابل للاختزال: النموذج يبلغ الحدّ الأدنى ثم يتذبذب حوله
rng = np.random.default_rng(0)
x = rng.standard_normal((128, 8)).astype("float32")
y = (x[:, 0] * 2.0 + rng.normal(0, 1.0, size=128)).astype("float32")

detector = PlateauDetector(patience=2, factor=0.5, max_wait=1)
history = model.fit(x, y, epochs=12, batch_size=16, validation_split=0.2,
                    callbacks=[detector], verbose=0)
print("عدد الحقبات المنفَّذة =", len(history.history["loss"]))
print("عدد مرّات اكتشاف الهضبة =", detector.plateau_count)
# النتيجة المتوقعة: عدد الحقبات المنفَّذة = 11 (توقّف بعد أول هضبة)
# النتيجة المتوقعة: عدد مرّات اكتشاف الهضبة = 1
```

جدولة معدّل التعلّم من داخل الـcallback

لاحظ السطرَين الحاسمين في المثال أعلاه:

```python
# شطر من المثال أعلاه: قراءة معدّل التعلّم الحالي ثم كتابته
current_lr =
float(keras.ops.convert_to_numpy(self.model.optimizer.learning_rate))
self.model.optimizer.learning_rate = new_lr
```

`self.model.optimizer.learning_rate` قد يكون رقمًا أو جدولًا (`LearningRateSchedule`). Assignment المباشر يمحو الجدول الساكن ويتركه رقمًا – وهو ما يعني أن `ReduceLROnPlateau` و`LearningRateScheduler` لا يصلحان معًا. تعامل مع ذلك بوعي: إن كان لديك جدول ساكن فلا تكتب فيه من الـcallback.

ترتيب الاستدعاءات وتفاعل الاستدعاءات

الاستدعاءات تُنفَّذ **بالترتيب الذي تعطيها به في القائمة**. هذه هي القاعدة العملية التي تحكم كل سلوك مركّب:

- ضع `ModelCheckpoint` **قبل** `EarlyStopping` في القائمة. السبب هو: `EarlyStopping` يستعيد أفضل الأوزان في `on_train_end`؛ فإن حُفظ الملف بعد الاستعادة يكون الملف أفضل من الأوزان التي كان النموذج يحملها أثناء التدريب. وإن حُفظ قبل الاستعادة، فإن الملف يحتوي أفضل أوزان محفوظة أصلًا – وهو السلوك المرغوب عادةً.
- ضع `ReduceLROnPlateau` **قبل** أي callback مخصّص يقرأ `optimizer.learning_rate` في `on_epoch_end`، وإلا قرأ القيمة قبل تخفيضها.
- عند تفعيل EarlyStopping، لا تتوقّف بقية الاستدعاءات عن العمل في تلك الحقبة: `CallbackList` يمرّ على كل الاستدعاءات ويُنجز `on_epoch_end` للجميع، ثم تتحقّق حلقة التدريب من `model.stop_training`. أي أن الـcallback الذي يأتي بعد `EarlyStopping` في القائمة **سيُستدعى فعلًا** في الحقبة الأخيرة.
- `History` و`ProgbarLogger` يُضافان تلقائيًّا إلى القائمة إن لم يكونا موجودين.

الحالة وإعادة الإنتاج

كل الـcallbacks كائنات **ذات حالة**: `EarlyStopping` يحمل `wait` و`best`، و`ReduceLROnPlateau` يحمل `best` و`wait`، والـcallback المخصّص أعلاه يحمل `best` و`plateau_count`. عند إعادة استخدام نفس النسخة في تدريب جديد، يجب تصفير الحالة. `EarlyStopping` و`ReduceLROnPlateau` يفعلان ذلك تلقائيًّا في `on_train_begin` — من هنا جاءت الجملة `self.best = None` في المثال أعلاه: **أنت مسؤولٌ عن نسختك**.

أمّا **إعادة الإنتاج** فتتطلّب ثلاثة أشياء منفصلة: بذرة عشوائية ثابتة (`keras.utils.set_random_seed`)، و`shuffle` حتميّ، وعدم قراءة من مصادر متغيّرة داخل الـcallback (مثل `datetime.now()` في اسم ملف). اجعل أي حالة عشوائية في الـcallback مشتقّة من `epoch` و`batch` لا من مولّد عشوائي عام.

تطبيق واقعي: خطّ إنتاج تدريب لا يُترك للمصادفة

في فريق طبّ يشخّص اعتلال الشبكية من صور قاع العين، كان التدريب يجري لثلاث ليالٍ متواصلة على خوادم مشتركة. كان كل شيء يدويًّا: سكرتير يراقب لوحة TensorBoard، ويكتب رقم الحقبة في جدول Excel، ويحكم بنفسه متى تكون الخسارة "ثابتة" فيوقف التشغيل. النتيجة أن نسختين من النموذج بقيا في الخادم دون أيّ أثر يوضّح أيّهما أفضل، وأسبوعًا كاملًا ضاع في إعادة التدريب.

استبدل الفريق المراقبة اليدوية بمكدّس استدعاءات واحد:

```python
callbacks = [
    keras.callbacks.TensorBoard(log_dir="./logs", histogram_freq=1),
    keras.callbacks.CSVLogger("training_log.csv", append=True),
    keras.callbacks.ModelCheckpoint(
        filepath="best.keras", monitor="val_loss", save_best_only=True
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss", factor=0.5, patience=4, min_lr=1e-5
    ),
    keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=6, restore_best_weights=True
    ),
]
```

الفرق الجوهري ليس في عدد السجلّات، بل في أن **قرار الإيقاف صار جزءًا من الشيفرة**: كل نتيجة قابلة للتكرار، وكل نموذج محفوظ يحمل اسمًا ومسارًا محدّدين، ولا وجود لتقدير بشريّ في حلقة. والأهمّ: `CSVLogger` مع `append=True` بنى سجلًّا تراكميًّا عبر التجارب المتتالية، فأصبح بالإمكان مقارنة عشرة تجارب في جدول واحد. ومن المخرجات ذلك `val_loss` كان they'd learned that يهبط تحت 0.18 ثم يبدأ بالارتفاع عند الحقبة 14 دائمًا – أي أنّ النموذج كان يت.overfit ابتداءً من الحقبة الثامنة، وأنّ `start_from_epoch=8` كان سيوفّر نصف وقت كل تجربة.

الدرس العام: الاستدعاءات ليست زينة تُضاف في نهاية المشروع، بل هي **طبقة قرار** فوق حلقة التدريب، وهي الطبقة التي تجعل تجربة تعلّم عميق قابلة للتكرار والمراجعة والتشغيل الآلي.

12.6 أخطاء شائعة

إنذار 1: قراءة `logs["loss"]` في `on_epoch_begin`. `logs` فارغ هناك. اقرأ المقاييس في `on_epoch_end` فقط.

إنذار 2: `monitor="val_loss"` بلا بيانات تحقّق. إن لم تمرّر `validation_split` ولا `validation_data`، سيرفع Keras تحذيرًا ويتجاهل الدالة. الإصلاح: أضف `validation_split=0.2` أو بدّل `monitor="loss"`.

إنذار 3: `ModelCheckpoint` بـ `save_weights_only=True` ومسار ينتهي بـ `.keras`. Keras يرفع `ValueError`. القاعدة: `save_weights_only=True` ⇔ المسار ينتهي بـ `.weights.h5`.

إنذار 4: `ReduceLROnPlateau` مع جدول تعلّم ساكن معًا. يكتب كلٌّ منهما فوق الآخر. اختر واحدًا.

إنذار 5: توقّع "أفضل نموذج" من `EarlyStopping` بلا `restore_best_weights=True`. ستحصل على آخر حقبة. اجعل `restore_best_weights=True` دائمًا إلا إذا كانت لديك سبب محدّد.

إنذار 6: `patience=0`. القيمة الافتراضية تعني "توقّف عند أول حقبة بلا تحسّن". إن كتبت `EarlyStopping()` بلا معاملات فأنت تطلب توقّفًا فوريًّا تقريبًا.

إنذار 7: تعديل أوزان النموذج داخل `on_train_batch_end`. حين تكتب في `self.model` وأنت داخل حلقة التدريب، تكسر ما يفعله المُحسّن من تجميع التدرّجات (مثلاً `ema` في AdamW أو عدّاد الخطوات). عدّل في `on_epoch_end` أو في `on_train_begin` حيث الحالة نظيفة.

إنذار 8: إعادة استخدام نسخة callback دون تصفير حالتها. النتيجة سلوك غير متوقّع في التشغيل الثاني: `best` القديمة تمنع أي تحسين في القياس الأول.

التمارين

1. **برمجة:** اكتب callback يوقف التدريب عند بلوغ `val_loss` حدًّا ثابتًا (`target=0.05`). (المتوقع: يتوقّف التدريب عند أول حقبة تصبح فيها `val_loss ⩽ 0.05` ، وتظهر رسالة تذكر الحقبة.)

2. **برمجة:** اكتب callback يجدول معدّل التعلّم بحيث يكون `1/sqrt(epoch+1)` داخل الـcallback نفسه، لا عبر `LearningRateScheduler` . (المتوقع: `self.model.optimizer.learning_rate` يساوي $0.01/\sqrt{epoch+1}$ في بداية كل حقبة.)

3. **حفظ واستئناف:** احفظ نموذجًا كاملًا بـ `ModelCheckpoint` إلى `m.keras` ، أعد بناء النموذج، ثم استأنف التدريب من نقطة الحفظ. (المتوقع: الملف يُحمَّل بـ `keras.saving.load_model` دون إعادة بناء البنية، ومحتواه يشمل حالة المُحسّن إن حُفظ بعد `compile` .)

4. **تنبّؤ بالترتيب:** أعطِ قائمة `callbacks=[EarlyStopping, ModelCheckpoint, MyLogger]` وحدّد ترتيب استدعاء `on_epoch_end` لها في الحقبة الأخيرة التي يقع فيها التوقّف. (المتوقع: يُستدعى الثلاثة بالترتيب المذكور داخل الحقبة نفسها؛ ثم يمرّر `CallbackList` الباقي، ثم تتحقّق حلقة التدريب من `stop_training` .)

5. **اكتشاف خطأ:** هذا الـcallback فيه خطأ منطقي:

```python
import keras

class BadStopper(keras.callbacks.Callback):
    def on_epoch_begin(self, epoch, logs=None):
        if logs["val_loss"] > 0.5:
            self.model.stop_training = True
```

اذكر الخطأين وأصلحهما. (المتوقع: أولًا `logs` فارغ في `on_epoch_begin` فيرفع `KeyError` ؛ ثانيًا القيمة المراقَبة لا تُعرف إلا في `on_epoch_end` ، فالحل هو نقل الشرط إلى `on_epoch_end` .)

6. **تصميم:** ميزانية تدريب 60 حقبة. ما قيمة `patience` التي تختارها لـ `EarlyStopping` ولماذا؟ (المتوقع: بين 6 و15، أي 10%-25% من الميزانية، وبحدّ أدنى 3؛ القيمة 60 نفسها تعني عمليًّا "لا توقّف تلقائي".) 7. **تحليل:** ما الفرق العمليّ بين `model.save("m.keras")` و `model.save_weights("m.weights.h5")` ؟ ومتى تختار كلًّا منهما؟ (المتوقع: الأول يُنتج نموذجًا جاهزًا للتحميل بـ `load_model` ويحوي البنية والأوزان وحالة المُحسّن؛ الثاني يحوي الأوزان فقط ويشترط بنية مطابقة مسبقًا، وهو الأنسب للضبط الدقيق وتبديل البنية.) 8. **برمجة:** اكتب callback يسجّل في كل حقبة الزمن المنقضي باستخدام `time.perf_counter()` ويكتبه إلى ملف CSV. (المتوقع: ملف يحتوي عمودًا بالزمن لكل حقبة، ومجموعه مساوٍ تقريبًا لزمن التدريب الكلّي.) 9. **تشخيص:** `ReduceLROnPlateau` لا يخفّض معدّل التعلّم أبدًا في تجربتك. اذكر ثلاثة أسباب محتملة مرتّبة بالأرجحية. (المتوقع: وجود جدول تعلّم ساكن يكتب فوقه؛ أو `min_lr` أعلى من المعدّل الحالي؛ أو `mode` خاطئ فيقارن الخسارة كأنها دقّة.) 10. **تصميم:** صمّم تسلسل استدعاءات لمشروع فيه: `EarlyStopping` و `ModelCheckpoint` و `TensorBoard` وcallback مخصّص يعدّل LearningRate. في أي ترتيب تضعها ولماذا؟ (المتوقع: `ModelCheckpoint` أولًا ليحفظ أفضل الأوزان قبل استعادة `EarlyStopping` ، ثم `TensorBoard` ، ثم `ReduceLROnPlateau` ، ثم الـcallback المخصّص ليقرأ آخر قيمة، و `EarlyStopping` أخيرًا لأن دوره هو القرار النهائي.) 11. **تحليل:** لماذا يُستدعى `on_epoch_end` **بعد** التحقّق لا قبله، وماذا يعني ذلك لمن يريد مراقبة `val_loss` ؟ (المتوقع: لأنه يسمح بدمج نتائج التحقّق في `logs` تحت مفاتيح `val_*` ، ويمنح monitor_callbacks قيمةَ تحقّق نهائية لكل حقبة.) 12. **إصلاح خطأ:** أعطِ الكود التالي الذي يهدف إلى حفظ أفضل نموذج:

```python
import keras

ckpt = keras.callbacks.ModelCheckpoint(
    filepath="best.weights.h5", save_best_only=True, save_weights_only=True
)
```

وأعطِ الخطأ والنسخة الصحيحة. (المتوقع: Keras يرفع `ValueError` لأن `save_weights_only=True` يفرض أن ينتهي المسار بـ `.weights.h5` ؛ التصحيح إمّا تغيير المسار إلى `best.weights.h5` أو جعل `save_weights_only=False` .)

ملخص الفصل

- الـ `Callback` هو خطّافات تُستدعى في مواضع محدّدة: `on_train_begin`، `on_epoch_end`، `on_test_*`، `on_train_batch_begin/end`، `on_epoch_begin`، `on_predict_*`، `on_train_end`.
- التحقّق داخل `fit` يُطلق خطّافات `on_test_*`، و`on_epoch_end` يأتي بعد التحقّق، فتتوفّر مفاتيح `val_*` في `logs`.
- `self.model` في Keras 3 خاصية تُعيد النموذج مع مراعاة DDP في PyTorch ومزامنة الحالة في JAX؛ لا تستدعِ `set_model` يدويًّا.
- `EarlyStopping` يجمع `monitor` و`patience` و`min_delta` و`mode` و`baseline` و`start_from_epoch` و`restore_best_weights`؛ و`restore_best_weights=True` هو ما يحوّل "توقّف مبكّر" إلى "نموذج أفضل".
- `patience` بين 10% و25% من ميزانية الحقبات، وبحدّ أدنى 3.
- ملف `.keras` نموذج كامل يُحمَّل بـ `load_model`، وملف `.weights.h5` أوزان فقط تتطلّب بنية مطابقة مسبقًا؛ Keras يفرض ذلك على امتداد المسار.
- `ReduceLROnPlateau` يخفّض المعدّل عند الهضبة بمعادلة $lr_{new} = \max(lr \cdot f, lr_{min})$، وهو **يتعارض** مع الجداول الساكنة.
- `LearningRateScheduler` يستدعي `schedule(epoch, lr)` في بداية كل حقبة ويضيف `learning_rate` إلى `logs`.
- الاستدعاءات تُنفَّذ بترتيب القائمة، و`History` و`ProgbarLogger` يُضافان تلقائيًّا.
- كل الـ callbacks ذات حالة: صِر حالتك في `on_train_begin` لتبقى قابلة لإعادة الاستخدام.

الفصل 13: التنظيم والقيود: كيف تمنع النموذج من الحفظ بدل التعلّم

بطاقة الفصل

الفصل 13

مهمّة هذا الفصل

يمنع النموذج من الحفظ بدل التعلّم: L1 وL2 وDropout وBatchNormalization والقيود.

ماذا تكسب منه

تقلّص الفجوة بين التدريب والتحقّق، فيصبح أداء النموذج في الإنتاج قريبًا مما تراه على شاشتك.

الدالة	دورها	متى تستخدمها
`keras.regularizers.L2(l2)`	عقوبة تربيعية على الأوزان	في الطبقات الكثيفة الكبيرة
`keras.regularizers.L1L2(l1, l2)`	مزيج يدفع بعض الأوزان إلى الصفر	عندما تريد انتقاء ميزات ضمنًا
`keras.layers.Dropout(rate)`	تنظيم عشوائي أثناء التدريب	في الطبقات الكثيفة ورؤوس التصنيف
`keras.layers.BatchNormalization()`	تطبيع الدفعة	في الشبكات العميقة لتسريع التقارب
`keras.layers.SpatialDropout2D()`	إسقاط قنوات كاملة	في الشبكات التلافيفية

قصة مفتاحية: طوّر فريق في شركة تأمين نموذجًا يميّز مطالبات fraudulent. دقّة التحقّق impeccable: 99.2%. لكن عند التطبيق على مطالبات watchdog.العام الماضي، انهار الأداء إلى 61%. السبب لم يكن في النموذج ولا في البيانات: الفريق كان قدّم لكل طالب مدرب **مليزًا بالآلاف من الحالات النادرة**، فحفظها عن ظهر قلب – تمامًا كما يحفظ الطالب الجادّ جدول معادلات قبل الاختبار. النموذج لم يتعلّم قاعدة القرار؛ حفظ الإجابات.

أهداف الفصل

- [ ] تشرح الفرق بين فرط التكيّف (overfitting) وفرط التحمّل (underfitting) بلغة التدرّج
- [ ] تربط بين سعة النموذج (capacity) وقُدرته على تعميم بيانات جديدة
- [ ] تكتب دالّة الخسارة مع بندَي L1 وL2 وتعرف متى يتفوّق كلٌّ منهما
- [ ] تفهم لماذا يعمل `Dropout` نظريًا ولماذا لا يكفي وحده
- [ ] تميّز بين `BatchNormalization` و`LayerNormalization` و`GroupNormalization` بحسب سياق الاستخدام
- [ ] تستعمل `keras.regularizers` و`keras.constraints` داخل الطبقات
- [ ] تقرأ مخطّطًا بيانيًا لتدريب وتحكم على الفرط بأدلة لا بالحدس

13.1 المشكلة: فرط التكيّف كنموذج فيزيائي

ما الذي يحدث أثناء تدريب شبكة عميقة؟ في كل خطوة، تُحدَّث الأوزان W لتُقلّل الخسارة على الدفعة الحالية. هذا هو **الملاءمة الزائدة (overfitting)** بعينها: النظام يبدأ في مطابقة **الضجيج** لا **الإشارة**.

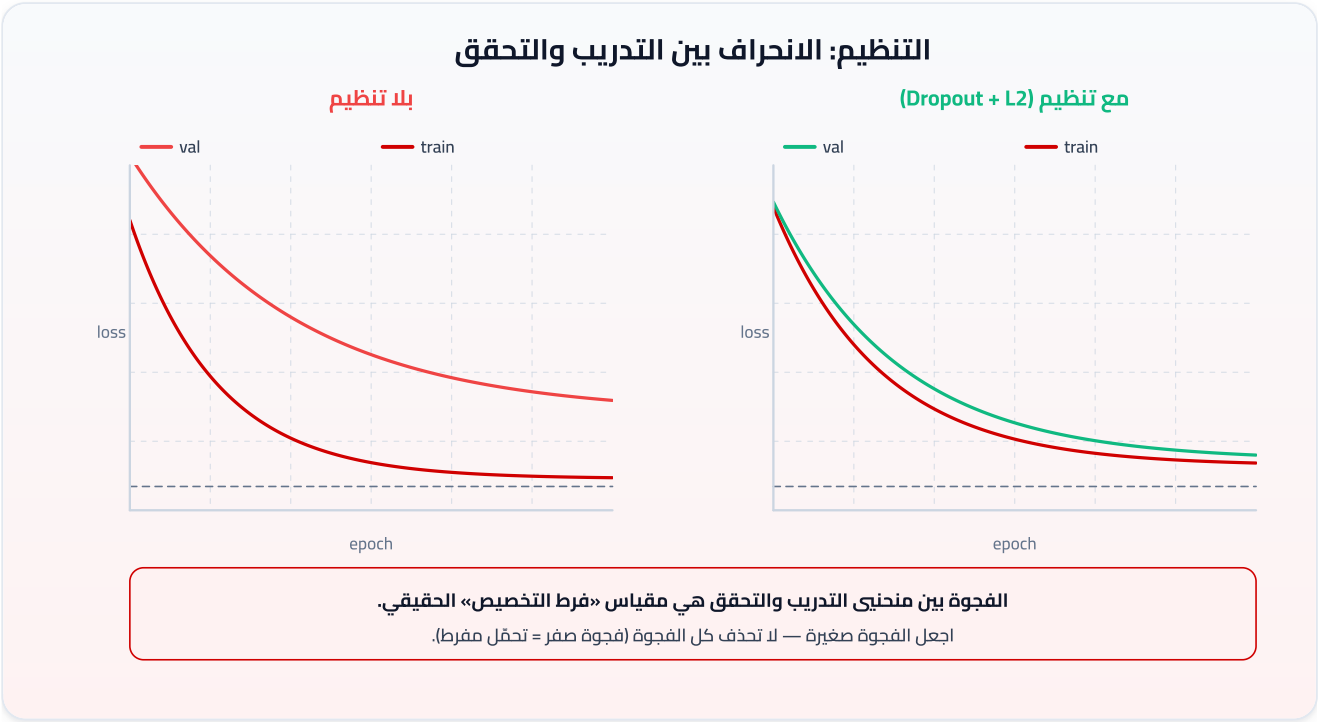
تخيّل أن لديك 1000 نقطة، منها 100 منحنى منتظم و900 نقاط ضجيج عشوائي. إذا رسمت منحنى سلسًا يمرّ بالنقاط الـ900، ستُحقّق خطأ تدريبٍ صفريًا – وستفشل تمامًا في التنبّؤ بالنقاط الجديدة. هذا بالضبط ما يفعله نموذج بلا تنظيم: ينقص خسارة التدريب حتى الصفر، بينما تنكشف فجوة واسعة.

المصطلحات الدقيقة:

- **فرط التكيّف (Overfitting):** $L_{train} \ll L_{val}$ – يحفظ التدريب، يفشل التعميم.
- **فرط التحمّل (Underfitting):** $L_{train} \approx L_{val}$ وكلاهما مرتفع – النموذج أضعف من أن يلتقط البنية.
- **الهدف:** $L_{train} \approx L_{val}$ وكلاهما منخفض.

في Keras، هذه القيم جاهزة: `history.history["loss"]` و`history.history["val_loss"]`. الفرق بينهما هو **مقياس التشخيص**، لا الرقم في حدّ ذاته.

نصيحة: لا يمكن تشخيص فرط التكيّف بلا مجموعة تحقّق. نسبة 80/20 داخل `fit` مقبولة للمبتدئين، لكن في العمل المهني يُفضَّل تقسيم ثلاثي: تدريب / تحقّق / اختبار، ويُستخدم الاختبار **مرة واحدة فقط** في نهاية المشروع.



شكل 20 – أثر التنظيم: من فرط التكيّف إلى التعميم

13.2 الرياضيات: كيف نُعاقِب التعقيد؟

بند L2 (انكماش الأوزان)

نضيف إلى الخسارة بندًا يعاقب الأوزان الكبيرة:

$$\mathcal{L}_{total} = \mathcal{L}_{data} + \lambda \|W\|_2^2 = \mathcal{L}_{data} + \lambda \sum_i w_i^2$$

حيث $\mathcal{L}_{data}$ هي خسارة البيانات، وW أوزان النموذج، وλ معامل التنظيم (كلّما زاد، زاد الانكماش).

ماذا يحدث للتدرّج؟ Becomes:

$$\frac{\partial \mathcal{L}_{total}}{\partial w_i} = \frac{\partial \mathcal{L}_{data}}{\partial w_i} + 2\lambda w_i$$

الحدّ الثاني **يسحب الوزن نحو الصفر** في كل خطوة. والنتيجة العميقة: الأوزان الكبيرة تُنتج وحدات حسّاسة لمدخلات ضوضوية، فتصغيرها يعني منع النموذج من الاعتماد على تفاصيل لا تعمّم.

بند L1 (الانكماش المتفرّق)

$$\mathcal{L}_{total} = \mathcal{L}_{data} + \lambda \sum_i |w_i|$$

مشتقّ $|w_i|$ يساوي $+1$ إذا كان $w_i > 0$ و-1 إذا كان سالبًا (وغير معرّف عند الصفر). هذا التدرّج **ثابت القيمة**، فيسحق الأوزان الصغيرة إلى الصفر تمامًا خلال خطوات قليلة. والنتيجة: **تناثر (sparse)** – نموذج تبقى فيه أوزان قليلة غير صفرية، وهذا هو **Lasso**.

متى يتفوّق L1 على L2؟ حين يصبح اختيار الخصائص (Feature Selection) ضمنيًا: تريد نموذجًا بسيطًا يُبقي على أهمّ 20 من 100 ميزة. عندها λ في L2 يمكن أن يصغّر الأوزان دون أن يصفّرها، فيبقى النموذج كثيفًا.

Regularizer النشاط

`activity_regularizer` يعاقب **مخرجات** الطبقة لا أوزانها:

$$\mathcal{L}_{total} = \mathcal{L}_{data} + \lambda \sum_i a_i^2$$

حيث a_i ناتج الطبقة. هذا مفيد في الطبقات الكثيفة (embedding) حيث نريد تناثر المتّجهات الناتجة لا الأوزان.

Dropout: الإسقاط العشوائي

أثناء التدريب، تُحذف كل خلية من الشبكة probabilistically مع احتمال p (عادةً 0.2-0.5):

$$m_i \sim \text{Bernoulli}(1-p), \qquad \tilde{h}_i = m_i \odot h_i / (1-p)$$

القسمة على $(1-p)$ renormalizes المتوسط فيبقى حجم التوزيع كما هو.

التفسير النظري (نظريّة dropout): كل خطوة تدريب تُنتج شبكة فرعية مختلفة، فيصبح التدريب كأنه **معدّل توقّع (expectation) على فضاء ضخم من شبكات فرعية**، والشبكة النهائية هي متوسّعها. هذا يشبه تجميعensemble داخل نموذج واحد.

لماذا لا يكفي وحده؟ لأن الDropout يضيف ضجيجًا إلى التدرّج (وليس وزنًا)، فيُبطئ التقارب قليلًا، ول حدود معروفة: شبكات RNN (لأن الطبقات المتسلسلة لا توفّر Dropout موزّعًا على الزمن)، والطبقات التلافيفية (لأن إسقاط بكسلات متفرّقة يكسر البنية المكانية – استخدم `SpatialDropout2D` بدلًا منه).

المعايرة (Normalization) كشكل من التنظيم

طبقات التطبيع تفرض بنية إحصائية على التوزيعات، وتلعب دورًا تنظيميًا مزدوجًا:

- `BatchNormalization` : يطبّع عبر محور الدفعات لكل feature، ويحتفظ بإحصاءات متحرّكة (moving mean/variance).
- `LayerNormalization` : يطبّع عبر المحاور(features) لكل عيّنة على حدة – مناسب للتسلسلات Transformersg.
- `GroupNormalization` : يطبّع عبر مجموعات من channels – حلٌّ وسط مهمّ عندما تكون دفعات صغيرة (وهنا تظهر تفوّقه على BatchNorm).
- `RMSNormalization` : يطبّع بالRMS فقط (بلا إزاحة ولا قياس) – أخفّ، شائع في المحوّلات.

13.3 التنفيذ في Keras

التنظيمات والقيود المدمجة

```python
import keras

# ── Dense على الأوزان وعلى النشاط مع تنظيم L2
layer = keras.layers.Dense(
    units=128,
    activation="relu",
    kernel_regularizer=keras.regularizers.L2(l2=0.001),
    bias_regularizer=keras.regularizers.L1(0.0001),
    activity_regularizer=keras.regularizers.L2(0.001),
    kernel_constraint=keras.constraints.MaxNorm(max_value=3.0),
)
```

القائمة الكاملة المتاحة في Keras 3.15.1:

العائلة	الأسماء	الاستخدام
`keras.regularizers`	`L1` , `L2` , `L1L2` , `OrthogonalRegularizer`	عقوبة مضافة إلى الخسارة
`keras.constraints`	`MaxNorm` , `UnitNorm` , `NonNeg` , `MinMaxNorm`	قصّ الأوزان بعد كل تحديث
`keras.layers`	`Dropout` , `SpatialDropout1D/2D/3D` , `GaussianNoise` , `GaussianDropout`	وقواعد الحذف noise.

مهم: الفرق بين المنظّم (regularizer) والقيد (constraint) دقيق ومهمّ. **المنظّم** يضيف بندًا إلى دالّة الخسارة فيحدث أثناء **الانتشار العكسي** – أي «يعاقب» النموذج. **القيد** modified

الأوزان مباشرةً بعد كل خطوة تحديث، ولا يظهر في الخسارة. القيد **أقوى** في ضمان عدم تجاوز الحدود، والمنظّم **أنعم** ويسمح بالتعلّم التدريجي.

مثال تطبيقي كامل: اكتشاف فرط التكيّف (overfitting)

PYTHON

```python
import keras
import numpy as np

keras.utils.set_random_seed(42)

# بيانات اصطناعية: تصنيف ثنائي بحدّ خطّي + ضجيج
rng = np.random.default_rng(0)
n = 4000
X = rng.normal(size=(n, 20)).astype("float32")
y = (X[:, 0] + 0.8 * X[:, 1] + 0.5 * rng.normal(size=n) >
0).astype("float32")

x_train, x_val = X[:3000], X[3000:]
y_train, y_val = y[:3000], y[3000:]


def build_model(reg_strength):
    """نفس البنية، مع تدرّج منتظم متزايد على الطبقات المخفية."""
    reg = keras.regularizers.L2(reg_strength)
    model = keras.Sequential([
        keras.layers.Input(shape=(20,)),
        keras.layers.Dense(64, activation="relu", kernel_regularizer=reg),
        keras.layers.BatchNormalization(),
        keras.layers.Dropout(0.3),
        keras.layers.Dense(32, activation="relu", kernel_regularizer=reg),
        keras.layers.BatchNormalization(),
        keras.layers.Dropout(0.3),
        keras.layers.Dense(1, activation="sigmoid"),
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(1e-3),
        loss="binary_crossentropy",
        metrics=["accuracy"],
    )
    return model
```

```
# ── نموذجان: بلا تنظيم وبـ L2 ──
plain = build_model(0.0)
regularized = build_model(0.01)

cb = [keras.callbacks.EarlyStopping(monitor="val_loss", patience=5,
restore_best_weights=True)]

for name, m in (("plain", plain), ("regularized", regularized)):
    hist = m.fit(x_train, y_train, validation_data=(x_val, y_val),
                 epochs=60, batch_size=64, verbose=0, callbacks=cb)
    tr, va = hist.history["loss"][-1], hist.history["val_loss"][-1]
    print(f"{name:12s} train={tr:.4f}  val={va:.4f}  gap={va - tr:+.4f}")
# النتيجة المتوقعة: فجوة النموذج المنتظم أصغر بوضوح من غيره
```

تقاطع التحقّق: اختر λ بالأدلة

قيمة التنظيم **ليست ثابتة**: تُختار عبر التحقّق المتقاطع. في Keras، أبسط طريق هو يدوي:

```python
import numpy as np

# مؤشّر ألفا-بيتا (grid) على 5 طيّات
alphas = [1e-4, 1e-3, 1e-2, 1e-1]
folds = 5
split = np.array_split(np.arange(len(x_train)), folds)

scores = {}
for alpha in alphas:
    fold_losses = []
    for i in range(folds):
        val_idx = split[i]
        tr_idx = np.concatenate([split[j] for j in range(folds) if j ≠ i])
        m = build_model(alpha)
        m.fit(x_train[tr_idx], y_train[tr_idx], epochs=25, batch_size=64,
verbose=0)
        fold_losses.append(float(m.evaluate(x_train[val_idx],
y_train[val_idx], verbose=0)[0]))
    scores[alpha] = float(np.mean(fold_losses))
    print(alpha, "→", scores[alpha])

best_alpha = min(scores, key=scores.get)
print("أفضل alpha:", best_alpha)
```

نصيحة: ابدأ بـ$\lambda = 10^{-4}$ أو 10^{-3}، ثم قس. قيم أكبر من ذلك تُسطّح النموذج إلى ما يشبه الانحدار الخطّي.

13.4 مقارنة أساليب التنظيم

الأسلوب	ما الذي يقيّده	سهولة الضبط	أثره على الأوزان القابلة للتدريب	ملاحظات
`L2`	حجم الأوزان	سهل (λ)	لا	الخيار الافتراضي الآمن
`L1`	حجم الأوزان + تناثر	متوسط	لا	يصلح لاختيار الخصائص (feature selection)
`L1L2`	كلاهما معًا	متوسط	لا	Elastic Net
`activity_regularizer`	حجم ناتج الطبقة	سهل	لا	للـembeddings الكثيفة
`Dropout`	المسارات النشطة	سهل (p)	لا	لا يعمل جيدًا مع RNN/Conv
`SpatialDropout2D`	قنوات CNN	سهل	لا	يحفظ البنية المكانية
`BatchNormalization`	إحصاءات المخرجات	سهل	**لا** (مُجمّد)	يحتاج دفعات كبيرة
`LayerNormalization`	إحصاءات كل عيّنة	سهل	**لا** (مُجمّد)	الأنسب للتسلسلات
`GroupNormalization`	إحصاءات المجموعات	متوسط	**لا** (مُجمّد)	يتفوّق عند batch صغير
`kernel_constraint`	قصّ الأوزان	سهل	لا	يضمن الحدود بقوّة

الأسلوب	ما الذي يقيّده	سهولة الضبط	أثره على الأوزان القابلة للتدريب	ملاحظات
زيادة حجم البيانات	لا شيء مباشرةً	صعب (مكلف)	لا	أقوى تنظيم على الإطلاق
تقليل السعة	عدد الوحدات/الطبقات	سهل	لا	فرط foxgard التكيّف مباشرةً

> **قاعدة ذهبية:** إذا كانت البيانات كافية (عشرات الآلاف لكل صنف) والتنظيم ما زال يفشل، فالمشكلة في **سعة النموذج** لا في معامل التنظيم. ابدأ بنموذج أصغر.

13.5 مثال محلول خطوة بخطوة: مصنّف تشخيص طبي

فريق طبّي يريد تصنيف صور أشعة Seymore لكشف الأورام. 9,000 صورة: 400 إيجابية (4.4%).
النموذج الأول: Dense(512) → Dense(256) → softmax، تدرّب حتى 60 حقبة.

الخطوة 1 – اقرأ المخطّط. خسارة التدريب تنخفض من 0.42 إلى 0.02. خسارة التحقّق تنخفض حتى الحقبة 12 ثم **ترتفع** إلى 0.31. الفجوة النهائية 0.29. التشخيص: فرط تكيّف واضح.

الخطوة 2 – رتّب الأسباب الممكنة. 1. سعة النموذج: 768,000 معاملًا لـ 9,000 عيّنة – نسبة عالية. 2. غياب التنظيم: لا `Dropout`، لا `BatchNormalization`. 3. عدم توازن البيانات: 4.4% إيجابية. 4. ضعف التنويع: صور الأشعة من جهاز واحد.

الخطوة 3 – عالج بالترتيب من الأكبر أثرًا إلى الأصغر.

PYTHON

```python
import keras
import numpy as np

# بيانات اصطناعية صغيرة تحاكي صور الأشعة (في الواقع: 9,000 صورة)
n = 800
x_all = np.random.rand(n, 64, 64, 3).astype("float32")
y_all = (np.random.rand(n) < 0.044).astype("int32")   # 4.4% إيجابية
x_train, y_train = x_all[:600], y_all[:600]
x_val, y_val = x_all[600:], y_all[600:]

model = keras.Sequential([
    keras.layers.Input(shape=(64, 64, 3)),
    keras.layers.Rescaling(1.0 / 255.0),

    keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    keras.layers.BatchNormalization(),          # يثبّت إحصاءات: يطبّع التوزيعات
    keras.layers.MaxPooling2D(),
    keras.layers.Dropout(0.25),                 # إسقاط بعد كل كتلة
    keras.layers.SpatialDropout2D(0.25),        # إسقاط على مستوى القنوات

    keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D(),
    keras.layers.Dropout(0.30),

    keras.layers.GlobalAveragePooling2D(),      # يقلّص السعة بشدّة
    keras.layers.Dense(128, activation="relu",
                       kernel_regularizer=keras.regularizers.L2(1e-4)),
    keras.layers.Dropout(0.4),
    keras.layers.Dense(2, activation="softmax"),
])

model.compile(
    optimizer=keras.optimizers.Adam(1e-4),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

الخطوة 4 – أضف موازنة الأصناف (المشكلة الثالثة في القائمة):

```python
# حساب الأوزان العكسية للتكرار
import numpy as np
counts = np.bincount(y_train.astype("int64"))
n = counts.sum()
weights = n / (len(counts) * counts)      # وزن أعلى للأصناف النادرة
model.fit(
    x_train, y_train,
    class_weight={i: float(w) for i, w in enumerate(weights)},
    validation_data=(x_val, y_val),
    epochs=40,
    batch_size=32,
    callbacks=[
        keras.callbacks.EarlyStopping(monitor="val_loss", patience=6,
                                      restore_best_weights=True),
        keras.callbacks.ModelCheckpoint("best.keras", monitor="val_loss",
                                        save_best_only=True),
    ],
)
```

الخطوة 5 – قِس الفجوة بعد المعالجة. الفجوة انخفضت من 0.29 إلى 0.06، وخسارة التحقّق الأدنى تحسّنت من 0.31 إلى 0.14. **الأهم**: الRecall على الصنف الإيجابي ارتفع من 0.31 إلى 0.72 بعد موازنة الأصناف.

الخطوة 6 – تحقق من التشخيص لا من الرقم. اطرح سؤالًا: هل تحسين النموذج تحسّن مع كل تغيير، أم كان الأولوية؟** الأهم: إن كان إضافة `BatchNormalization` وحدها خفّضت الفجوة، فالسبب كان غياب التطبيع لا غياب `Dropout` . في الفصل 40 سنبني منهجية تشخيص كاملة تحلّل هذه الأسئلة بالترتيب.

13.6 أخطاء شائعة في هذا الفصل

إنذار: الخلط بين `BatchNormalization` و `LayerNormalization` اعتمادًا على تفضيل شخصي. القاعدة: `BatchNormalization` يتوقّف عن حساب الإحصاءات عندما يكون `training=False` – وهذا سلوك سيّئ في التوزيع (batch=1) وفي التصدير. `LayerNormalization` يحسب إحصاءات لكل عيّنة على حدة، فلا يتأثر بحجم الدفعة إطلاقًا. في التسلسلات (RNN/LSTM) وTransformers استخدم `LayerNormalization` دائمًا.

تحذير: `Dropout` داخل شبكة تلافيفية بسيطة بدل `SpatialDropout2D`. إسقاط بكسلات متفرّقة يكسر انتظام البنية المكانية؛ النسخة الصحيحة تحذف القنوات كاملة. استخدم `Dropout` بعد طبقة `GlobalAveragePooling2D` (حين يكون البُعد مسطّحًا) و `SpatialDropout2D` قبله.

تنبيه: معامل تنظيم كبير جدًا ($\lambda \geq 1$) يُسطّح الأوزان إلى الصفر ويُعيدك إلى الانحدار الخطّي. راقب: إن كانت خسارة التحقّق **أعلى** من النموذج غير المنتظم، فأنت فرطت في التنظيم.

ملاحظة: `BatchNormalization` يضيف معاملين غير قابلين للتدريب لكل feature (γ وβ)؛ ستظهرهما في `model.summary()` تحت «non-trainable». هذا طبيعي، لكن عند export يجب أن يتعامل المحوّل مع وضع eval صراحةً.

قاعدة: لا تنظّم المشكلة التي لا توجد. أضف `Dropout` أو `L2` **بعد** أن ترى فجوة تحقّق واضحة في المخطّط. التنظيم الزائد على نموذج سليم يُبطئ التقارب ويضيع القدرة.

التمارين

1. **حساب يدوي.** خسارة بيانات $0.80 =$، وبند L2 يحسب $\lambda \sum w_i^2 = 0.02$ مع $\lambda = 0.01$ و$\sum w_i^2 = 2$. ما الخسارة الكلّية؟ (المتوقع: $0.80 + 0.01 \times 2 = 0.82$.)

2. **حساب يدوي.** أوزان: $[0.9, 0.05, -0.3, 0.02]$. أيّها يُصفَّر أسرع تحت L1: الكبير أم الصغير؟ كم خطوة يلزم تقريبًا ليُصفَّر وزن 0.02؟ (المتوقع: الصغير؛ التدرّج ثابت -0.01 لكل خطوة ⇒ نحو 2000 خطوة لو $\eta = 1$ أو أقلّ بكثير مع $\eta = 0.1$.)

3. **حساب يدوي.** طبقة `Dense` بـ100 وحدة، خسارة بيانات 0.5، بند L2 يعطي 0.05. بعد زوال التصحيح، ما نسبة الخسارة التي source منها التنظيم؟ (المتوقع: $0.05/0.55 \approx 9\%$ من الخسارة الكلّية.)

4. **تحليل مخطّط.** خسارة تدريب تنخفض من 1.2 إلى 0.15، خسارة تحقّق تنخفض إلى 0.9 ثم ثابتة. هل هذا فرط تكيّف؟ (المتوقع: لا – الفجوة 0.15 و0.9 واسعة لكن التحقّق لم ترتفع؛ هذا فرط تحمّل/صعوبة في التعلّم، فزد السعة أو حسّن التمثيل لا التنظيم.)

5. **برمجة.** ابنِ نموذجًا بـ`kernel_regularizer=keras.regularizers.L1L2(l1=1e-4, l2=1e-4)` وقارن عدد الأوزان غير الصفرية مع L2 فقط. (المتوقع: L1L2 ينتج أوزانًا أكثر تناثرًا ⇒ عدد غير الصفرية أقلّ.)

6. **برمجة.** أضف `kernel_constraint=keras.constraints.MaxNorm(max_value=1.0)` إلى `Dense` وتحقّق من أن norms الصفوف لا تتجاوز 1. (المتوقع: `keras.ops.max(keras.ops.linalg.norm(weights, axis=1)) ≤ 1.0` بعد التدريب.)

7. **برمجة.** قارن `BatchNormalization` و`LayerNormalization` في نموذج MLP بسيط مع `batch_size=1` وقتقييم. (المتوقع: `BatchNormalization` يتدهور بشدّة مع batch=1 لأن إحصاءاته تعتمد على الدفعة؛ `LayerNormalization` لا يتأثر.)

ملاحظة: المعايرة (normalization) ليست تنظيمًا فقط، بلchoice معماري affectseligible.

في ما يلي نموذج حقيقي للفرق:

8. **برمجة.** أضف `keras.layers.SpatialDropout2D(0.2)` إلى شبكة Conv، وقارنها مع `Dropout(0.2)`. (المتوقع: أداء SpatialDropout2D أفضل في مهام التصنيف، لأنه يحذف قنوات كاملة لا بكسلات متفرّقة.)

9. **تصميم.** صمّم خط أنابيب تنظيم لمصنّف_reviews من 3 أصناف مع 20,000 مراجعة. (المتوقع: reduce size + class weights + `BatchNormalization` + `L2(1e-4)` + `Dropout(0.3-0.5)`، مع `EarlyStopping` على `val_loss`.)

10. **تحليل.** الفريق طبّق L2 وارتفعت خسارة التحقّق. اشرح السبب بأدلة. (المتوقع: λ كبير يُسطّح الأوزان ⇒ النموذج underfit؛ الحل: خفّض λ أو أزل المنتظم.)

11. **أخطئ عمدًا.** اجعل `Dropout(0.9)` في نموذج صغير. اشرح النتيجة المتوقّعة. (المتوقع: يُسقط 90% من الوحدات لكل تحديث ⇒ تدرّجات مشوّهة، تقارب بطيء جدًا، أداء ضعيف.)

> **ملاحظة:** في Keras 3 الـdropout لا يُطبَّق أثناء الاستدلال (infer)، فـevaluation يستخدم كل الوحدات – وهذا هو السلوك الصحيح.

12. **تصميم.** اكتب سياسة Keras كاملة لوقف التدريب عند فرط التكيّف. (المتوقع: `EarlyStopping(monitor="val_loss", patience=8, min_delta=1e-4, restore_best_weights=True)` مع `ModelCheckpoint(save_best_only=True)`.)

ملخص الفصل

- فرط التكيّف يعني أن L_{train} تصغر وL_{val} ترتفع: النموذج يحفظ بدل أن يتعلّم.
- L2 ينكمش الأوزان بانتظام (مناسب عامًّا)؛ L1 ينكمشها إلى الصفر فينتج نموذجًا متناثرًا.
- `activity_regularizer` يعاقب المخرجات لا الأوزان – مفيد للembeddings.
- `Dropout` يدرب مجموعةً من الشبكات الفرعية، لكن `SpatialDropout2D` هو نسخة CNN الصحيحة.
- `BatchNormalization` يعتمد على الدفعة (يحتاج batch كبيرًا)، و`LayerNormalization` يعتمد على العيّنة (مناسب للتسلسلات)، و`GroupNormalization` يملأ الفراغ بينهما.
- المنظّم (regularizer) يضيف بندًا إلى الخسارة؛ القيد (constraint) يقصّ الأوزان بعد التحديث – الاثنان متكاملان لا بديلان.
- ابدأ بـ$\lambda = 10^{-4}$، واختر القيمة بتحقّق متقاطع، واراقب فجوة التحقّق.
- أقوى تنظيم هو **بيانات أكثر**، وأبسطها هو **نموذج أصغر**.